

Data Mining Project - Group 42

1. Customers Dataset

Importing, Exploring and Pre-processing

1.1. Import the needed libraries

```
In [146... # Importing the needed libraries
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
from math import ceil
from sklearn.preprocessing import MinMaxScaler
```

1.2. Import the dataset

```
In [147... customers = pd.read_csv('DM_AIAI_CustomerDB.csv')
```

1.3. Exploratory Data Analysis

In this section, we will explore the structure, quality, and distribution of the data in the **customers dataset**.

We will use descriptive statistics and visualizations to better understand patterns and potential relationships between variables.

```
In [148... customers.head()
```

Out[148...

	Unnamed: 0	Loyalty#	First Name	Last Name	Customer Name	Country	Province or State	
0	0	480934	Cecilia	Householder	Cecilia Householder	Canada	Ontario	
1	1	549612	Dayle	Menez	Dayle Menez	Canada	Alberta	E
2	2	429460	Necole	Hannon	Necole Hannon	Canada	British Columbia	V
3	3	608370	Queen	Hagee	Queen Hagee	Canada	Ontario	
4	4	530508	Claire	Latting	Claire Latting	Canada	Quebec	

In [149...

```
customers.shape
```

Out[149...

```
(16921, 21)
```

The costumers dataset consists of 16921 observations and 21 variables.

In [150...

```
customers.columns
```

Out[150...

```
Index(['Unnamed: 0', 'Loyalty#', 'First Name', 'Last Name', 'Customer Name',  
      'Country', 'Province or State', 'City', 'Latitude', 'Longitude',  
      'Postal code', 'Gender', 'Education', 'Location Code', 'Income',  
      'Marital Status', 'LoyaltyStatus', 'EnrollmentDateOpening',  
      'CancellationDate', 'Customer Lifetime Value', 'EnrollmentType'],  
      dtype='object')
```

In [151...

```
# Checking the type and non-null values (not missing values) of each variable  
customers.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 16921 entries, 0 to 16920
Data columns (total 21 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Unnamed: 0                            16921 non-null  int64
1   Loyalty#                              16921 non-null  int64
2   First Name                            16921 non-null  object
3   Last Name                             16921 non-null  object
4   Customer Name                         16921 non-null  object
5   Country                               16921 non-null  object
6   Province or State                     16921 non-null  object
7   City                                  16921 non-null  object
8   Latitude                              16921 non-null  float64
9   Longitude                             16921 non-null  float64
10  Postal code                           16921 non-null  object
11  Gender                                16921 non-null  object
12  Education                             16921 non-null  object
13  Location Code                         16921 non-null  object
14  Income                                16901 non-null  float64
15  Marital Status                        16921 non-null  object
16  LoyaltyStatus                         16921 non-null  object
17  EnrollmentDateOpening                 16921 non-null  object
18  CancellationDate                      2310 non-null   object
19  Customer Lifetime Value               16901 non-null  float64
20  EnrollmentType                        16921 non-null  object
dtypes: float64(4), int64(2), object(15)
memory usage: 2.7+ MB

```

- Here, besides checking the type of each variable, we can already check how many missing values each variable has.
- The only variables who seem to have **missing values** are "Income" (20 NaN's), "Customer Lifetime Value" (20 NaN's) and "CancellationDate" (14611 NaN's) - We will treat this missing values later

```

In [152... # Checking only the type of each variable
customers.dtypes

```

```
Out[152... Unnamed: 0          int64
Loyalty#          int64
First Name        object
Last Name         object
Customer Name     object
Country           object
Province or State object
City             object
Latitude          float64
Longitude         float64
Postal code       object
Gender            object
Education         object
Location Code     object
Income           float64
Marital Status    object
LoyaltyStatus     object
EnrollmentDateOpening object
CancellationDate  object
Customer Lifetime Value float64
EnrollmentType   object
dtype: object
```

- We can already conclude that the variables "EnrollmentDateOpening" and "CancellationDate" should be ***datetime***, so we are going to need to convert them later.

```
In [153... customers.describe(include="all").T
```

Out[153...

	count	unique	top	freq	mean	
Unnamed: 0	16921.0	NaN	NaN	NaN	8440.023639	4
Loyalty#	16921.0	NaN	NaN	NaN	550197.393771	259
First Name	16921	4941	Stacey	13	NaN	
Last Name	16921	15404	Ypina	4	NaN	
Customer Name	16921	16921	Emma Martin	1	NaN	
Country	16921	1	Canada	16921	NaN	
Province or State	16921	11	Ontario	5468	NaN	
City	16921	29	Toronto	3390	NaN	
Latitude	16921.0	NaN	NaN	NaN	47.1745	
Longitude	16921.0	NaN	NaN	NaN	-91.814768	
Postal code	16921	75	V6E 3D9	917	NaN	
Gender	16921	2	female	8497	NaN	
Education	16921	5	Bachelor	10586	NaN	
Location Code	16921	3	Suburban	5716	NaN	
Income	16901.0	NaN	NaN	NaN	37758.0384	30
Marital Status	16921	3	Married	9842	NaN	
LoyaltyStatus	16921	3	Star	7761	NaN	
EnrollmentDateOpening	16921	2449	4/3/2015	34	NaN	
CancellationDate	2310	1260	7/7/2020	8	NaN	
Customer Lifetime Value	16901.0	NaN	NaN	NaN	7990.460188	6
EnrollmentType	16921	2	Standard	15773	NaN	

- After this we will check these statistics for specific variables, so it's easier to see, but, from this we can conclude that the variable **"Unnamed:0" will be dropped** - This variable has no analytical meaning, it simply represents the row number (index) and doesn't provide any useful information about customers or their behavior.

In [154...

```
# Filtering numeric variables so we can easier see their statistics
# We are only doing this for the variables that are more relevant
# (it doesn't make sense to do this for the variables "Unnamed" and "Loyalty
customers[['Income', 'Customer Lifetime Value']].describe()
```

Out[154...

	Income	Customer Lifetime Value
count	16901.000000	16901.000000
mean	37758.038400	7990.460188
std	30368.992499	6863.173093
min	0.000000	1898.010000
25%	0.000000	3979.720000
50%	34161.000000	5780.180000
75%	62396.000000	8945.690000
max	99981.000000	83325.380000

- These statistics are useful to check, for example, if the variables have outliers
- "Income": The presence of both 0 and very high values indicates **potential outliers** or an asymmetric (right-skewed) distribution.
- Also, the mean income is around 37758, but the standard deviation is very high (around 30368), which shows large variability among customers.
- "Customer Lifetime Value": The minimum value (around 1898) and maximum (around 83325) are also kind of far apart, which shows potential outliers and right-skewness.

In [155...

```
# Basic statistics for categorical variables
customers.describe(include="object")
```

Out[155...

	First Name	Last Name	Customer Name	Country	Province or State	City	Postal code	Gender
count	16921	16921	16921	16921	16921	16921	16921	16921
unique	4941	15404	16921	1	11	29	75	2
top	Stacey	Ypina	Emma Martin	Canada	Ontario	Toronto	V6E 3D9	female
freq	13	4	1	16921	5468	3390	917	8497

- We can already conclude that the variable **"Country" will be dropped** (since the customers seem to be all from Canada: unique=1)
- The variables **"First Name", "Last Name" and "Customer Name" might also be dropped** (they aren't really relevant since we have the variable "Loyalty#" to identify each customer)

1.3.1. Checking for Missing Values

Here we are only going to check if there are any missing values in the Customers Dataset, we are not going to treat them yet.

```
In [156... customers.isna().sum()
```

```
Out[156... Unnamed: 0          0
Loyalty#         0
First Name       0
Last Name        0
Customer Name    0
Country          0
Province or State 0
City             0
Latitude         0
Longitude        0
Postal code      0
Gender           0
Education        0
Location Code    0
Income           20
Marital Status   0
LoyaltyStatus    0
EnrollmentDateOpening 0
CancellationDate 14611
Customer Lifetime Value 20
EnrollmentType  0
dtype: int64
```

- As we have checked before (in .info), we only have missing values in the variables "Income" (20 NaN's), "Customer Lifetime Value" (20 NaN's) and "CancellationDate" (14611 NaN's).
- "CancellationDate": We know that the total number of rows is 16921. Since the number of missing values in this variable is 14611, we can conclude that **2310 people left the program.**

```
In [157... # Check if the rows with missing values in "Income" are the same rows with n
missing_values_income = customers['Income'].isna()
missing_values_clv = customers['Customer Lifetime Value'].isna()

# Compare the two boolean masks
(missing_values_income == missing_values_clv).all()
```

```
Out[157... np.True_
```

Since the output is "True", we can assume that "Income" and "Customer Lifetime Value" have the same missing values.

This might mean that ?????

```
In [158... # Checking the rows where both Income and Customer Lifetime Value are NaN  
customers[customers['Income'].isna() & customers['Customer Lifetime Value']].
```

	Unnamed: 0	Loyalty#	First Name	Last Name	Customer Name	Country	Province or State
16901	0	999987	Layla	Murphy	Layla Murphy	Canada	New Brunswick
16902	1	999988	Jana	Parker	Jana Parker	Canada	Quebec
16903	2	999989	Ethan	Parker	Ethan Parker	Canada	Ontario
16904	3	999990	Ryan	Anderson	Ryan Anderson	Canada	New Brunswick
16905	4	999991	Olivia	Cote	Olivia Cote	Canada	New Brunswick
16906	5	999992	Ella	Roy	Ella Roy	Canada	Ontario
16907	6	999993	Elijah	Cook	Elijah Cook	Canada	British Columbia
16908	7	999994	Ethan	Chan	Ethan Chan	Canada	Ontario
16909	8	999995	Liam	Wong	Liam Wong	Canada	Ontario
16910	9	999996	Isabella	Ross	Isabella Ross	Canada	Ontario
16911	10	999997	Grace	Miller	Grace Miller	Canada	New Brunswick
16912	11	999998	Noah	Thompson	Noah Thompson	Canada	Quebec
16913	12	999999	Noah	Walker	Noah Walker	Canada	Alberta
16914	13	100017	Grace	Fortin	Grace Fortin	Canada	British Columbia
16915	14	100011	Amelia	Ross	Amelia Ross	Canada	Ontario
16916	15	100012	Ethan	Thompson	Ethan Thompson	Canada	Quebec
16917	16	100013	Layla	Young	Layla Young	Canada	Alberta
16918	17	100014	Amelia	Bennett	Amelia Bennett	Canada	New Brunswick
16919	18	100015	Benjamin	Wilson	Benjamin Wilson	Canada	Quebec
16920	19	100016	Emma	Martin	Emma Martin	Canada	British Columbia

1.3.2. Checking for Duplicated Values

```
In [159... customers.duplicated().sum()
```

```
Out[159... np.int64(0)
```

- We can conclude that in the Customers Dataset there aren't any rows that are duplicated.
- But, even though we don't have rows that are completely equal to other rows, we might still have some duplication errors.
- Since Loyalty# should be a unique customer identifier it is important to check if the **same Loyalty# appears with different customer names**.

```
In [160... # Checking for duplicated Loyalty# with different names
duplicates = customers.groupby('Loyalty#')['Customer Name'].nunique()
duplicates[duplicates > 1]
```

```
Out[160... Loyalty#
101902      2
106001      2
106509      2
112142      2
114414      2
..
981508      2
988392      2
989528      2
990512      2
992168      2
Name: Customer Name, Length: 163, dtype: int64
```

- Apparently there are 163 duplicated "Loyalty#", this is, **163 customers with the same ID but different names**.
- We need to verify those duplicates in the dataset. If we check that for each one of the duplicates' rows, the values in every variable are the same (and only the ID differs) then we can assume this was just an inconsistency and we can keep the first duplicated row and delete the second one (since they refer to the same customer).
- But, if we check that only the ID is the same, and all the other variables have different values, then we can conclude that they are different customers. In that case, we probably shouldn't delete one of the customers, cause that way we would be losing important information.

??? But for now I am going to keep the first row and delete the others, and after talking to the teacher we decide what to do

```
In [161... # Checking if the duplicated #Loyalty refer different customers or not  
# If for example rows with same ID have live in different countries, or have  
# then we can assume that these are different customers  
  
duplicates_ids = customers[customers.duplicated(subset='Loyalty#', keep=False)  
duplicates_ids.sort_values(['Loyalty#', 'Customer Name']).head(20)
```

Out[161...

	Unnamed: 0	Loyalty#	First Name	Last Name	Customer Name	Country	Province
1646	1646	101902	Hans	Schlottmann	Hans Schlottmann	Canada	Ontario
2668	2668	101902	Yi	Nesti	Yi Nesti	Canada	Ontario
700	700	106001	Ivette	Peifer	Ivette Peifer	Canada	Quebec
15988	15988	106001	Maudie	Hyland	Maudie Hyland	Canada	Brunswick
9413	9413	106509	Ardelia	Whitehorse	Ardelia Whitehorse	Canada	Ontario
13053	13053	106509	Stacy	Schwebke	Stacy Schwebke	Canada	Ontario
3678	3678	112142	Arnold	Cantoran	Arnold Cantoran	Canada	Ontario
9604	9604	112142	Marty	Kijak	Marty Kijak	Canada	Quebec
5253	5253	114414	Kala	Bachor	Kala Bachor	Canada	Ontario
11411	11411	114414	Temple	Resper	Temple Resper	Canada	Ontario
9459	9459	126100	Francie	Tappe	Francie Tappe	Canada	Brunswick
13207	13207	126100	Pattie	Sweigart	Pattie Sweigart	Canada	Quebec
10781	10781	126490	Gennie	Mangone	Gennie Mangone	Canada	Ontario
13287	13287	126490	Vesta	Lanfair	Vesta Lanfair	Canada	Eastern Ontario
4453	4453	130331	Cammie	Pangan	Cammie Pangan	Canada	Quebec
8947	8947	130331	Lucienne	Flythe	Lucienne Flythe	Canada	Ontario
13656	13656	135421	Beverly	Deoliveira	Beverly Deoliveira	Canada	Quebec
3735	3735	135421	Salvatore	Ayyad	Salvatore Ayyad	Canada	Quebec
243	243	156031	Carter	Balette	Carter Balette	Canada	Quebec

Unnamed: 0	Loyalty#	First Name	Last Name	Customer Name	Country	Province
2416	2416	156031	Onie	Grech	Onie Grech	Canada

This shows that the name, gender, income, etc is different for the customers with the same Loyalty#, which means that the error happening error isn't just a copy of the same customer, but it is an error in only the ID (the system gave the same ID to different customers).

```
In [162... # Removing duplicated Loyalty# entries (keeping only the first occurrence)
print("Initial dataset shape:", customers.shape)

customers = customers.drop_duplicates(subset='Loyalty#', keep='first')

print("Dataset shape after removing duplicated Loyalty#:", customers.shape)
```

Initial dataset shape: (16921, 21)

Dataset shape after removing duplicated Loyalty#: (16757, 21)

1.4. Initial Data Preparation

Here, we are only going to **deal with some of the initial inconsistencies**, like deleting variables that aren't relevant for our analysis (and for the models) and also converting variables to their correct data type.

1.4.1. Deleting variables that aren't relevant

- We decided to drop the variable "Unnamed:0", as we wrote before, because it has no analytical meaning. It only represents the row number and doesn't provide any useful information about customers or their behavior. Keeping it would only add noise and take up unnecessary space in the dataset.
- We also decided to drop the variables 'First Name', 'Last Name' and 'Customer Name', since we already have a variable that identifies each customer ('#Loyalty').
- Finally, we decided to drop the variable 'Country', because we checked that all the rows in this column are equal, all the customers are from Canada.

```
In [163... customers = customers.drop(['Unnamed: 0', 'First Name', 'Last Name', 'Customer Name'])
```

```
In [164... # Shows the dataset after deleting the irrelevant variables
display(customers)
```

	Loyalty#	Province or State	City	Latitude	Longitude	Postal code	Gender	I
0	480934	Ontario	Toronto	43.653225	-79.383186	M2Z 4K1	female	
1	549612	Alberta	Edmonton	53.544388	-113.490930	T3G 6Y6	male	
2	429460	British Columbia	Vancouver	49.282730	-123.120740	V6E 3D9	male	
3	608370	Ontario	Toronto	43.653225	-79.383186	P1W 1K4	male	
4	530508	Quebec	Hull	45.428730	-75.713364	J8Y 3Z5	male	
...	...	...	...	...	...	...	...	
16916	100012	Quebec	Quebec City	46.759733	-71.141009	Y0C 7D6	male	
16917	100013	Alberta	Edmonton	53.524829	-113.546357	L3S 9Y3	female	
16918	100014	New Brunswick	Moncton	46.051866	-64.825428	G2S 2B6	male	
16919	100015	Quebec	Quebec City	46.862970	-71.133444	B1Z 8T3	female	
16920	100016	British Columbia	Dawson Creek	55.720562	-120.160090	M4A 1E4	female	

16757 rows × 16 columns

1.4.2. Converting variables to datetime

In the beginning of the exploratory data analysis we noticed that the variables "EnrollmentDateOpening" and "CancellationDate" were of the type object, and they should be datetime. So we are going to convert them now.

```
In [165... customers['EnrollmentDateOpening'] = pd.to_datetime(customers['EnrollmentDateOpening'])
customers['CancellationDate'] = pd.to_datetime(customers['CancellationDate'])
```

```
In [166... # Checking if the type is now correct
customers[['EnrollmentDateOpening', 'CancellationDate']].info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 16757 entries, 0 to 16920
Data columns (total 2 columns):
#   Column                      Non-Null Count  Dtype
---  -
0   EnrollmentDateOpening        16757 non-null  datetime64[ns]
1   CancellationDate             2286 non-null   datetime64[ns]
dtypes: datetime64[ns](2)
memory usage: 392.7 KB
```

1.5. Visual Exploration

1.5.1. Visualization of Numeric Variables

```
In [167... sns.set()

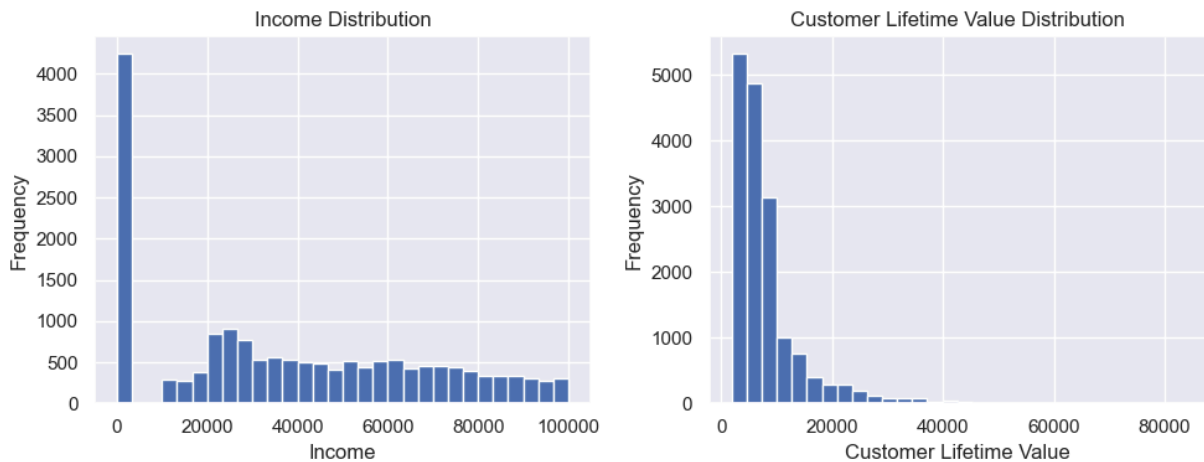
subplot_figsize = (10, 4)

fig, axes = plt.subplots(1, 2, figsize=subplot_figsize)

axes[0].hist(customers['Income'], bins=30)
axes[0].set_title('Income Distribution')
axes[0].set_xlabel('Income')
axes[0].set_ylabel('Frequency')

axes[1].hist(customers['Customer Lifetime Value'], bins=30)
axes[1].set_title('Customer Lifetime Value Distribution')
axes[1].set_xlabel('Customer Lifetime Value')
axes[1].set_ylabel('Frequency')

plt.tight_layout()
plt.show()
```



The histograms show that most customers have low income and a low customer lifetime value, while only a few have very high values. This means the data is not evenly distributed — a small number of customers generate much more value

than the rest. This kind of pattern is common in real customer data and shows why it's important to identify and focus on the most valuable clients.

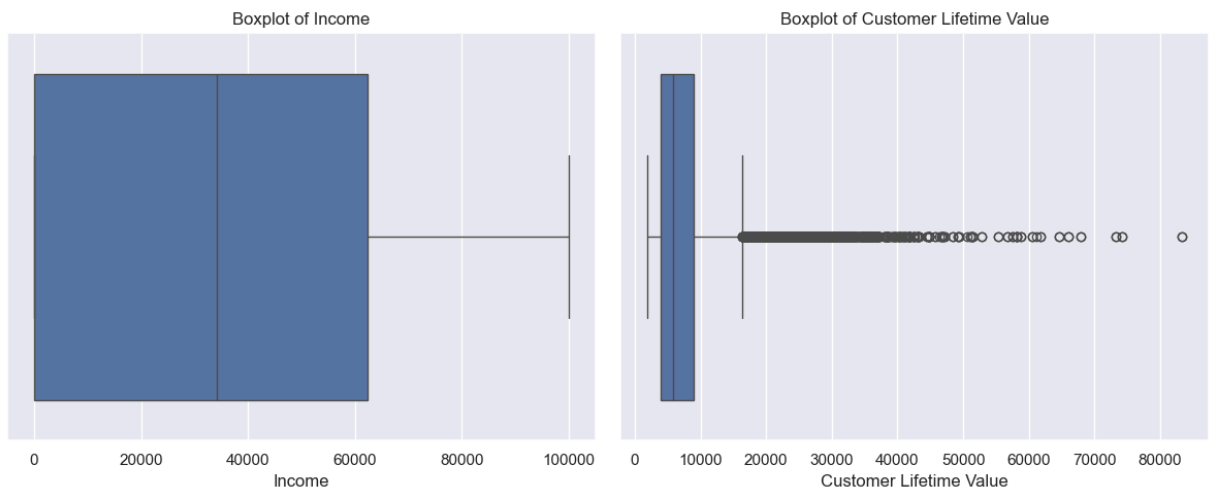
```
In [168... # Comparative boxplots (Income vs Customer Lifetime Value)
sns.set()

fig, axes = plt.subplots(1, 2, figsize=(12, 5), tight_layout=True)

# Boxplot 1 - Income
sns.boxplot(x=customers["Income"], ax=axes[0])
axes[0].set_title("Boxplot of Income")

# Boxplot 2 - Customer Lifetime Value
sns.boxplot(x=customers["Customer Lifetime Value"], ax=axes[1])
axes[1].set_title("Boxplot of Customer Lifetime Value")

plt.show()
```



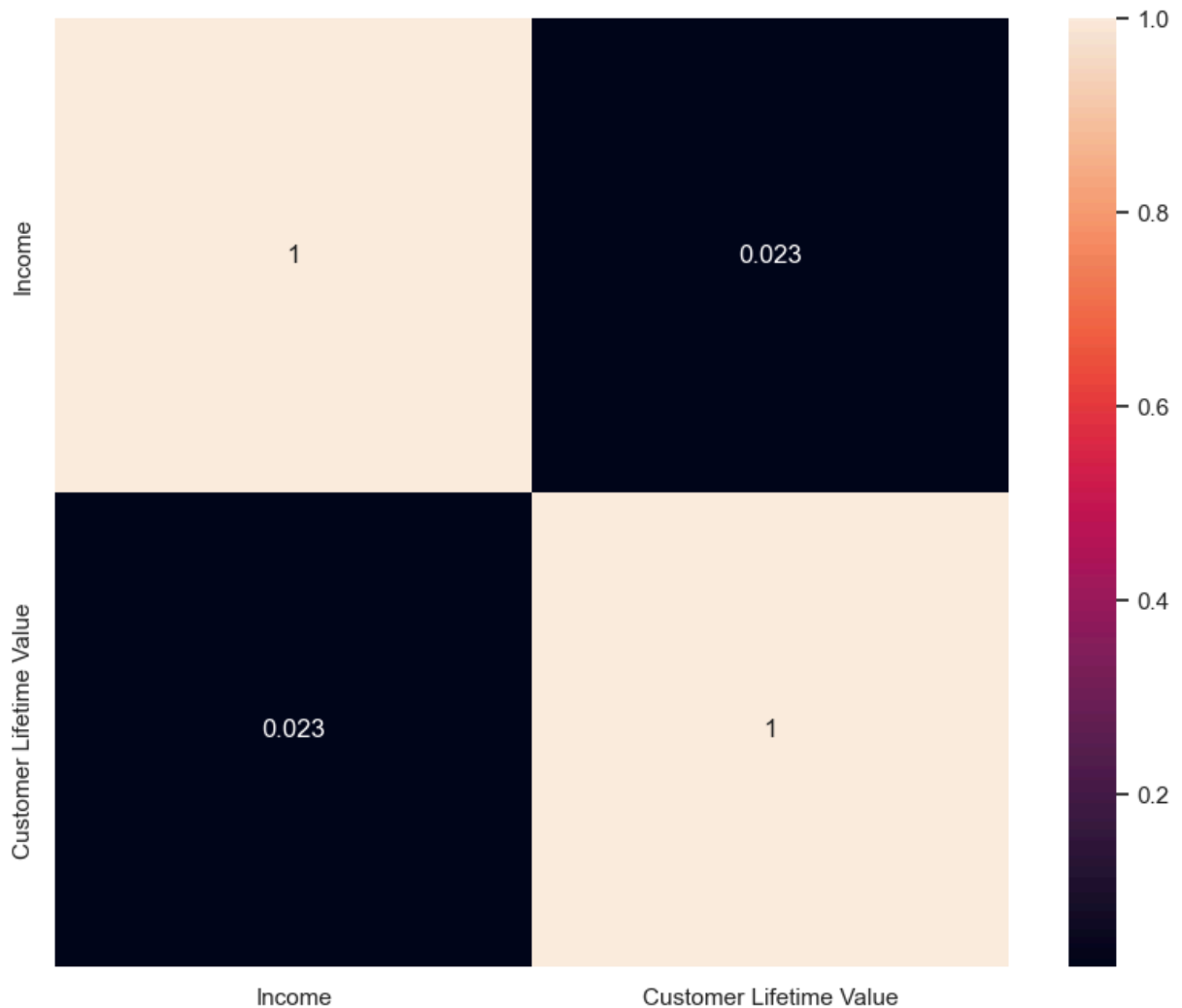
The boxplots show that both Income and Customer Lifetime Value are highly variable and contain several outliers. Most customers have values concentrated in the lower part of the distribution, while a few customers show much higher values. The boxplot of Customer Lifetime Value has a long right tail with many extreme points, confirming the presence of high-value customers identified in the histograms. This indicates that the data is not evenly distributed and that a small group of customers generates disproportionately higher value compared to the majority.

```
In [169... # Scatterplot entre Income e Customer Lifetime Value
plt.scatter(customers["Income"], customers["Customer Lifetime Value"])
plt.xlabel("Income")
plt.ylabel("Customer Lifetime Value")
plt.title("Scatterplot of Income vs Customer Lifetime Value")
plt.show()
```



The scatterplot shows the relationship between Income and Customer Lifetime Value. Each point represents a customer. The plot reveals no strong linear relationship between the two variables — customers with higher income do not necessarily have a higher lifetime value. Most points are concentrated in the lower ranges of both variables, confirming that the majority of customers have modest income and generate relatively low value for the company. This suggests that other factors beyond income may explain customer value differences.

```
In [170... corr = customers[['Income', 'Customer Lifetime Value']].corr(method='pearson')
fig = plt.figure(figsize=(10, 8))
sns.heatmap(data=corr, annot=True, )
plt.show()
```



The correlation heatmap shows a very weak positive relationship (0.024) between Income and Customer Lifetime Value. This means that customers with higher income do not necessarily generate a higher lifetime value for the company. The almost zero correlation confirms what was seen in the scatterplot — the two variables are largely independent, suggesting that other factors, such as customer behavior or engagement with the program, play a more important role in determining the overall value of each customer.

1.5.2. Visualization of Categorical Variables

```
In [171]: sns.set(style="white")

categorical_features = [
    'Gender',
    'LoyaltyStatus',
    'EnrollmentType',
    'Education',
    'Marital Status',
    'Location Code'
]
```

```

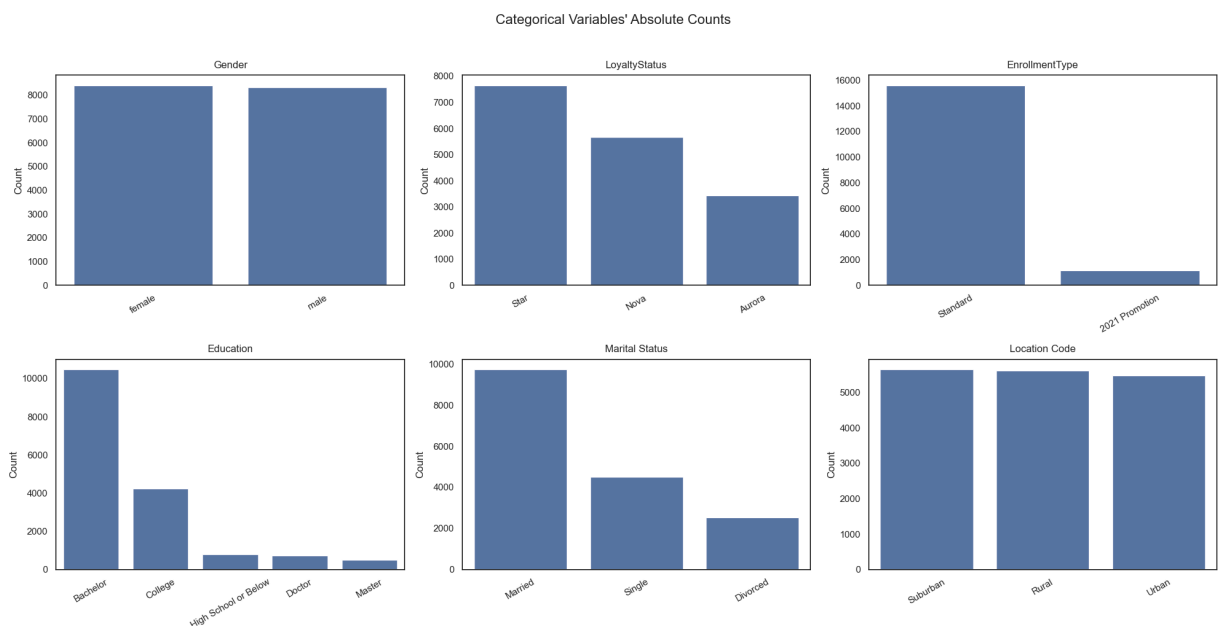
fig, axes = plt.subplots(
    2,
    ceil(len(categorical_features) / 2),
    figsize=(20, 10)
)

for ax, feat in zip(axes.flatten(), categorical_features):
    sns.countplot(
        x=customers[feat],
        ax=ax,
        order=customers[feat].value_counts().index,
    )
    ax.set_title(feat, fontsize=12, pad=8)
    ax.set_xlabel('')
    ax.set_ylabel('Count')
    ax.tick_params(axis='x', rotation=30)

plt.suptitle("Categorical Variables' Absolute Counts", fontsize=16, y=1.02)

plt.tight_layout()
plt.show()

```



The bar charts show the distribution of the categorical variables in the dataset. The Gender variable is balanced, with a similar number of male and female customers. Most customers have a loyalty status of “Star” or “Nova”, while fewer belong to the “Aurora” group. In terms of Enrollment Type, the vast majority are part of the “Standard” program, with only a small share enrolled through the “2021 Promotion”.

Regarding Education, most customers hold a bachelor’s degree, followed by college-level education, and very few have a masters degree, only high school or

lower or a doctors degree. The Marital Status variable shows that married customers represent the largest group, followed by singles and divorced individuals. Finally, the Location Code distribution is fairly balanced across suburban, rural, and urban areas, suggesting that the customer base is geographically diverse.

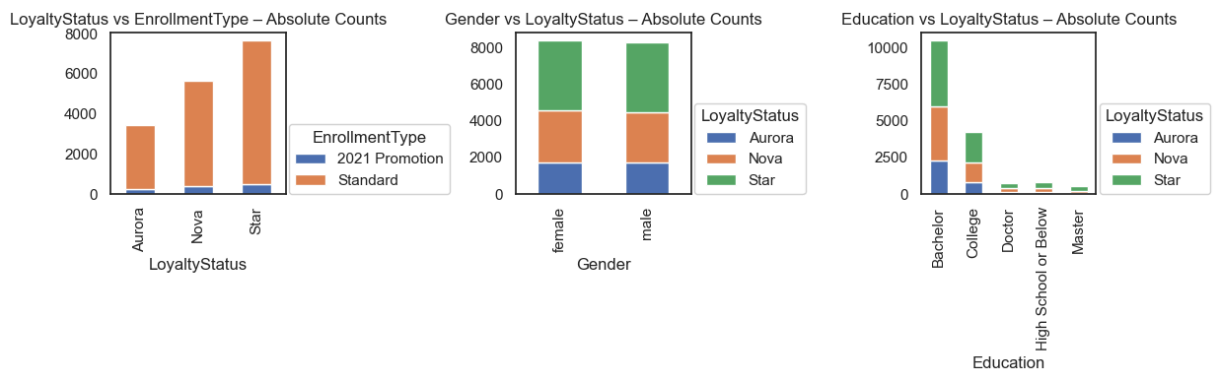
```
In [172... fig, axes = plt.subplots(1,3, figsize=(12,4))

pd.crosstab(customers['LoyaltyStatus'], customers['EnrollmentType']).plot(kind='bar',
axes[0].set_title('LoyaltyStatus vs EnrollmentType – Absolute Counts')
axes[0].legend(title='EnrollmentType', loc=(1.02,0))

pd.crosstab(customers['Gender'], customers['LoyaltyStatus']).plot(kind='bar',
axes[1].set_title('Gender vs LoyaltyStatus – Absolute Counts')
axes[1].legend(title='LoyaltyStatus', loc=(1.02,0))

pd.crosstab(customers['Education'], customers['LoyaltyStatus']).plot(kind='bar',
axes[2].set_title('Education vs LoyaltyStatus – Absolute Counts')
axes[2].legend(title='LoyaltyStatus', loc=(1.02,0))

plt.tight_layout()
plt.show()
```



The first chart (Loyalty Status vs Enrollment Type) shows that most customers in each loyalty category belong to the Standard enrollment type, while the 2021 Promotion group represents only a small share. The “Star” segment is the largest and most dominant across enrollment types, followed by “Nova” and then “Aurora.”

The second chart (Gender vs Loyalty Status) indicates that the distribution of loyalty status is very similar for male and female customers. Both genders show comparable proportions of “Star,” “Nova,” and “Aurora” members, suggesting that gender does not play a significant role in determining loyalty level within this dataset.

In the third chart within each education group, the distribution across loyalty statuses (Star, Nova, and Aurora) remains relatively similar, with Star members consistently forming the largest segment. This suggests that education level

does not strongly influence customer loyalty status, although customers with higher education tend to be more represented overall in the program.

1.5.3. Visualization of Relationships Between Variables

```
In [173]: sns.set(style="whitegrid")

fig, axes = plt.subplots(2, 2, figsize=(12, 8))

#LoyaltyStatus × Customer Lifetime Value
sns.boxplot(
    data=customers,
    x='LoyaltyStatus',
    y='Customer Lifetime Value',
    ax=axes[0, 0]
)
axes[0, 0].set_title('Customer Lifetime Value by Loyalty Status')
axes[0, 0].set_xlabel('Loyalty Status')
axes[0, 0].set_ylabel('Customer Lifetime Value')

#LoyaltyStatus × Income
sns.boxplot(
    data=customers,
    x='LoyaltyStatus',
    y='Income',
    ax=axes[0, 1]
)
axes[0, 1].set_title('Income by Loyalty Status')
axes[0, 1].set_xlabel('Loyalty Status')
axes[0, 1].set_ylabel('Income')

#Education × Income
sns.boxplot(
    data=customers,
    x='Education',
    y='Income',
    ax=axes[1, 0]
)
axes[1, 0].set_title('Income by Education Level')
axes[1, 0].set_xlabel('Education')
axes[1, 0].set_ylabel('Income')

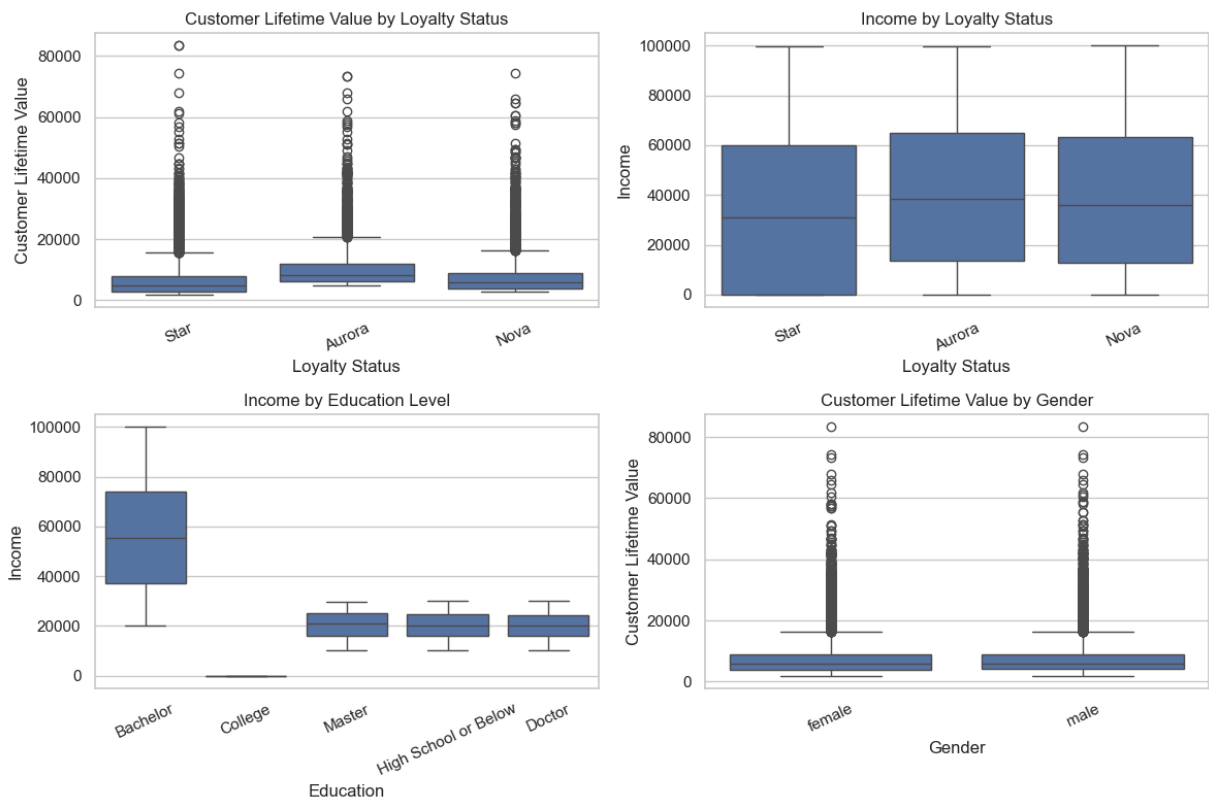
#Gender × Customer Lifetime Value
sns.boxplot(
    data=customers,
    x='Gender',
    y='Customer Lifetime Value',
    ax=axes[1, 1]
)
axes[1, 1].set_title('Customer Lifetime Value by Gender')
axes[1, 1].set_xlabel('Gender')
axes[1, 1].set_ylabel('Customer Lifetime Value')
```

```

for ax in axes.flatten():
    ax.tick_params(axis='x', rotation=25)

plt.tight_layout()
plt.show()

```



The boxplots reveal how Income and Customer Lifetime Value (CLV) vary across different categorical variables.

In the first chart, Customer Lifetime Value by Loyalty Status, customers with a “Star” status tend to have slightly higher lifetime value than “Aurora” or “Nova” members, although there is significant variability and many outliers across all groups.

In Income by Loyalty Status, income levels appear similar among loyalty groups, suggesting that loyalty is not strongly related to customers’ income.

The Income by Education Level plot shows a clearer pattern: customers with a bachelor’s degree have higher and more variable income, while those with lower education levels (such as “High School or Below”) tend to earn less.

Finally, Customer Lifetime Value by Gender indicates no substantial difference between male and female customers — both groups show similar distributions and ranges of lifetime value.

Overall, these plots suggest that while education influences income, gender and loyalty status do not have a strong impact on customer lifetime value.

1.6. Pre-Processing

1.6.1. Filling Missing Values

- As we saw before, there are missing values in the variables "Income" (20 NaN's), "Customer Lifetime Value" (20 NaN's) and "CancellationDate" (14611 NaN's).
- We will start by treating the **missing values in "Income" and "Customer Lifetime Value"**

```
In [174... # Check the number of missing values again
customers[['Income', 'Customer Lifetime Value']].isna().sum()
```

```
Out[174... Income                20
Customer Lifetime Value    20
dtype: int64
```

- We decided to use the **median** to fill the missing values of these two variables.
- Given the right-skewed distribution of both variables, median imputation was selected as the most appropriate method.
- The median is less sensitive to extreme values compared to the mean, ensuring that the imputed data reflects the central tendency of the distributions without introducing artificial bias.
- Also, we thought that using **KNN** could be unnecessary and possibly excessive, because we only have 20 missing values ($\approx 0.12\%$), all in the same rows. Using KNN here might add noise rather than improve data quality, since it depends on distance calculations between variables that may not be properly scaled.
- This approach preserves all records, maintains dataset consistency, and minimizes potential distortion in subsequent clustering analysis.

```
In [175... # Compute median values
median_income = customers['Income'].median()
median_clv = customers['Customer Lifetime Value'].median()

# Fill missing values with the median
customers['Income'].fillna(median_income, inplace=True)
customers['Customer Lifetime Value'].fillna(median_clv, inplace=True)
```

```
# Confirm there are no missing values left
customers[['Income', 'Customer Lifetime Value']].isna().sum()
```

C:\Users\User\AppData\Local\Temp\ipykernel_17372\2050996778.py:6: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
customers['Income'].fillna(median_income, inplace=True)
```

C:\Users\User\AppData\Local\Temp\ipykernel_17372\2050996778.py:7: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
customers['Customer Lifetime Value'].fillna(median_clv, inplace=True)
```

```
Out[175... Income                0
Customer Lifetime Value    0
dtype: int64
```

- Now, we will treat the **missing values in "CancellationDate"**.
- This variable shows when a client left the program. If the value is NaN, it means that the client never left the program, so the client is still active.
- We decided to create a new variable, "Churn", that has this logic:
 - 1 if the client is still active (NaN in "CancellationDate")
 - 0 if client left the program (there is a date in "CancellationDate")

```
In [176... # Creation of variable Churn
customers['Churn'] = customers['CancellationDate'].notna().astype(int)

customers['Churn'].value_counts()
```

```
Out[176... Churn
0      14471
1       2286
Name: count, dtype: int64
```

This means that 14471 customers left the program, and 2286 didn't.

- The variable "CancellationDate" contained 14611 missing values, corresponding to customers who remain active in the loyalty program. Instead of treating these missing values as data quality issues, they were reinterpreted as meaningful information.
- A new binary variable, "Churn", was created to indicate whether a customer is currently active (1) or has canceled their membership (0). This feature will be useful in subsequent analyses to identify behavioral differences between active and inactive customers and to support segmentation based on retention patterns.

1.6.2. Feature Engineering

After creating the variable "Churn", there are a few more variables that we decided to create to enhance the analytical depth of the dataset.

These new variables were created to represent temporal, demographic, and behavioral dimensions that are not directly available in the raw data but are highly relevant for customer segmentation.

"AccountDuration"

- The first variable we decided to create, "AccountDuration" shows the total ***number of days that a customer was or has been active in the program.***
- It was calculated as the difference between "CancellationDate" and "EnrollmentDateOpening". For customers still active in the program, missing cancellation dates were replaced with the current date, allowing the feature to represent ongoing membership duration.
- This variable provides valuable temporal context for segmentation, as it reflects customer's duration in the program and potential loyalty behaviors.

```
In [177... # Calculate account duration (in days)
customers['AccountDuration'] = (
    (customers['CancellationDate'].fillna(pd.Timestamp.now()) - customers['E
    .dt.days
)
```

"YearsInProgram"

- This variable is similar to "AccountDuration" but instead of days it's years (more interpretable in graphs)
- It represents the total number of years a customer has been enrolled in the loyalty program. This feature helps analyze whether long-term members tend to have higher Customer Lifetime Value or exhibit greater loyalty.

```
In [178... customers['YearsInProgram'] = customers['AccountDuration'] / 365
```

"HasHighIncome"

- This new binary feature categorizes customers as high-income (1) or low/medium-income (0), based on the median income value.
- This feature supports analyses on whether income level correlates with customer profitability or loyalty.
- To create the variable, we computed the median value of the "Income" variable, because the median splits the dataset into two equal parts — 50% of customers earn less and 50% earn more.
- This way, the values above the median are assigned 1 (high income) and the values below the median are assigned 0 (low/medium income).

```
In [179... median_income = customers['Income'].median()
customers['HasHighIncome'] = (customers['Income'] > median_income).astype(ir
```

"MaritalGroup"

- Simplifies the Marital Status variable (which originally had the categories "Married", "Divorced" and "Single") into two broader categories: Married and Not Married. So, joining "Single" and "Divorced".
- This reduces noise and helps ensure more stable cluster formation, avoiding underrepresented categories.

```
In [180... customers['MaritalGroup'] = customers['Marital Status'].replace({
    'Married': 'Married',
    'Single': 'Not Married',
    'Divorced': 'Not Married'
})
```

"EnrollmentYear"

- This variable extracts the year of enrollment from "EnrollmentDateOpening".

- For example, maybe many customers joined the program in 2020 because there was a specific campaign.
- It allows tracking customer acquisition trends over time and understanding whether older members differ from recent ones.

```
In [181... customers['EnrollmentYear'] = customers['EnrollmentDateOpening'].dt.year
```

"EducationLevelCode"

- Encodes education levels as ordinal values (from lowest to highest).
- This transformation supports correlation analysis and machine learning models requiring numeric inputs.

```
In [182... education_order = {'High School or Below': 1, 'College': 2, 'Bachelor': 3, '
customers['EducationLevelCode'] = customers['Education'].map(education_order
```

"ProvinceActivity"

- Represents the density of customers per province (counts the number of customers in the same province as the customer)
- It can reveal regional concentration patterns and help identify geographic differences in loyalty behavior. So, it can pretty much show if a certain customer belongs to a province that is more or less active.

```
In [183... province_counts = customers['Province or State'].value_counts()
customers['ProvinceActivity'] = customers['Province or State'].map(province_
```

```
In [184... # Showing the dataset with the new variables
customers.head()
```

Out[184...

	Loyalty#	Province or State	City	Latitude	Longitude	Postal code	Gender	Edu
0	480934	Ontario	Toronto	43.653225	-79.383186	M2Z 4K1	female	Ba
1	549612	Alberta	Edmonton	53.544388	-113.490930	T3G 6Y6	male	C
2	429460	British Columbia	Vancouver	49.282730	-123.120740	V6E 3D9	male	C
3	608370	Ontario	Toronto	43.653225	-79.383186	P1W 1K4	male	C
4	530508	Quebec	Hull	45.428730	-75.713364	J8Y 3Z5	male	Ba

In [185...

```
customers.dtypes
```

Out[185...

```
Loyalty#                int64
Province or State       object
City                   object
Latitude               float64
Longitude              float64
Postal code            object
Gender                 object
Education              object
Location Code          object
Income                float64
Marital Status         object
LoyaltyStatus          object
EnrollmentDateOpening  datetime64[ns]
CancellationDate       datetime64[ns]
Customer Lifetime Value float64
EnrollmentType         object
Churn                  int64
AccountDuration         int64
YearsInProgram          float64
HasHighIncome          int64
MaritalGroup           object
EnrollmentYear         int32
EducationLevelCode     int64
ProvinceActivity       int64
dtype: object
```

1.7. Visual Exploration Including The New Variables

1.7.1. Visualing Numeric Variables

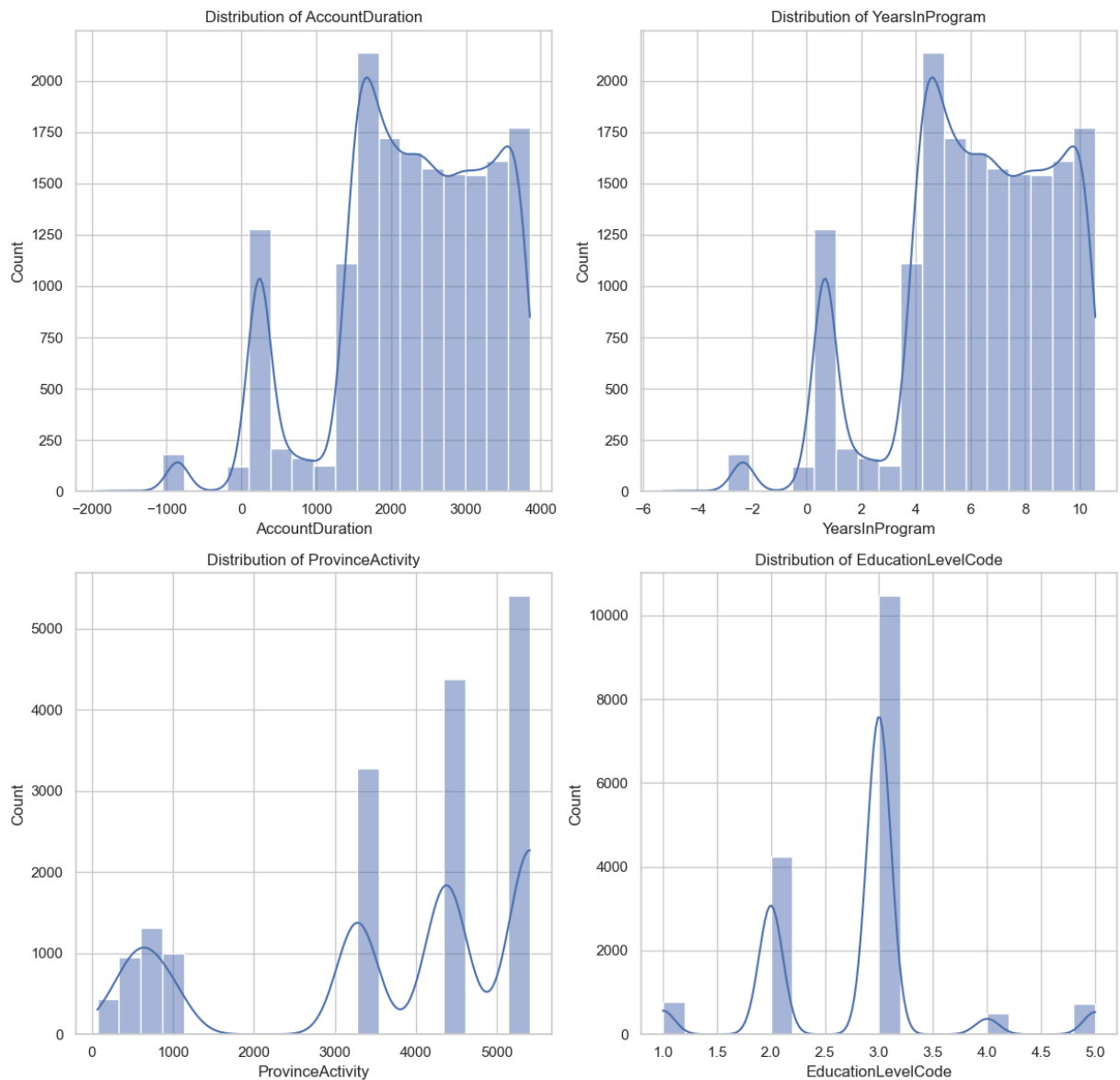
In [186...

```
# List of the new numeric variables
numeric_features = ['AccountDuration', 'YearsInProgram', 'ProvinceActivity',
```

```
In [187... sp_rows = ceil(len(numeric_features) / 2)
sp_cols = 2
fig, axes = plt.subplots(sp_rows, sp_cols, figsize=(12, 6 * sp_rows))

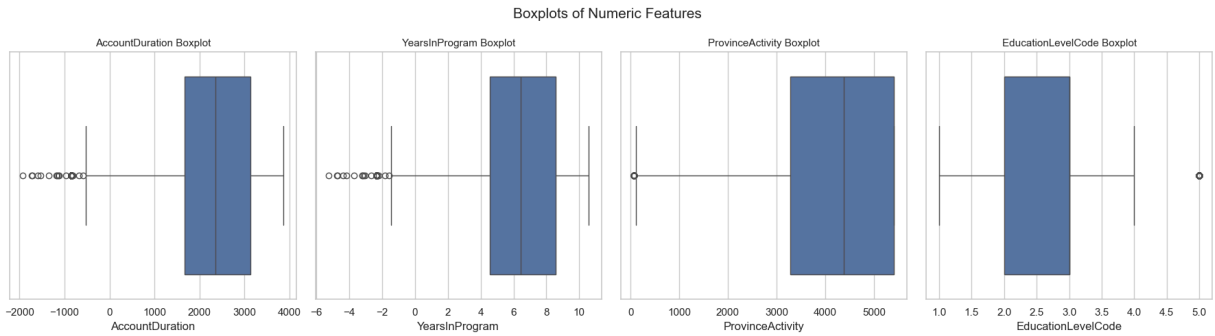
for ax, feat in zip(axes.flatten(), numeric_features):
    sns.histplot(customers[feat], bins=20, kde=True, ax=ax)
    ax.set_title(f'Distribution of {feat}', fontsize=12)
plt.suptitle("Histograms of Engineered Numeric Features", fontsize=16, y=1.05)
plt.tight_layout()
plt.show()
```

Histograms of Engineered Numeric Features



```
In [188... # Boxplots
fig, axes = plt.subplots(1, len(numeric_features), figsize=(18, 5), tight_la
for ax, feat in zip(axes.flatten(), numeric_features):
    sns.boxplot(x=customers[feat], ax=ax)
    ax.set_title(f'{feat} Boxplot', fontsize=11)
```

```
plt.suptitle("Boxplots of Numeric Features", fontsize=15)
plt.show()
```



In []:

```
In [189... cat_features = ['Churn', 'HasHighIncome', 'MaritalGroup', 'EnrollmentYear']

sp_rows = 2
sp_cols = ceil(len(cat_features) / 2)
fig, axes = plt.subplots(sp_rows, sp_cols, figsize=(14, 8))

for ax, feat in zip(axes.flatten(), cat_features):
    sns.countplot(x=customers[feat], order=customers[feat].value_counts().ir
    ax.set_title(f'{feat} Counts', fontsize=12)
    ax.set_xlabel('')
    ax.set_ylabel('Count')
plt.suptitle("Categorical Engineered Features – Absolute Frequencies", fonts
plt.tight_layout()
plt.show()
```

C:\Users\User\AppData\Local\Temp\ipykernel_17372\3048727431.py:8: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(x=customers[feat], order=customers[feat].value_counts().index, ax=ax, palette="crest")
```

C:\Users\User\AppData\Local\Temp\ipykernel_17372\3048727431.py:8: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(x=customers[feat], order=customers[feat].value_counts().index, ax=ax, palette="crest")
```

C:\Users\User\AppData\Local\Temp\ipykernel_17372\3048727431.py:8: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

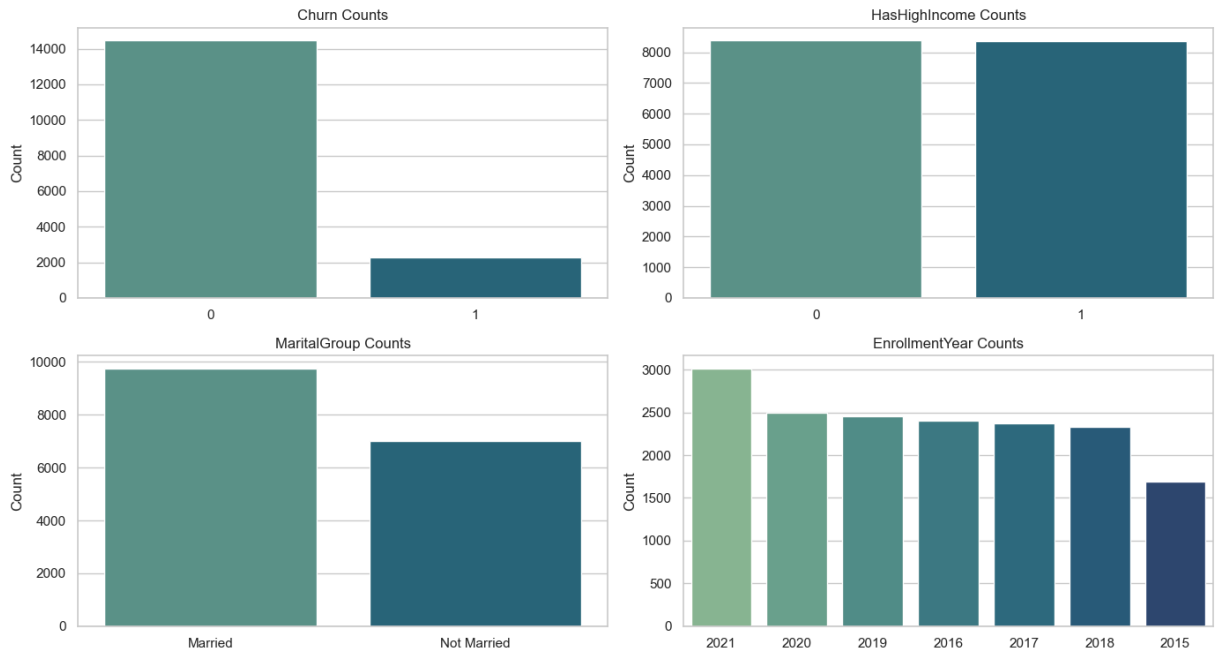
```
sns.countplot(x=customers[feat], order=customers[feat].value_counts().index, ax=ax, palette="crest")
```

C:\Users\User\AppData\Local\Temp\ipykernel_17372\3048727431.py:8: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(x=customers[feat], order=customers[feat].value_counts().index, ax=ax, palette="crest")
```

Categorical Engineered Features – Absolute Frequencies

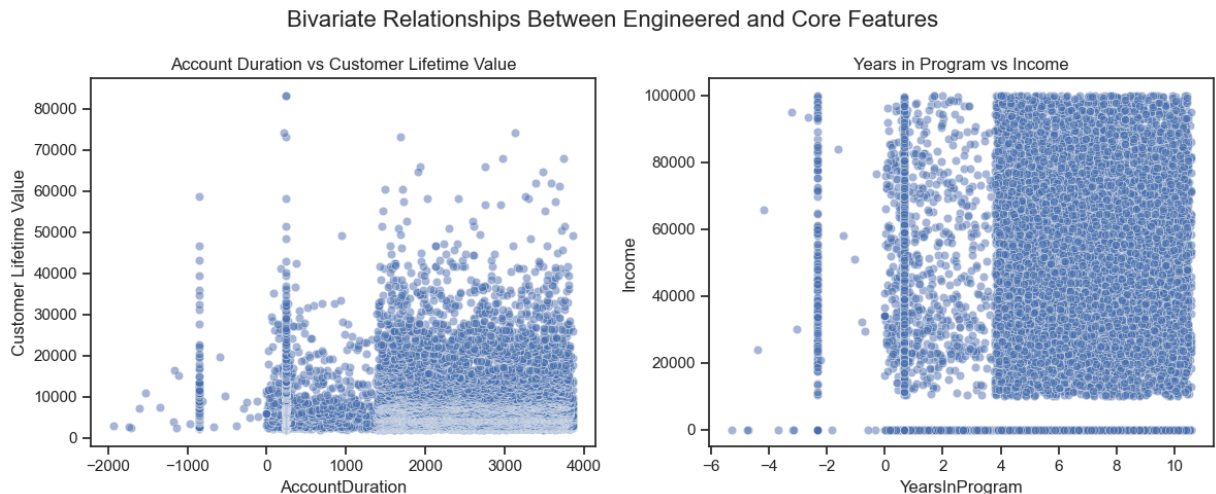


```
In [190]: sns.set_style("ticks")
fig, axes = plt.subplots(1, 2, figsize=(12, 5), tight_layout=True)

sns.scatterplot(data=customers, x='AccountDuration', y='Customer Lifetime Value', alpha=0.5, ax=axes[0])
axes[0].set_title('Account Duration vs Customer Lifetime Value')

sns.scatterplot(data=customers, x='YearsInProgram', y='Income', alpha=0.5, ax=axes[1])
axes[1].set_title('Years in Program vs Income')

plt.suptitle("Bivariate Relationships Between Engineered and Core Features", y=1.05)
plt.show()
```



```
In [191]: sns.set(style="whitegrid")
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

sns.boxplot(data=customers, x='Churn', y='Customer Lifetime Value', ax=axes[0])
axes[0].set_title('Customer Lifetime Value by Churn Status')
axes[0].set_xlabel('Churn (1 = Left, 0 = Active)')

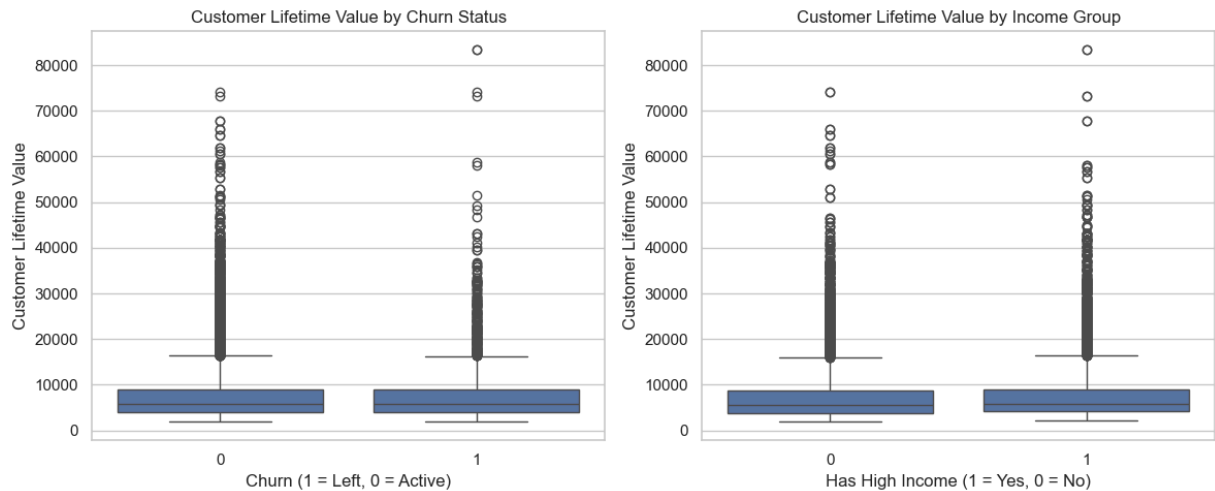
sns.boxplot(data=customers, x='Churn', y='Income', ax=axes[1])
axes[1].set_title('Income by Churn Status')
axes[1].set_xlabel('Churn (1 = Left, 0 = Active)')
```

```

sns.boxplot(data=customers, x='HasHighIncome', y='Customer Lifetime Value',
axes[1].set_title('Customer Lifetime Value by Income Group')
axes[1].set_xlabel('Has High Income (1 = Yes, 0 = No)')

plt.tight_layout()
plt.show()

```

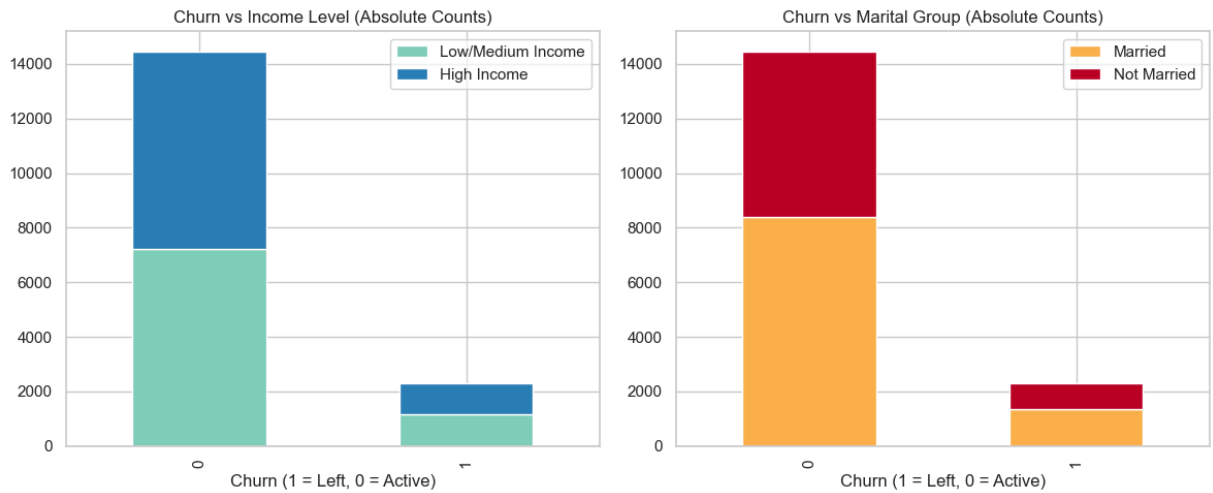


```

In [192... corr_features = ['Income', 'Customer Lifetime Value', 'AccountDuration', 'Ye
corr = customers[corr_features].corr(method='pearson').round(2)

fig = plt.figure(figsize=(10, 7))
sns.heatmap(corr, annot=True, cmap="coolwarm", center=0)
plt.title("Correlation Matrix – Engineered and Core Features")
plt.show()

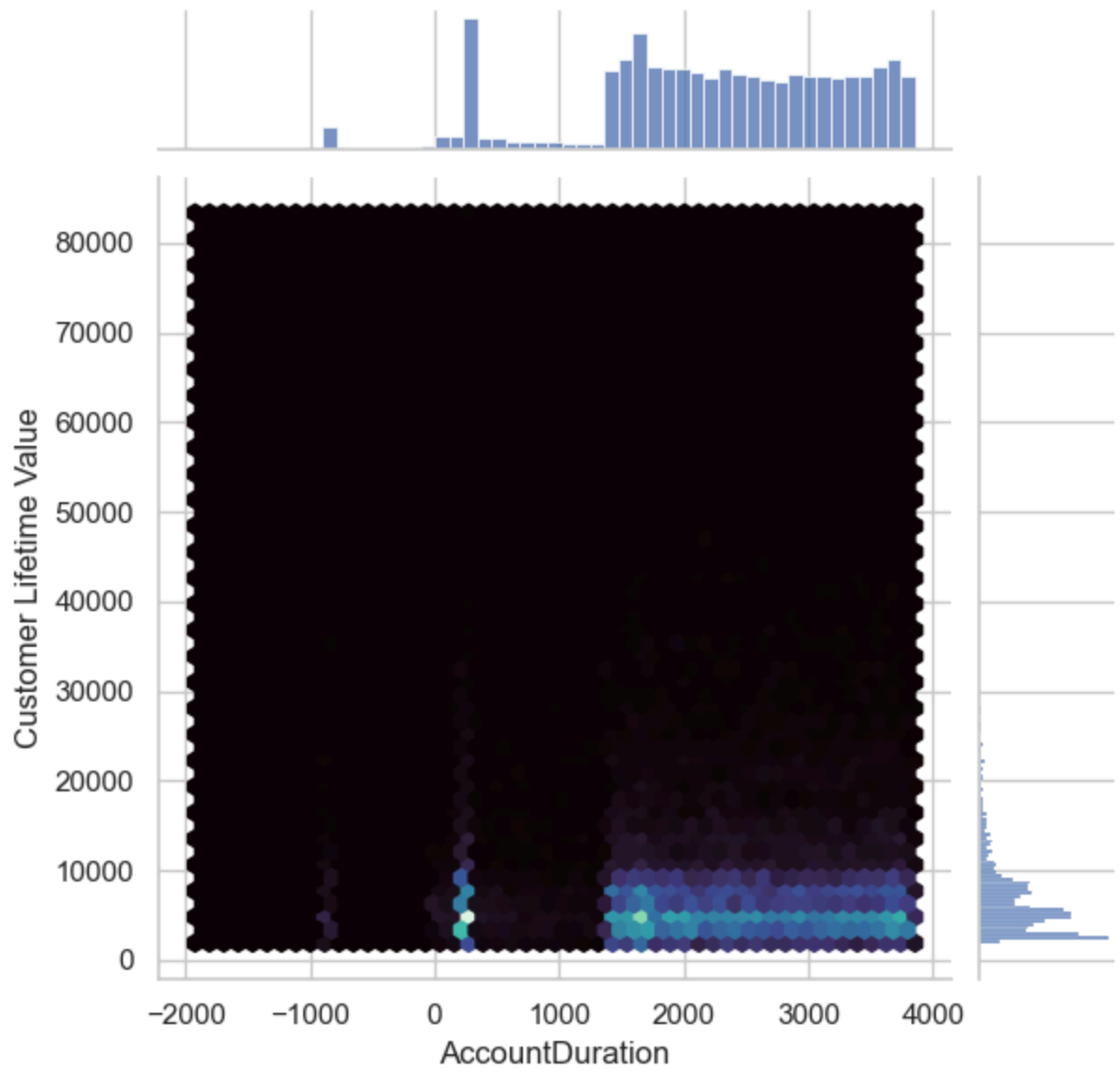
```

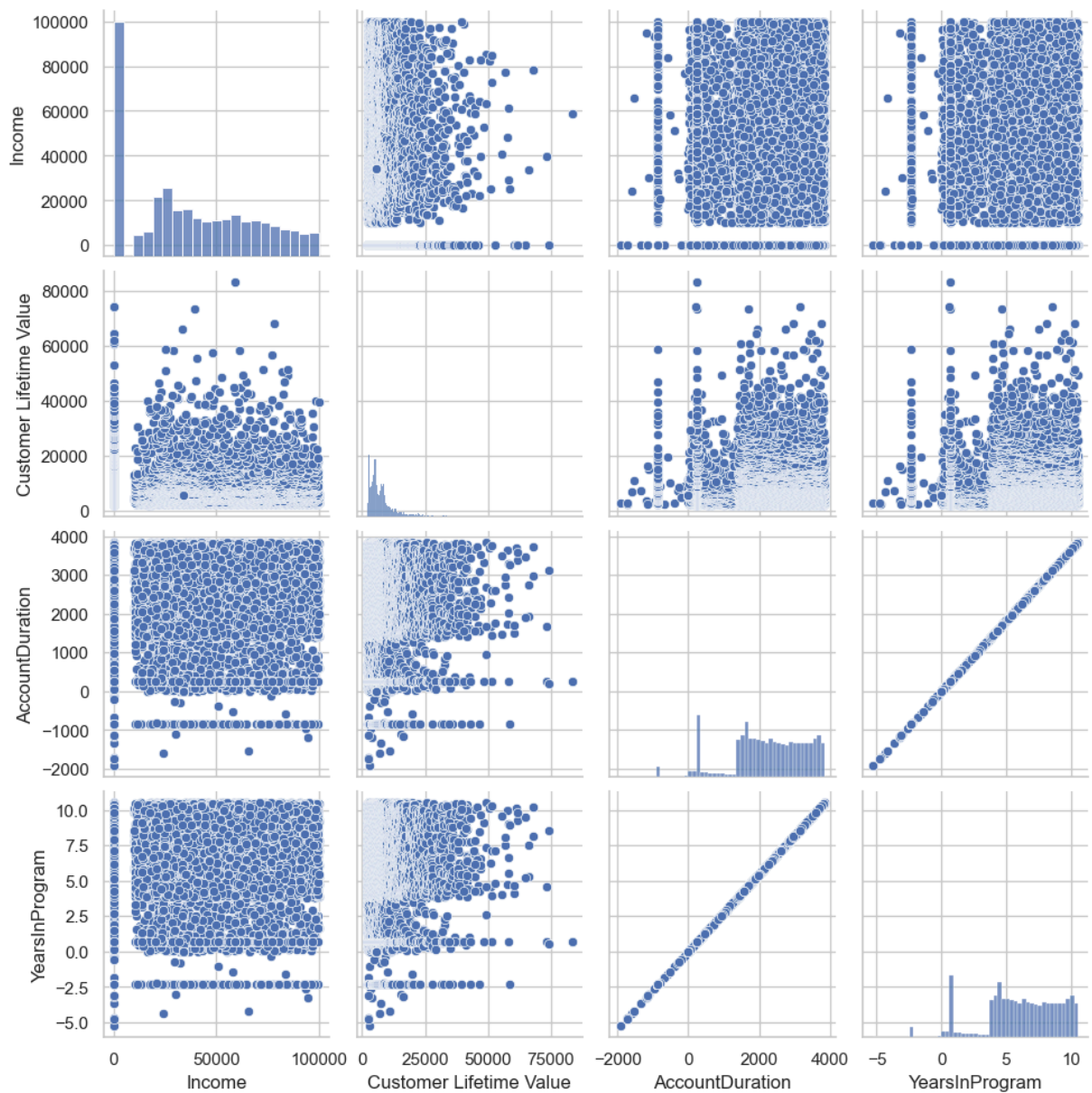
```
In [194... sns.jointplot(data=customers, x='AccountDuration', y='Customer Lifetime Value',
plt.suptitle('Joint Distribution: Account Duration vs CLV', y=1.02)
plt.show()

sns.pairplot(customers[['Income', 'Customer Lifetime Value', 'AccountDuration'])
plt.suptitle("Pairwise Relationships of Engineered Variables", y=1.02)
plt.show()
```

Joint Distribution: Account Duration vs CLV



Pairwise Relationships of Engineered Variables



2. Flights Dataset

Importing, Exploring and Pre-processing

2.1. Import the dataset

```
In [195... flights = pd.read_csv('DM_AIAI_FlightsDB.csv')
```

2.2. Exploratory Data Analysis

```
In [196...] flights.head()
```

```
Out[196...]   Loyalty#  Year  Month  YearMonthDate  NumFlights  NumFlightsWithComp
```

	Loyalty#	Year	Month	YearMonthDate	NumFlights	NumFlightsWithComp
0	413052	2021	12	12/1/2021	2.0	
1	464105	2021	12	12/1/2021	0.0	
2	681785	2021	12	12/1/2021	10.0	
3	185013	2021	12	12/1/2021	16.0	
4	216596	2021	12	12/1/2021	9.0	

```
In [197...] flights.shape
```

```
Out[197...] (608436, 10)
```

The flights dataset consists of 608436 observations and 10 variables.

```
In [198...] flights.columns
```

```
Out[198...] Index(['Loyalty#', 'Year', 'Month', 'YearMonthDate', 'NumFlights',  
                  'NumFlightsWithCompanions', 'DistanceKM', 'PointsAccumulated',  
                  'PointsRedeemed', 'DollarCostPointsRedeemed'],  
                  dtype='object')
```

```
In [199...] flights.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 608436 entries, 0 to 608435  
Data columns (total 10 columns):  
#   Column                                Non-Null Count  Dtype  
---  -  
0   Loyalty#                             608436 non-null int64  
1   Year                                 608436 non-null int64  
2   Month                               608436 non-null int64  
3   YearMonthDate                        608436 non-null object  
4   NumFlights                          608436 non-null float64  
5   NumFlightsWithCompanions            608436 non-null float64  
6   DistanceKM                          608436 non-null float64  
7   PointsAccumulated                   608436 non-null float64  
8   PointsRedeemed                      608436 non-null float64  
9   DollarCostPointsRedeemed            608436 non-null float64  
dtypes: float64(6), int64(3), object(1)  
memory usage: 46.4+ MB
```

```
In [200...] flights.dtypes
```

```

Out[200...] Loyalty#          int64
            Year          int64
            Month         int64
            YearMonthDate  object
            NumFlights     float64
            NumFlightsWithCompanions float64
            DistanceKM     float64
            PointsAccumulated float64
            PointsRedeemed  float64
            DollarCostPointsRedeemed float64
            dtype: object

```

- The variable "YearMonthDate" should be datetime

```

In [201...] flights[['NumFlights', 'NumFlightsWithCompanions', 'DistanceKM',
                    'PointsAccumulated', 'PointsRedeemed', 'DollarCostPointsRedeemed']]

```

```

Out[201...]

```

	NumFlights	NumFlightsWithCompanions	DistanceKM	PointsAccu
count	608436.000000	608436.000000	608436.000000	608436
mean	3.908107	0.983944	7939.341419	793
std	5.057889	2.003785	10260.421873	1025
min	0.000000	0.000000	0.000000	0
25%	0.000000	0.000000	0.000000	0
50%	0.000000	0.000000	856.400000	85
75%	7.200000	0.900000	15338.175000	1533
max	21.000000	11.000000	42040.000000	4204

Summary:

- Most customers don't fly every month, but there is a small group of highly active travellers.
- Flight frequency and distance are heavily skewed, showing a mix of occasional and heavy users.
- Point accumulation exceeds redemption, suggesting latent loyalty value that the airline could leverage (e.g., campaigns to encourage point spending).
- Some customers travel long distances or frequently with companions, likely representing premium or family-oriented segments.

2.2.3. Checking for Missing Values

```
In [202... flights.isna().sum()
```

```
Out[202... Loyalty#          0
Year            0
Month           0
YearMonthDate   0
NumFlights      0
NumFlightsWithCompanions 0
DistanceKM      0
PointsAccumulated 0
PointsRedeemed   0
DollarCostPointsRedeemed 0
dtype: int64
```

```
In [203... # Making sure that there are really no missing values in this dataset
# Because sometimes instead of NaN, the rows have ""

# replace "" by nans
flights.replace("", np.nan, inplace=True)

# count of missing values
flights.isna().sum()
```

```
Out[203... Loyalty#          0
Year            0
Month           0
YearMonthDate   0
NumFlights      0
NumFlightsWithCompanions 0
DistanceKM      0
PointsAccumulated 0
PointsRedeemed   0
DollarCostPointsRedeemed 0
dtype: int64
```

2.2.4. Checking for Duplicated Values

```
In [204... # Checking for duplicated observations
flights.duplicated().sum()
```

```
Out[204... np.int64(2903)
```

```
In [205... # Removing all duplicates (2903)
flights = flights.drop_duplicates()
```

```
In [206... flights.duplicated().sum()
```

```
Out[206... np.int64(0)
```

```
In [207... # Checking for rows with the same client (loyalty#), year and month
# But the other columns don't need to be the same

flights[flights.duplicated(subset=['Loyalty#', 'Year', 'Month'], keep=False)]
```

Out[207...		Loyalty#	Year	Month	YearMonthDate	NumFlights	NumFlightsWith
	27	279419	2021	12	12/1/2021	12.0	
	171	990512	2021	12	12/1/2021	5.0	
	248	322169	2021	12	12/1/2021	2.0	
	297	106001	2021	12	12/1/2021	7.0	
	678	402697	2021	12	12/1/2021	12.0	
	...	...	...	...	...	...	
	608222	988392	2019	12	12/1/2019	0.0	
	608248	989528	2019	12	12/1/2019	11.7	
	608249	989528	2019	12	12/1/2019	0.0	
	608298	992168	2019	12	12/1/2019	11.7	
	608299	992168	2019	12	12/1/2019	5.4	

5996 rows × 10 columns

```
In [208... # Checking how many cases of clients with multiple rows in the same year and
flights.duplicated(subset=['Loyalty#', 'Year', 'Month']).sum()
```

Out[208... np.int64(3001)

```
In [209... # Remove duplicated rows keeping only the first occurrence
flights = flights.drop_duplicates(subset=['Loyalty#', 'Year', 'Month'], keep

# Confirm there are no more duplicates
flights.duplicated(subset=['Loyalty#', 'Year', 'Month']).sum()
```

Out[209... np.int64(0)

```
In [210... flights.shape
```

Out[210... (602532, 10)

After reviewing the duplicated entries, we confirmed that they represent multiple records of the same customer within the same year and month.

We decided to keep only the first occurrence and remove the remaining duplicates.

This ensures that each customer has a single record per month while maintaining data integrity.

2.3. Initial Data Preparation

2.3.1. Converting variables to datetime

```
In [211... flights['YearMonthDate'] = pd.to_datetime(flights['YearMonthDate'])
```

```
In [212]: flights[['YearMonthDate']].info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 602532 entries, 0 to 608435
Data columns (total 1 columns):
#   Column          Non-Null Count  Dtype
---  -
0   YearMonthDate    602532 non-null  datetime64[ns]
dtypes: datetime64[ns](1)
memory usage: 9.2 MB
```

2.3.2. Converting variables to integer

```
In [213... #turn into int
flights['NumFlightsWithCompanions'] = flights['NumFlightsWithCompanions'].rc
```

```
In [214... #turn into int
flights['NumFlights'] = flights['NumFlights'].round().astype(int)
```

```
In [215... flights[['NumFlights']].info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 602532 entries, 0 to 608435
Data columns (total 1 columns):
#   Column          Non-Null Count  Dtype
---  -
0   NumFlights      602532 non-null  int64
dtypes: int64(1)
memory usage: 9.2 MB
```

```
In [216]: flights[['NumFlightsWithCompanions']].info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 602532 entries, 0 to 608435
Data columns (total 1 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   NumFlightsWithCompanions             602532 non-null  int64
dtypes: int64(1)
memory usage: 9.2 MB
```

2.3.3. Checking for inconsistencies

[illegible]

```

        (flights['DollarCostPointsRedeemed'] > 0) |
        (flights['NumFlightsWithCompanions'] > 0))]]
print("Inconsistent rows:", len(invalid_rows))
print("Percentage of dataset:", round(len(invalid_rows) / len(flights) * 100))

```

Inconsistent rows: 5856

Percentage of dataset: 0.97 %

- Since it is less than 1% of the rows I think that we could delete the rows

```

In [218... flights = flights.drop(invalid_rows.index).reset_index(drop=True)
print("Removed inconsistent rows:", len(invalid_rows))
print("Remaining rows:", len(flights))

```

Removed inconsistent rows: 5856

Remaining rows: 596676

2.4. Data Visualization

2.4.1. Visualization of Numeric Variables (Univariate Analysis)

```

In [219... sns.set(style="whitegrid")

fig, axes = plt.subplots(2, 2, figsize=(12, 8))

axes[0, 0].hist(flights['NumFlights'], bins=30)
axes[0, 0].set_title('Distribution of Number of Flights')
axes[0, 0].set_xlabel('Number of Flights')
axes[0, 0].set_ylabel('Frequency')

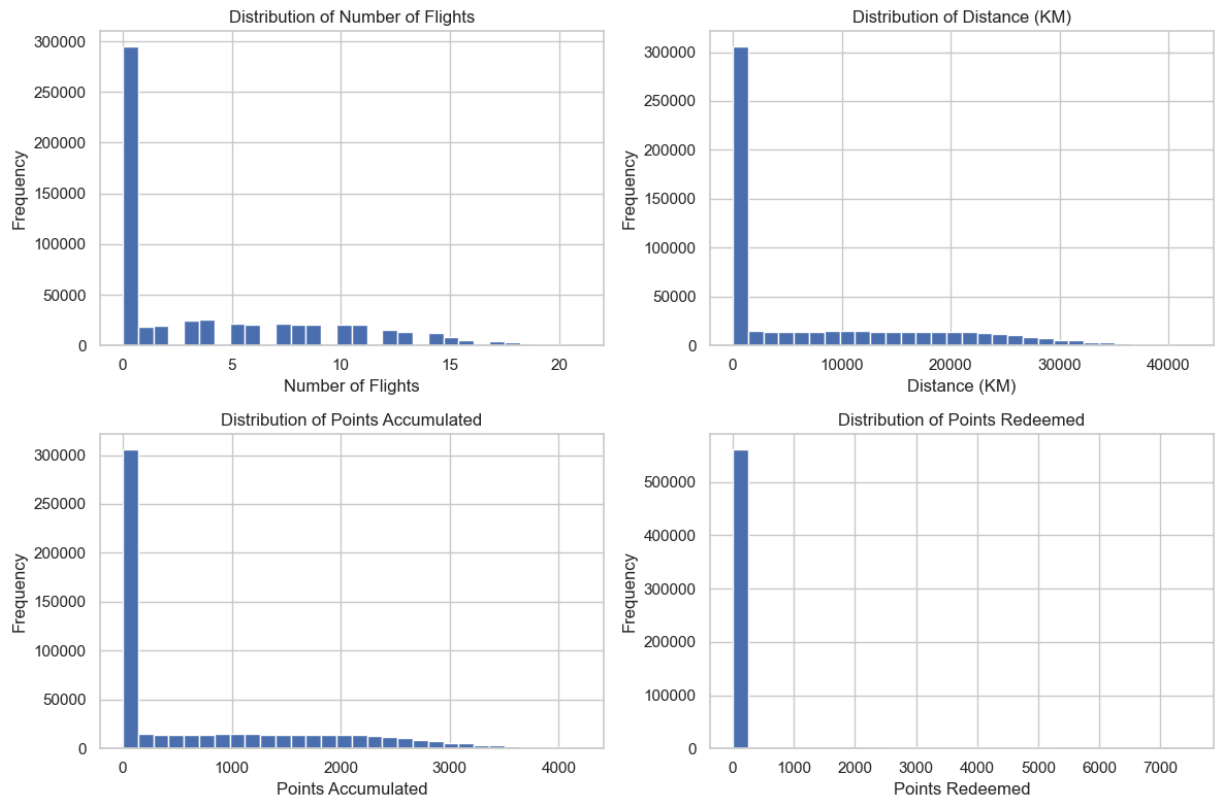
axes[0, 1].hist(flights['DistanceKM'], bins=30)
axes[0, 1].set_title('Distribution of Distance (KM)')
axes[0, 1].set_xlabel('Distance (KM)')
axes[0, 1].set_ylabel('Frequency')

axes[1, 0].hist(flights['PointsAccumulated'], bins=30)
axes[1, 0].set_title('Distribution of Points Accumulated')
axes[1, 0].set_xlabel('Points Accumulated')
axes[1, 0].set_ylabel('Frequency')

axes[1, 1].hist(flights['PointsRedeemed'], bins=30)
axes[1, 1].set_title('Distribution of Points Redeemed')
axes[1, 1].set_xlabel('Points Redeemed')
axes[1, 1].set_ylabel('Frequency')

plt.tight_layout()
plt.show()

```



The distributions of flight-related variables are highly right-skewed, showing that most customers have very limited flight activity. The majority of observations correspond to customers with few or no flights, short total travel distances, and low numbers of points accumulated or redeemed. Only a small group of frequent flyers shows much higher values, forming the long right tails visible in each plot. This pattern suggests that the program has a large base of low-activity members and a smaller segment of highly active travelers who contribute disproportionately to the total points and distance metrics.

```
In [220... sns.set(style="whitegrid")

fig, axes = plt.subplots(2, 2, figsize=(12, 8))

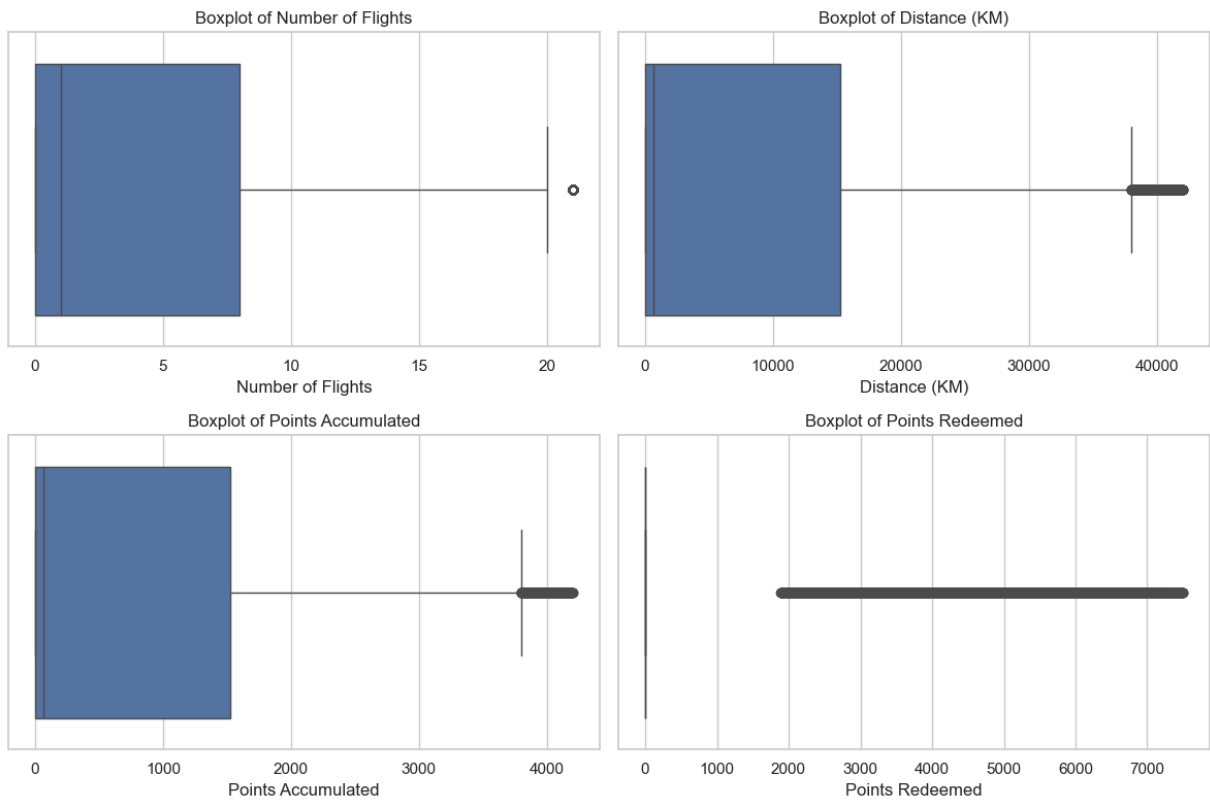
sns.boxplot(x=flights['NumFlights'], ax=axes[0, 0])
axes[0, 0].set_title('Boxplot of Number of Flights')
axes[0, 0].set_xlabel('Number of Flights')

sns.boxplot(x=flights['DistanceKM'], ax=axes[0, 1])
axes[0, 1].set_title('Boxplot of Distance (KM)')
axes[0, 1].set_xlabel('Distance (KM)')

sns.boxplot(x=flights['PointsAccumulated'], ax=axes[1, 0])
axes[1, 0].set_title('Boxplot of Points Accumulated')
axes[1, 0].set_xlabel('Points Accumulated')

sns.boxplot(x=flights['PointsRedeemed'], ax=axes[1, 1])
axes[1, 1].set_title('Boxplot of Points Redeemed')
axes[1, 1].set_xlabel('Points Redeemed')
```

```
plt.tight_layout()
plt.show()
```



These boxplots illustrate the presence of significant variability and multiple outliers across all flight-related variables. For each metric — Number of Flights, Distance (KM), Points Accumulated, and Points Redeemed — most observations are concentrated near the lower end of the scale, while a small group of customers shows extremely high values. This confirms that the dataset is highly skewed, with a large majority of low-activity members and a few very active flyers who travel more frequently, cover longer distances, and accumulate or redeem substantially more points.

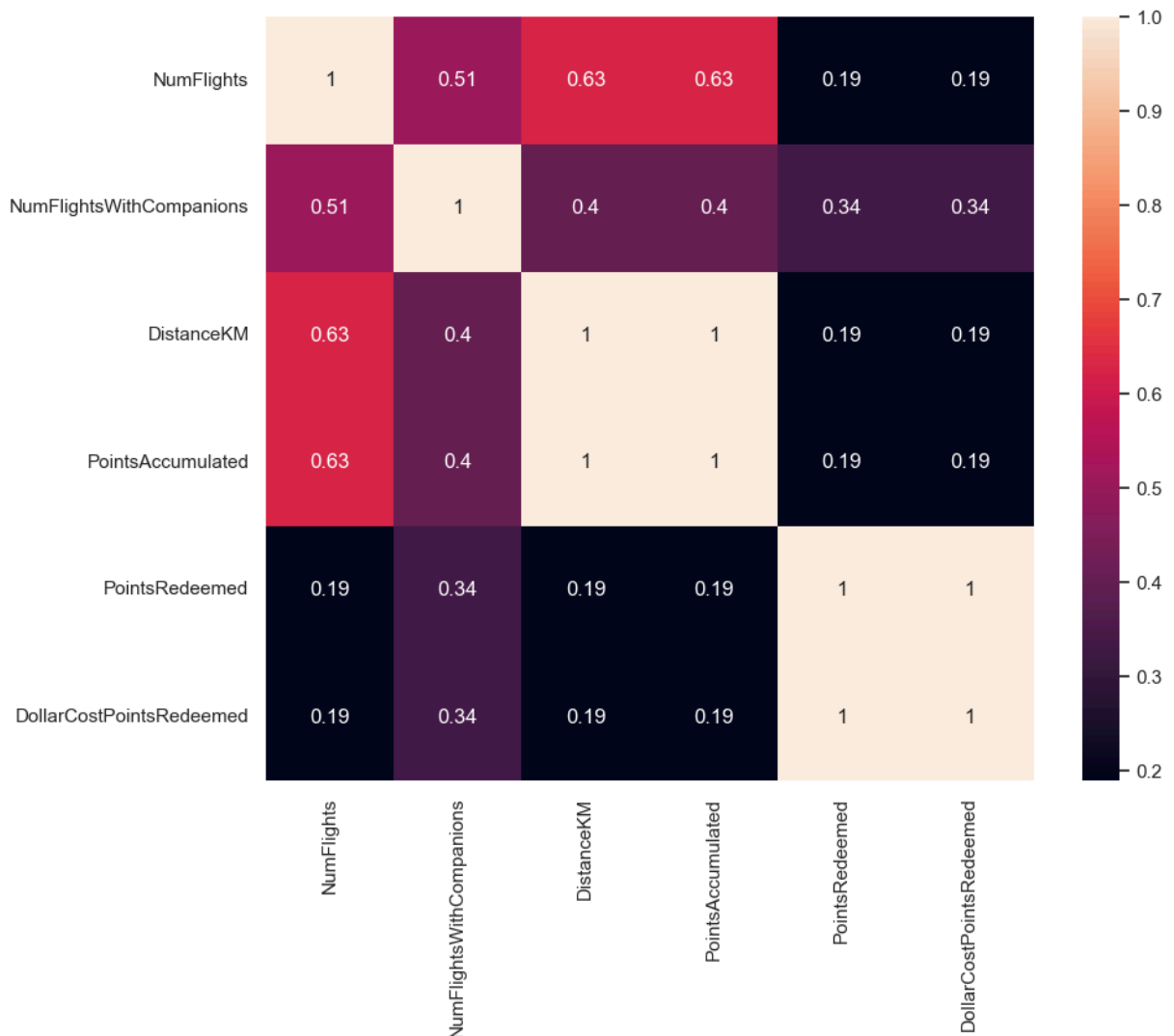
2.4.2 Correlation Analysis (Bivariate Relationships)

```
In [221]: sns.set(style="white")

numeric_features = [
    'NumFlights',
    'NumFlightsWithCompanions',
    'DistanceKM',
    'PointsAccumulated',
    'PointsRedeemed',
    'DollarCostPointsRedeemed'
]
```

```
corr = flights[numeric_features].corr(method='pearson')

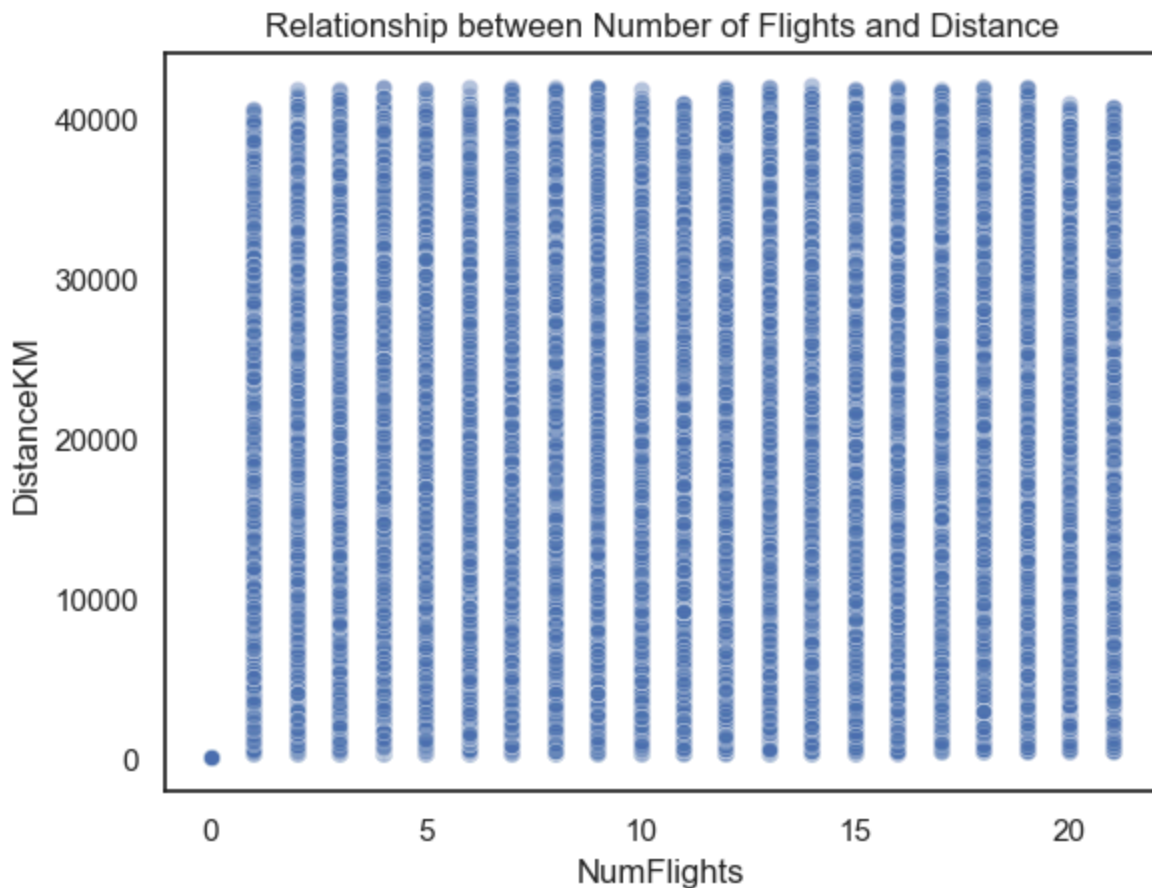
fig = plt.figure(figsize=(10, 8))
sns.heatmap(data=corr, annot=True, )
plt.show()
```



The correlation heatmap reveals moderate to strong positive relationships among the flight activity variables. Specifically, Number of Flights, Distance (KM), and Points Accumulated are strongly correlated (around 0.6), indicating that customers who fly more frequently also travel longer distances and earn more points — an expected pattern in a mileage-based loyalty program. The Number of Flights with Companions shows weaker correlations with these variables, suggesting that traveling with others is less tied to overall flight activity. In contrast, Points Redeemed and Dollar Cost of Points Redeemed exhibit only weak correlations with the other metrics, implying that redemption behavior is more independent from flight frequency and distance traveled.

```
In [222...] sns.scatterplot(data=flights, x='NumFlights', y='DistanceKM', alpha=0.4)
plt.title('Relationship between Number of Flights and Distance')
```

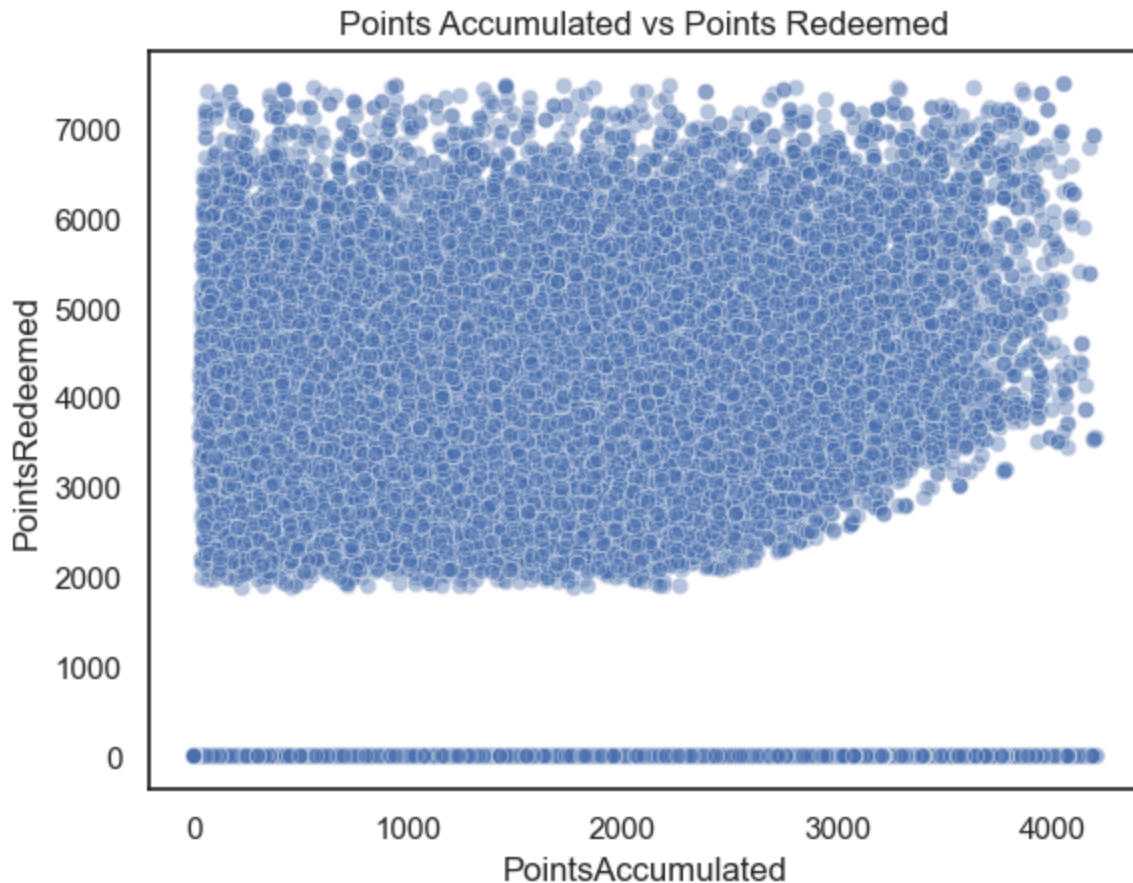
```
Out[222...] Text(0.5, 1.0, 'Relationship between Number of Flights and Distance')
```



This scatterplot shows how the total flight distance varies with the number of flights per customer. The points are vertically aligned because the number of flights is an integer, while distances are continuous. Most customers fly few times, but those who travel more tend to cover larger total distances — as expected in a frequent-flyer context.

```
In [223...] sns.scatterplot(data=flights, x='PointsAccumulated', y='PointsRedeemed', alpha=0.5,
plt.title('Points Accumulated vs Points Redeemed'))
```

```
Out[223...] Text(0.5, 1.0, 'Points Accumulated vs Points Redeemed')
```

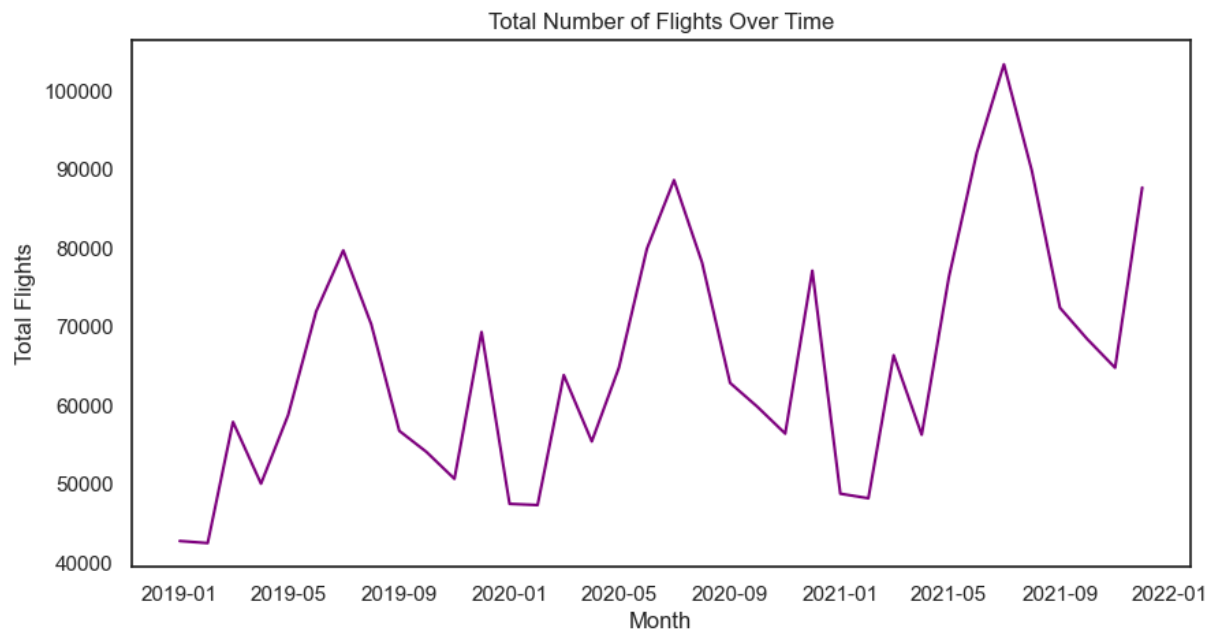


This chart explores the relationship between loyalty points earned and points redeemed. There is a visible positive relationship — customers who accumulate more points generally redeem more. However, many observations lie along the bottom axis (zero redemptions), suggesting that a large portion of customers never use their accumulated points.

2.4.3 Temporal Patterns

```
In [224... # Line chart - Total flights over time
flights_per_month = flights.groupby('YearMonthDate')['NumFlights'].sum().res

plt.figure(figsize=(10,5))
plt.plot(flights_per_month['YearMonthDate'], flights_per_month['NumFlights'])
plt.title('Total Number of Flights Over Time')
plt.xlabel('Month')
plt.ylabel('Total Flights')
plt.show()
```



The line chart shows fluctuations in the total number of flights over time. Despite month-to-month variability, there is a general upward trend with periodic peaks that may reflect seasonality or promotional periods. The data also reveals temporary declines, possibly associated with external events or lower travel seasons.

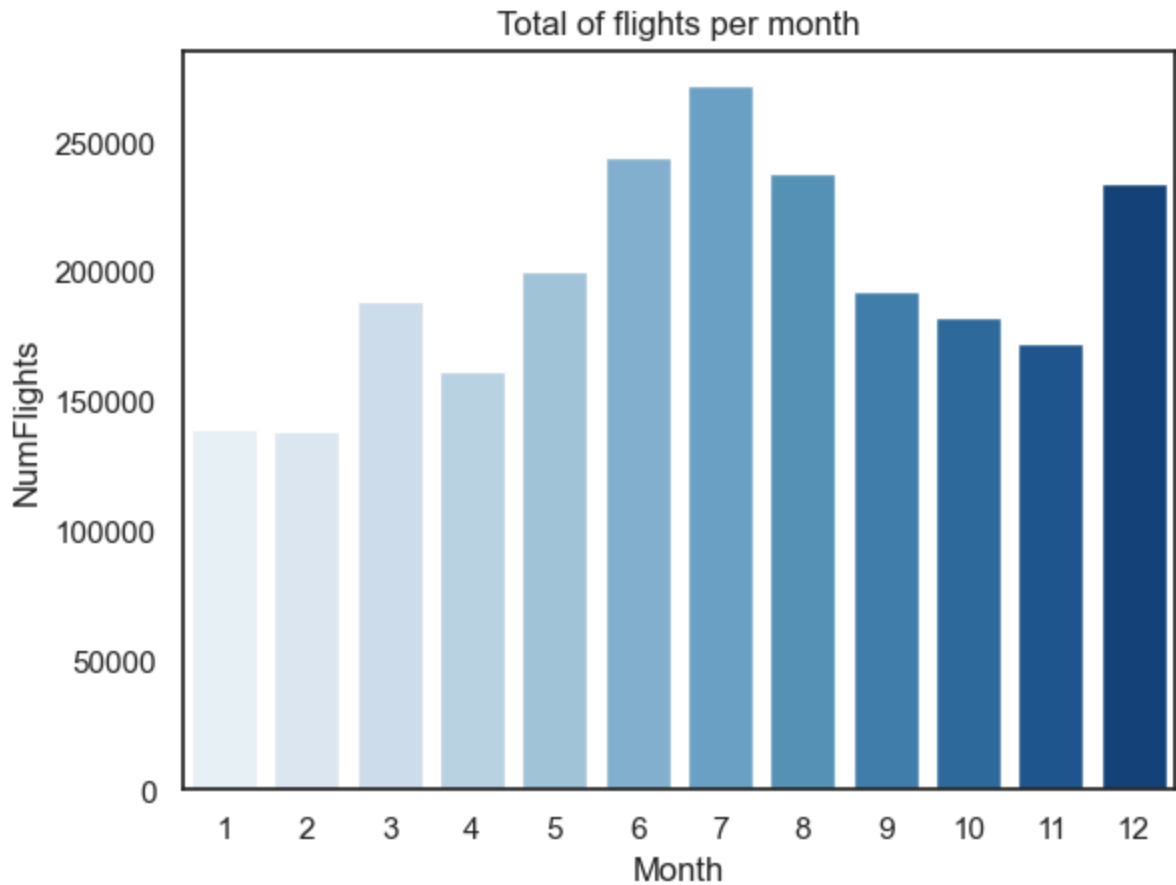
```
In [225... sns.barplot(data=flights.groupby('Month')['NumFlights'].sum().reset_index(),
             x='Month', y='NumFlights', palette='Blues')
plt.title('Total of flights per month')
```

C:\Users\User\AppData\Local\Temp\ipykernel_17372\2424916187.py:1: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(data=flights.groupby('Month')['NumFlights'].sum().reset_index(),
```

```
Out[225... Text(0.5, 1.0, 'Total of flights per month')
```

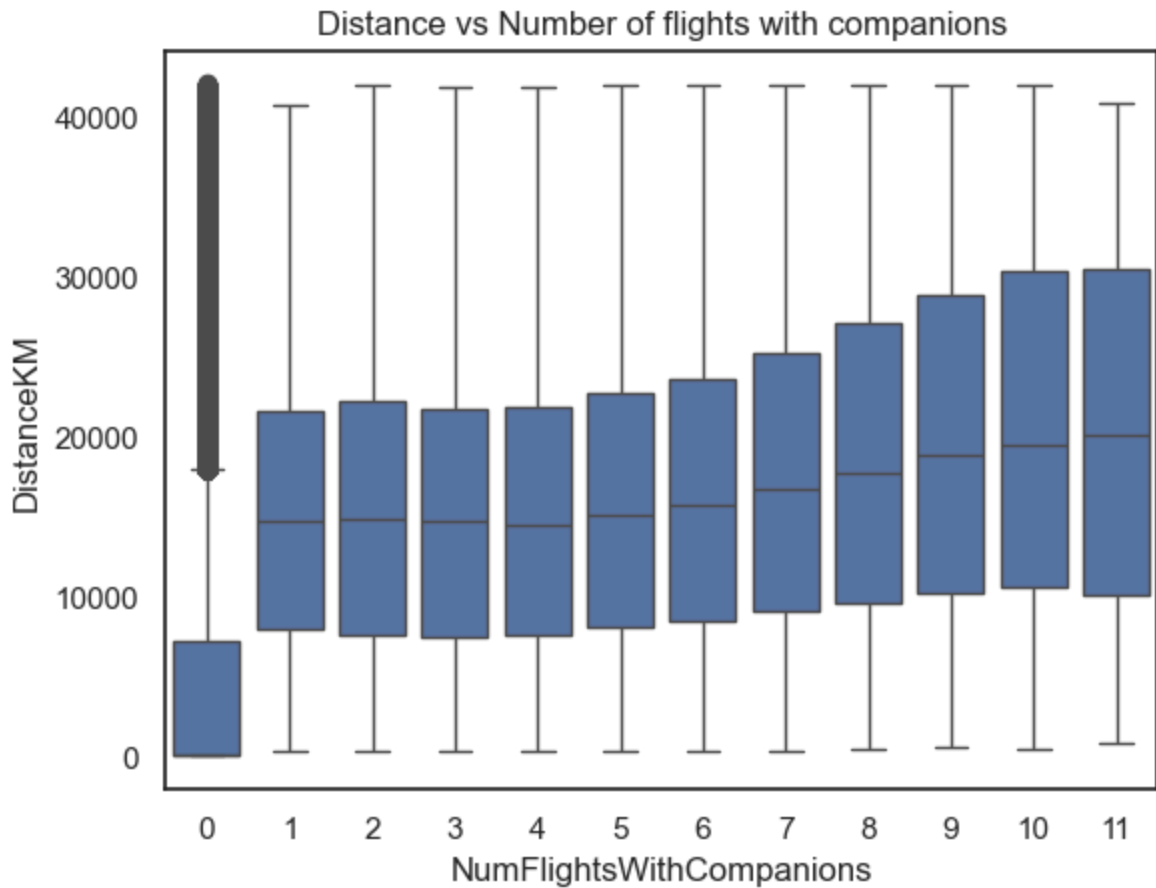


This bar chart shows the total number of flights per month. The number of flights increases steadily from the beginning of the year, peaking around July, and then slightly drops afterward. This pattern suggests strong seasonality, likely related to summer holidays and end-of-year travel.

2.4.4 Behavioral Insights

```
In [226... sns.boxplot(data=flights, x='NumFlightsWithCompanions', y='DistanceKM')  
plt.title('Distance vs Number of flights with companions')
```

```
Out[226... Text(0.5, 1.0, 'Distance vs Number of flights with companions')
```

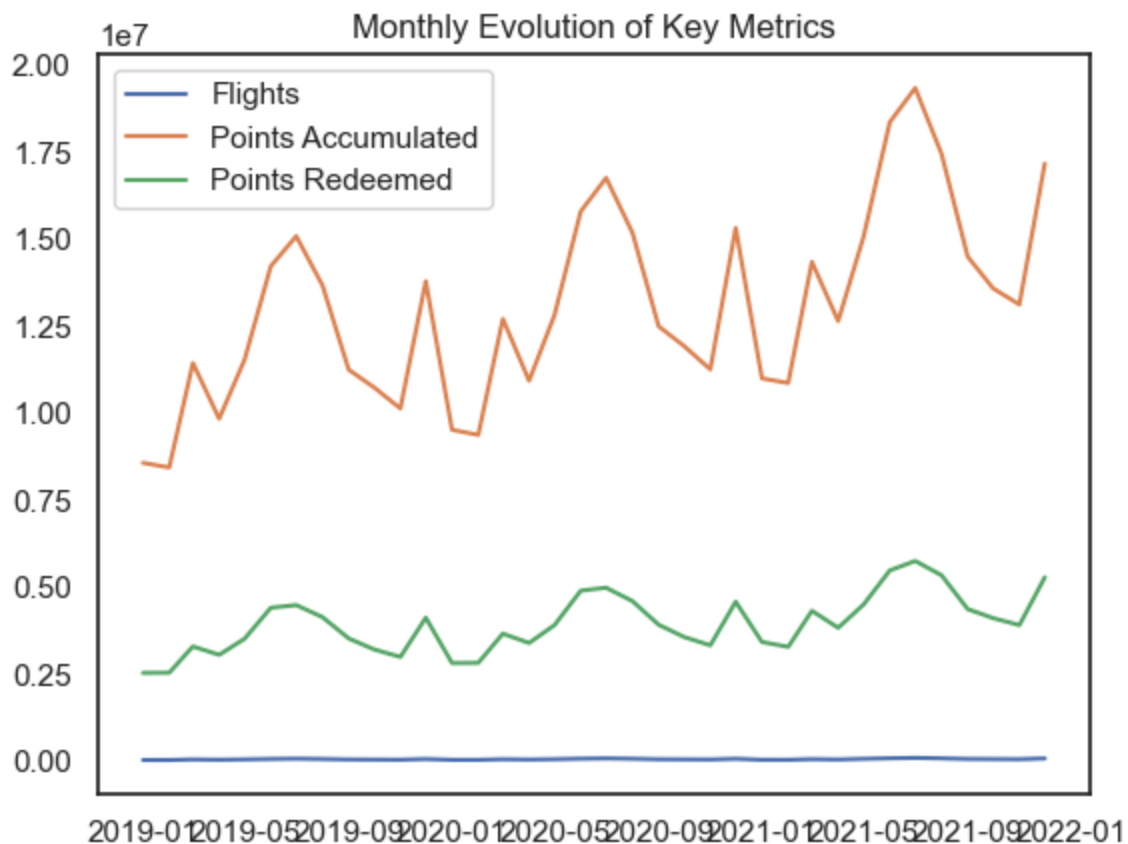


This boxplot examines how flight distance varies with the number of flights taken with companions. As the number of companion flights increases, the median distance also tends to rise. This could indicate that travelers who fly more frequently with others often take longer trips, such as vacations or group travel.

```
In [227... metrics_per_month = flights.groupby('YearMonthDate')[['NumFlights', 'PointsA

plt.plot(metrics_per_month['YearMonthDate'], metrics_per_month['NumFlights'])
plt.plot(metrics_per_month['YearMonthDate'], metrics_per_month['PointsAccumu
plt.plot(metrics_per_month['YearMonthDate'], metrics_per_month['PointsRedeen
plt.legend()
plt.title('Monthly Evolution of Key Metrics')
```

```
Out[227... Text(0.5, 1.0, 'Monthly Evolution of Key Metrics')
```



This line chart compares the monthly evolution of the three main metrics — total flights, points accumulated, and points redeemed. Points accumulated and redeemed show clear cyclical patterns over time, while flights remain relatively stable. The similar peaks in all lines suggest that flying activity and loyalty program engagement move together.

2.5. Feature Engineering

In [228... `# create new variables`

ActiveMonth

- Many customers have months with zero flights and zero points.
- This binary flag indicates whether a customer was active during that month, making it easier to analyze loyalty frequency and churn.

In [229... `#If the customer flew in a determined month`
`flights['ActiveMonth'] = (flights['NumFlights'] > 0).astype(int)`

RedeemRate

- The histogram showed that most customers never redeem points (PointsRedeemed = 0).
- This variable quantifies how much of the accumulated points are actually used, helping to distinguish engaged from inactive customers.

```
In [230...] flights['RedeemRate'] = flights['PointsRedeemed'] / flights['PointsAccumulat
```

AvgDistancePerFlight

- The scatterplot between NumFlights and DistanceKM showed some customers take many short flights, while others take few long flights.
- Calculating the average distance per flight helps identify travel patterns.
- It also reduces redundancy between two strongly correlated variables (correlation ≈ 0.62).

```
In [231...] flights['AvgDistancePerFlight'] = flights['DistanceKM'] / flights['NumFlight
```

CompanionshipRatio

- The boxplot showed that customers who travel more frequently with companions tend to take longer trips.
- This ratio measures the percentage of flights taken with others, which may indicate family or leisure travel.
- It can help segment customers into social vs solo travelers.

```
In [232...] flights['CompanionshipRatio'] = flights['NumFlightsWithCompanions'] / flight
```

ActivityLevel

- The heatmap and time-series plots revealed strong correlations between flights, points accumulated, and points redeemed.
- These can be combined into one overall activity index, representing a customer's engagement level in the loyalty program.

```
In [233...] scaler = MinMaxScaler()
flights[['NumFlights_scaled', 'PointsAccumulated_scaled', 'PointsRedeemed_scaled']] = scaler.fit_transform(
    flights[['NumFlights', 'PointsAccumulated', 'PointsRedeemed']]
)
flights['ActivityLevel'] = (flights['NumFlights_scaled'] + flights['PointsAccumulated_scaled'] + flights['PointsRedeemed_scaled'])
```

MonthSeason

- The bar plot of flights per month showed strong seasonality, with peaks in July and December.

- Grouping months into seasons (Winter, Spring, Summer, Autumn) captures this cyclical pattern and can be useful in time-based models.

```
In [234... def get_season(month):  
    if month in [12, 1, 2]:  
        return 'Winter'  
    elif month in [3, 4, 5]:  
        return 'Spring'  
    elif month in [6, 7, 8]:  
        return 'Summer'  
    else:  
        return 'Autumn'  
  
flights['MonthSeason'] = flights['Month'].apply(get_season)
```

```
In [235... flights.head()
```

```
Out[235...   Loyalty#  Year  Month  YearMonthDate  NumFlights  NumFlightsWithComp
```

0	413052	2021	12	2021-12-01	2	
1	464105	2021	12	2021-12-01	0	
2	681785	2021	12	2021-12-01	10	
3	185013	2021	12	2021-12-01	16	
4	216596	2021	12	2021-12-01	9	

```
In [236... flights.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 596676 entries, 0 to 596675
Data columns (total 19 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Loyalty#                             596676 non-null  int64
1   Year                                 596676 non-null  int64
2   Month                               596676 non-null  int64
3   YearMonthDate                        596676 non-null  datetime64[ns]
4   NumFlights                          596676 non-null  int64
5   NumFlightsWithCompanions            596676 non-null  int64
6   DistanceKM                          596676 non-null  float64
7   PointsAccumulated                   596676 non-null  float64
8   PointsRedeemed                      596676 non-null  float64
9   DollarCostPointsRedeemed            596676 non-null  float64
10  ActiveMonth                         596676 non-null  int64
11  RedeemRate                          301621 non-null  float64
12  AvgDistancePerFlight                301621 non-null  float64
13  CompanionshipRatio                  301621 non-null  float64
14  NumFlights_scaled                   596676 non-null  float64
15  PointsAccumulated_scaled            596676 non-null  float64
16  PointsRedeemed_scaled               596676 non-null  float64
17  ActivityLevel                       596676 non-null  float64
18  MonthSeason                         596676 non-null  object
dtypes: datetime64[ns](1), float64(11), int64(6), object(1)
memory usage: 86.5+ MB

```

```

In [237... flights.isna().sum()
          flights.isna().mean() * 100

```

```

Out[237... Loyalty#                0.000000
Year                0.000000
Month              0.000000
YearMonthDate      0.000000
NumFlights         0.000000
NumFlightsWithCompanions 0.000000
DistanceKM         0.000000
PointsAccumulated  0.000000
PointsRedeemed     0.000000
DollarCostPointsRedeemed 0.000000
ActiveMonth        0.000000
RedeemRate         49.449785
AvgDistancePerFlight 49.449785
CompanionshipRatio 49.449785
NumFlights_scaled  0.000000
PointsAccumulated_scaled 0.000000
PointsRedeemed_scaled 0.000000
ActivityLevel      0.000000
MonthSeason        0.000000
dtype: float64

```

2.5.1. Distribution Analysis of Engineered Features

In [238...

```
sns.set(style="whitegrid")

fig, axes = plt.subplots(2, 2, figsize=(12, 8))

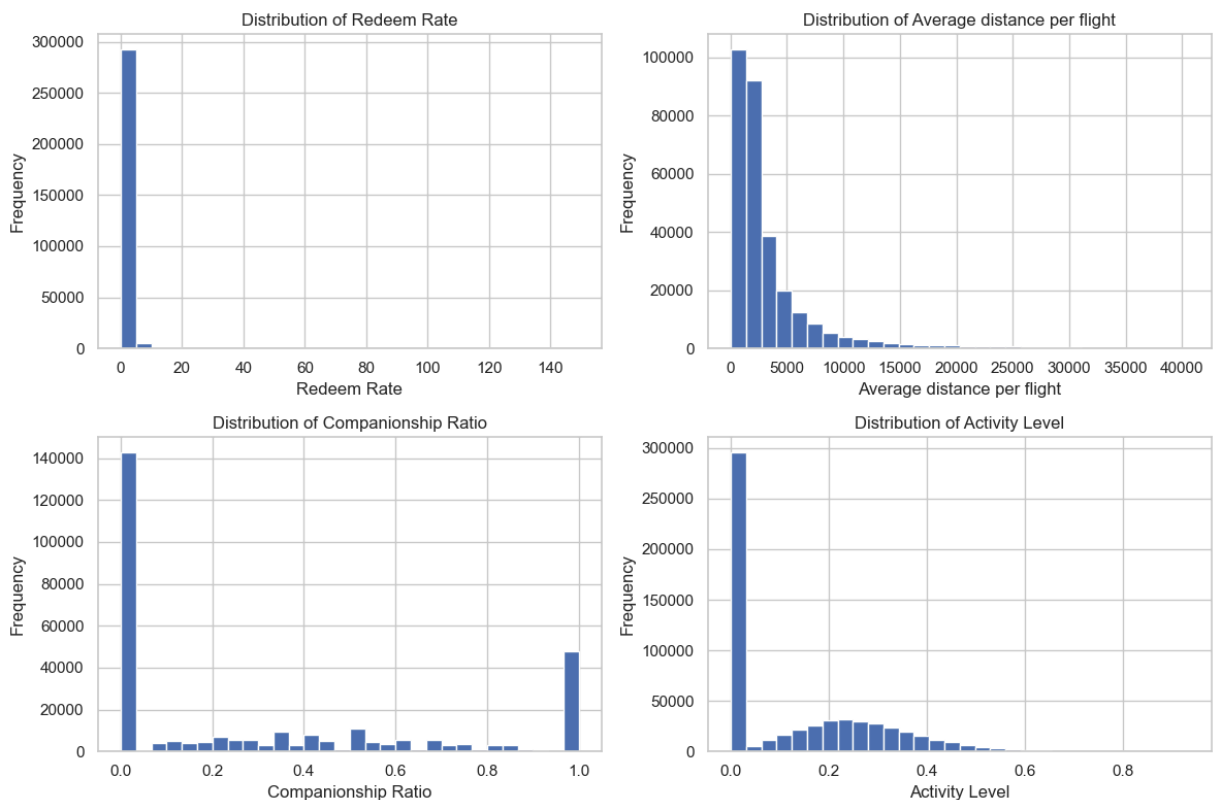
axes[0, 0].hist(flights['RedeemRate'], bins=30)
axes[0, 0].set_title('Distribution of Redeem Rate')
axes[0, 0].set_xlabel('Redeem Rate')
axes[0, 0].set_ylabel('Frequency')

axes[0, 1].hist(flights['AvgDistancePerFlight'], bins=30)
axes[0, 1].set_title('Distribution of Average distance per flight')
axes[0, 1].set_xlabel('Average distance per flight')
axes[0, 1].set_ylabel('Frequency')

axes[1, 0].hist(flights['CompanionshipRatio'], bins=30)
axes[1, 0].set_title('Distribution of Companionship Ratio')
axes[1, 0].set_xlabel('Companionship Ratio')
axes[1, 0].set_ylabel('Frequency')

axes[1, 1].hist(flights['ActivityLevel'], bins=30)
axes[1, 1].set_title('Distribution of Activity Level')
axes[1, 1].set_xlabel('Activity Level')
axes[1, 1].set_ylabel('Frequency')

plt.tight_layout()
plt.show()
```



The histograms show highly skewed distributions across all engineered variables. The Redeem Rate and Companionship Ratio are mostly concentrated near zero, meaning that most customers rarely redeem points or travel with companions.

The Average Distance per Flight also has a strong right skew, indicating that most flights cover short distances, with few long-haul outliers. The Activity Level presents a similar pattern, suggesting that a small group of customers shows consistently higher engagement.

```
In [239... sns.set(style="whitegrid")

fig, axes = plt.subplots(2, 2, figsize=(12, 8))

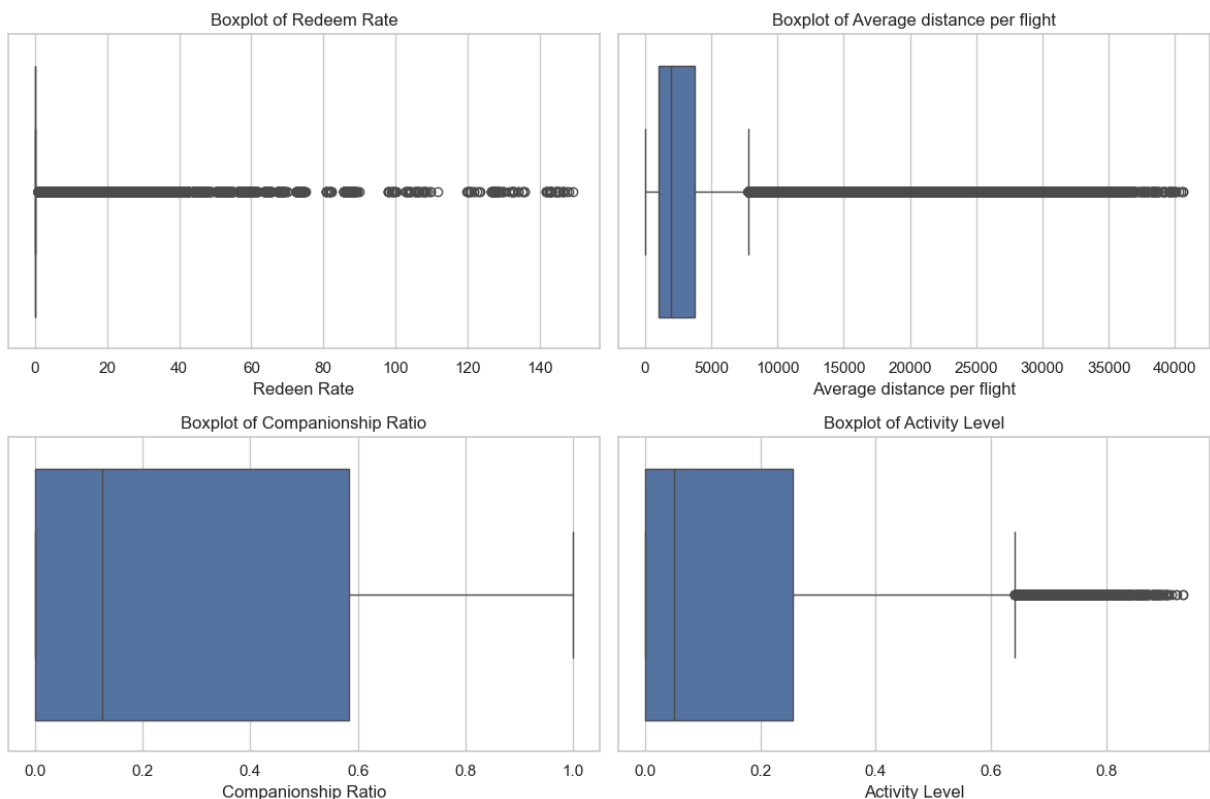
sns.boxplot(x=flights['RedeemRate'], ax=axes[0, 0])
axes[0, 0].set_title('Boxplot of Redeem Rate')
axes[0, 0].set_xlabel('Redeen Rate')

sns.boxplot(x=flights['AvgDistancePerFlight'], ax=axes[0, 1])
axes[0, 1].set_title('Boxplot of Average distance per flight')
axes[0, 1].set_xlabel('Average distance per flight')

sns.boxplot(x=flights['CompanionshipRatio'], ax=axes[1, 0])
axes[1, 0].set_title('Boxplot of Companionship Ratio')
axes[1, 0].set_xlabel('Companionship Ratio')

sns.boxplot(x=flights['ActivityLevel'], ax=axes[1, 1])
axes[1, 1].set_title('Boxplot of Activity Level')
axes[1, 1].set_xlabel('Activity Level')

plt.tight_layout()
plt.show()
```

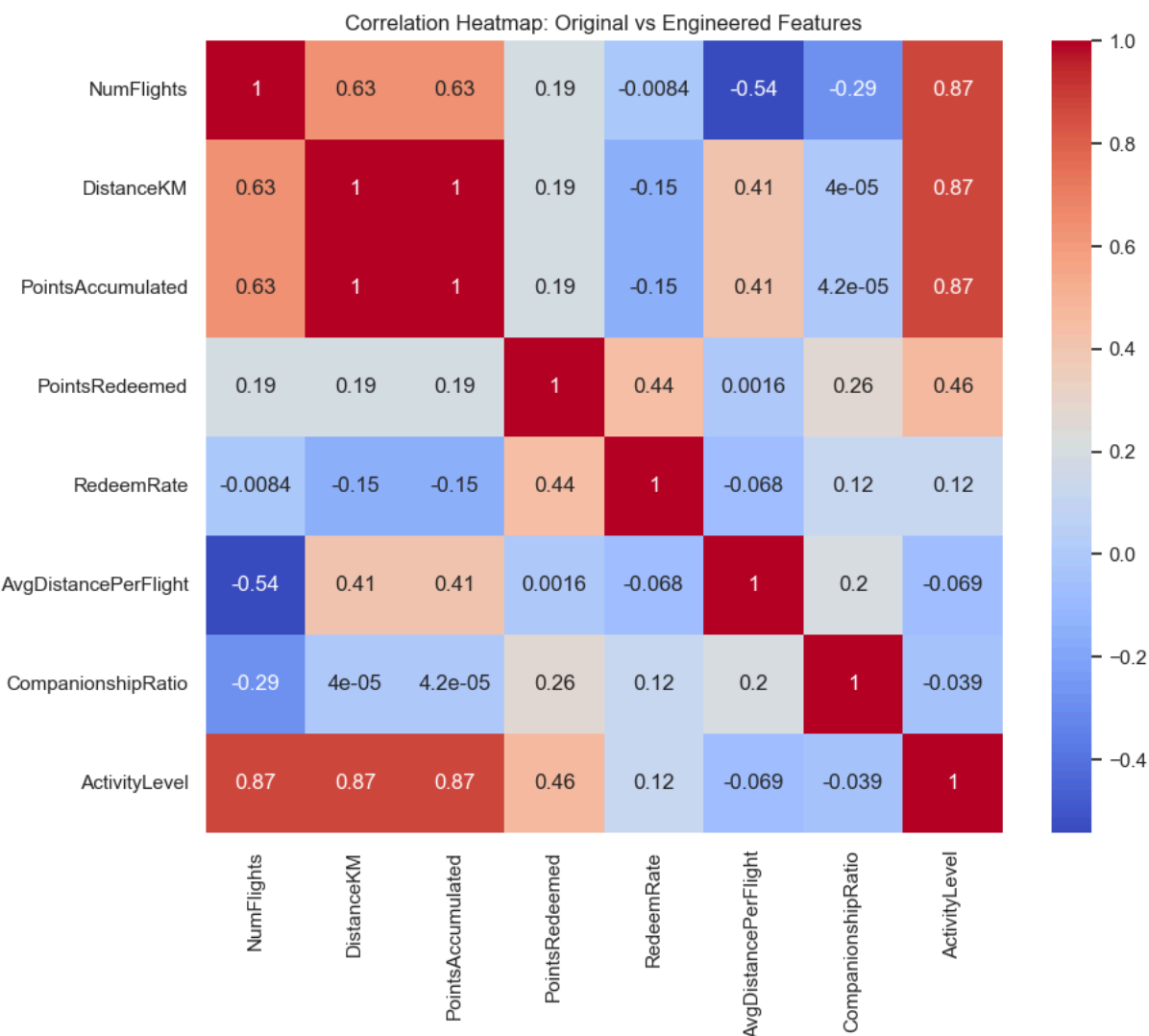


The boxplots confirm the presence of significant outliers, particularly in Redeem Rate and Average Distance per Flight. The interquartile range is narrow, meaning that most customers fall within a limited behavioral pattern. The wide spread of outliers indicates the existence of a small subset of highly active or exceptional customers who accumulate or redeem many points or travel much farther than the average.

2.5.2. Correlation between Original and Engineered Variables

```
In [240... selected = ['NumFlights', 'DistanceKM', 'PointsAccumulated', 'PointsRedeemed', 'RedeemRate', 'AvgDistancePerFlight', 'CompanionshipRatio', 'ActivityLevel']

plt.figure(figsize=(10, 8))
sns.heatmap(flights[selected].corr(), annot=True, cmap='coolwarm')
plt.title('Correlation Heatmap: Original vs Engineered Features')
plt.show()
```



The heatmap highlights strong correlations between NumFlights, DistanceKM, PointsAccumulated, and the new feature ActivityLevel, confirming that this derived variable effectively summarizes customer engagement. RedeemRate has weak correlations with most other features, which is expected since point redemption behavior is relatively independent of flight frequency or distance. AvgDistancePerFlight shows a negative relationship with NumFlights, suggesting that frequent travelers tend to take shorter trips.

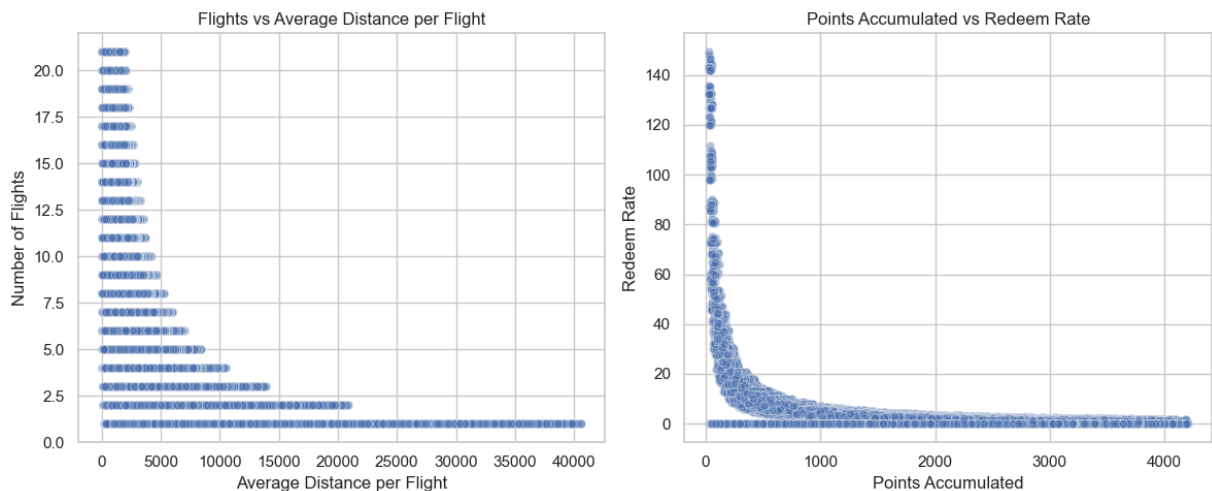
2.5.3. Relationships between Flights and Points Variables

```
In [241]: fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Scatter 1: AvgDistancePerFlight vs NumFlights
sns.scatterplot(data=flights, x='AvgDistancePerFlight', y='NumFlights',
                alpha=0.4, ax=axes[0])
axes[0].set_title('Flights vs Average Distance per Flight')
axes[0].set_xlabel('Average Distance per Flight')
axes[0].set_ylabel('Number of Flights')

# Scatter 2: PointsAccumulated vs RedeemRate
sns.scatterplot(data=flights, x='PointsAccumulated', y='RedeemRate',
                alpha=0.4, ax=axes[1])
axes[1].set_title('Points Accumulated vs Redeem Rate')
axes[1].set_xlabel('Points Accumulated')
axes[1].set_ylabel('Redeem Rate')

plt.tight_layout()
plt.show()
```



The first scatterplot shows an inverse relationship between AvgDistancePerFlight and NumFlights, supporting the idea that customers who fly frequently tend to take shorter routes. The second scatterplot reveals a decreasing trend between PointsAccumulated and RedeemRate — as customers accumulate more points,

the proportion of those points being redeemed declines. This may indicate point-saving behavior or delayed redemption among loyal customers.

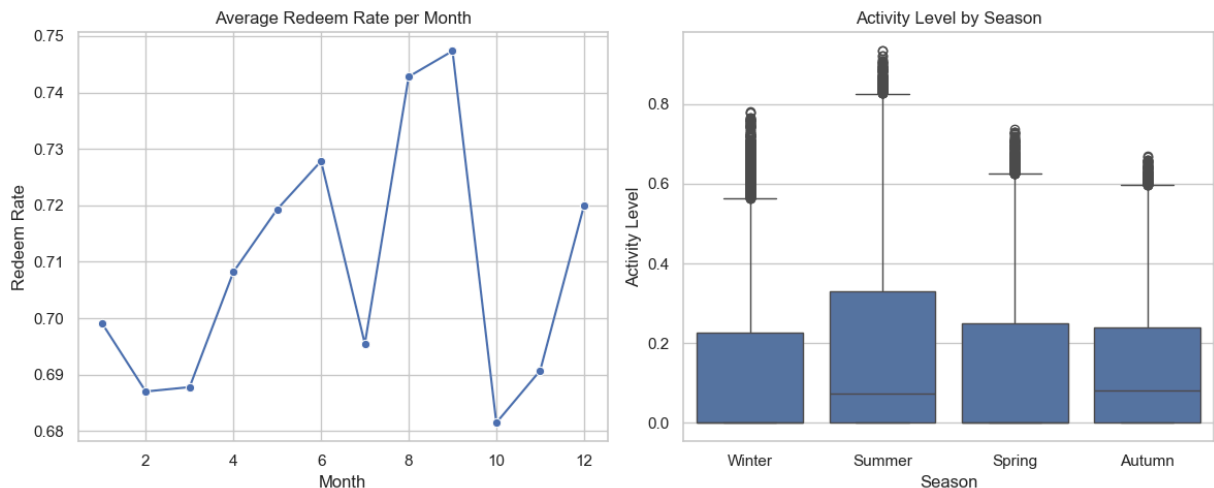
2.5.4. Temporal and Seasonal Behavior

```
In [242... fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Lineplot - Average Redeem Rate per Month
sns.lineplot(
    data=flights.groupby('Month')['RedeemRate'].mean().reset_index(),
    x='Month', y='RedeemRate', marker='o', ax=axes[0]
)
axes[0].set_title('Average Redeem Rate per Month')
axes[0].set_xlabel('Month')
axes[0].set_ylabel('Redeem Rate')

# Boxplot - Activity Level by Season
sns.boxplot(
    data=flights, x='MonthSeason', y='ActivityLevel', ax=axes[1]
)
axes[1].set_title('Activity Level by Season')
axes[1].set_xlabel('Season')
axes[1].set_ylabel('Activity Level')

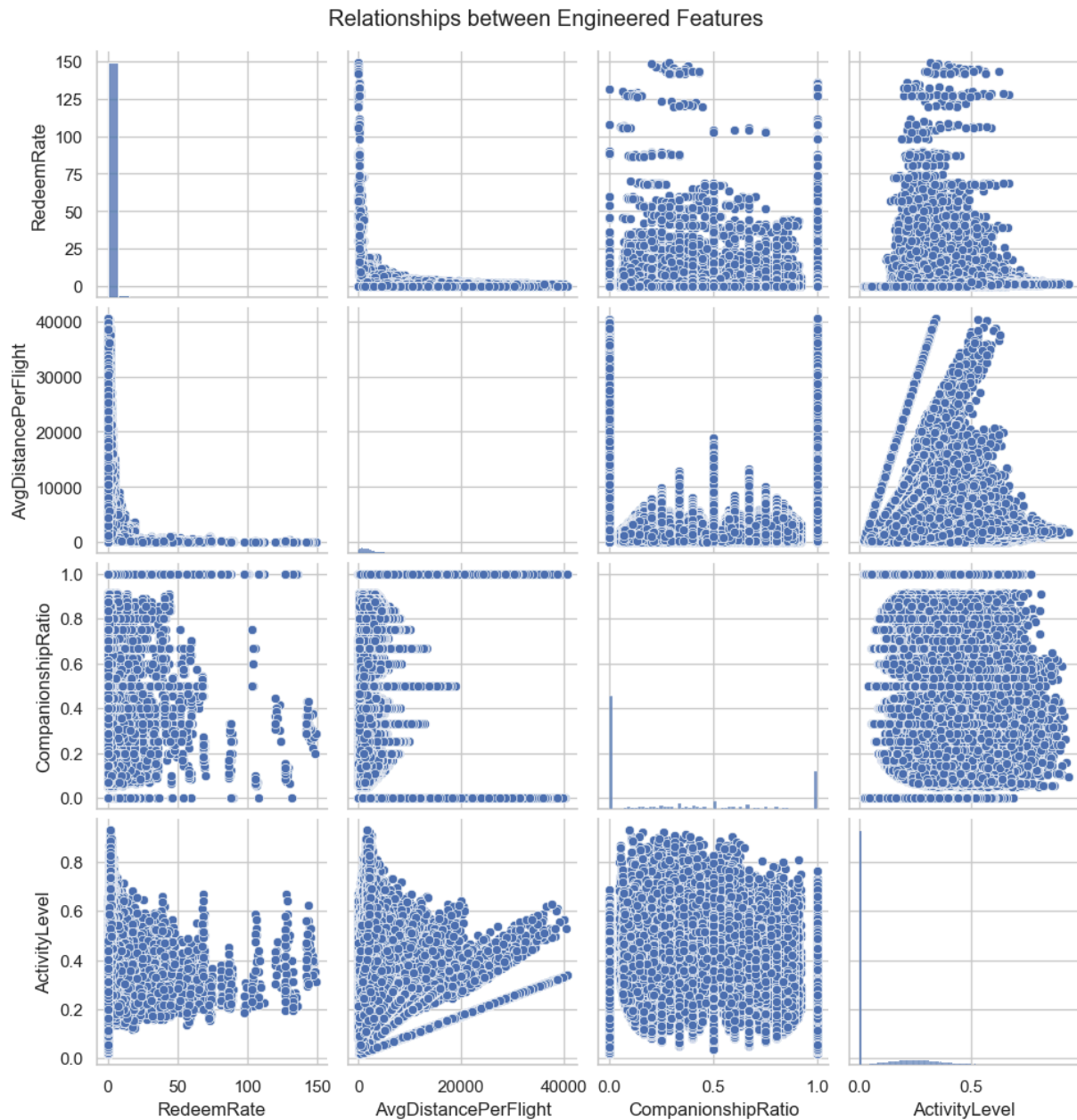
plt.tight_layout()
plt.show()
```



The Average Redeem Rate per Month shows small monthly fluctuations, with slight peaks during mid-year months, possibly associated with holiday travel or promotional campaigns. The Activity Level by Season boxplot indicates higher activity levels in the summer, consistent with increased flight frequency and engagement during vacation periods.

2.5.5. Interactions among Engineered Features

```
In [243... sns.pairplot(flights[['RedeemRate', 'AvgDistancePerFlight', 'CompanionshipRa
plt.suptitle('Relationships between Engineered Features', y=1.02)
plt.show()
```



The pairplot allows a broader view of how the engineered features relate to each other. The plots confirm that most variables are weakly correlated, meaning they capture distinct aspects of customer behavior. A slight inverse trend appears between AvgDistancePerFlight and ActivityLevel, showing that customers who fly shorter distances tend to be more active overall. The concentration of data points near zero in multiple plots reflects the predominance of low-engagement users in the dataset.

2.5.6. Treating Missing Values

Missing values in *RedeemRate*, *AvgDistancePerFlight*, and *CompanionshipRatio* occur logically for customers with no recorded flights or accumulated points in a given month. Instead of treating these as errors, they were replaced with zeros, reflecting the absence of activity rather than missing information. This approach ensures consistency with the variable *ActiveMonth*, which already identifies periods of inactivity.

```
In [244... flights[['RedeemRate', 'AvgDistancePerFlight', 'CompanionshipRatio']] = (
    flights[['RedeemRate', 'AvgDistancePerFlight', 'CompanionshipRatio']].fi
)
```

```
In [245... flights.isna().sum()
flights.isna().mean() * 100
```

```
Out[245... Loyalty#                0.0
Year                  0.0
Month                0.0
YearMonthDate        0.0
NumFlights            0.0
NumFlightsWithCompanions 0.0
DistanceKM           0.0
PointsAccumulated     0.0
PointsRedeemed        0.0
DollarCostPointsRedeemed 0.0
ActiveMonth           0.0
RedeemRate            0.0
AvgDistancePerFlight  0.0
CompanionshipRatio    0.0
NumFlights_scaled     0.0
PointsAccumulated_scaled 0.0
PointsRedeemed_scaled 0.0
ActivityLevel         0.0
MonthSeason           0.0
dtype: float64
```

3. Flights Dataset (After Aggregation)

3.1. Creating The New Dataset

```
In [246... # create dataset equal to the flights dataset but instead of having multiple
# we can only have one row for each customer

# i think we need to aggregate the rows in the flights dataset that refer to
# but we have to do this without removing the Month, Year, Date variables
```

```
In [247... flights.head()
```

Out[247... **Loyalty#** **Year** **Month** **YearMonthDate** **NumFlights** **NumFlightsWithCompa**

0	413052	2021	12	2021-12-01	2
1	464105	2021	12	2021-12-01	0
2	681785	2021	12	2021-12-01	10
3	185013	2021	12	2021-12-01	16
4	216596	2021	12	2021-12-01	9

In [248... *# Before aggregating, we remove all '_scaled' columns from the flights dataset
because scaling distorts the original numerical values.
Aggregation should be performed on the raw data to ensure meaningful total*
flights = flights.drop(columns=[col for col in flights.columns if '_scaled'

In [249... flights.head()

Out[249... **Loyalty#** **Year** **Month** **YearMonthDate** **NumFlights** **NumFlightsWithCompa**

0	413052	2021	12	2021-12-01	2
1	464105	2021	12	2021-12-01	0
2	681785	2021	12	2021-12-01	10
3	185013	2021	12	2021-12-01	16
4	216596	2021	12	2021-12-01	9

In [250... flights.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 596676 entries, 0 to 596675
Data columns (total 16 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Loyalty#                             596676 non-null  int64
1   Year                                 596676 non-null  int64
2   Month                               596676 non-null  int64
3   YearMonthDate                        596676 non-null  datetime64[ns]
4   NumFlights                           596676 non-null  int64
5   NumFlightsWithCompanions             596676 non-null  int64
6   DistanceKM                           596676 non-null  float64
7   PointsAccumulated                    596676 non-null  float64
8   PointsRedeemed                       596676 non-null  float64
9   DollarCostPointsRedeemed             596676 non-null  float64
10  ActiveMonth                           596676 non-null  int64
11  RedeemRate                           596676 non-null  float64
12  AvgDistancePerFlight                  596676 non-null  float64
13  CompanionshipRatio                   596676 non-null  float64
14  ActivityLevel                        596676 non-null  float64
15  MonthSeason                           596676 non-null  object
dtypes: datetime64[ns](1), float64(8), int64(6), object(1)
memory usage: 72.8+ MB
```

Variables Agregation

This aggregation step was performed to transform the dataset from a monthly flight-level structure into a customer-level summary, with one row per loyalty member. By grouping the data by customer and calculating totals (such as total flights, distance flown, and points accumulated) as well as averages (such as average distance per flight and average activity level), we obtain a compact and comprehensive view of each customer's overall travel behavior. This allows for easier comparison between customers and supports further analysis of loyalty patterns, engagement, and flight activity intensity.

```
In [251... # Aggregate flights dataset to have one row per customer (Loyalty#)
# We calculate key totals, averages, and activity indicators

flights_agg = flights.groupby('Loyalty#').agg({
    'NumFlights': ['sum', 'mean'],
    'NumFlightsWithCompanions': ['sum', 'mean'],
    'DistanceKM': ['sum', 'mean'],
    'PointsAccumulated': ['sum', 'mean'],
    'PointsRedeemed': ['sum', 'mean'],
    'DollarCostPointsRedeemed': ['sum', 'mean'],
    'RedeemRate': 'mean',
    'AvgDistancePerFlight': 'mean',
    'CompanionshipRatio': 'mean',
    'ActivityLevel': 'mean',
    'ActiveMonth': 'sum',
}).reset_index()

# Simplify column names
flights_agg.columns = ['Loyalty#',
                        'TotalFlights', 'AvgFlightsPerMonth',
                        'TotalFlightsWithCompanions', 'AvgFlightsWithCompanions',
                        'TotalDistanceKM', 'AvgDistanceKM',
                        'TotalPointsAccumulated', 'AvgPointsAccumulated',
                        'TotalPointsRedeemed', 'AvgPointsRedeemed',
                        'TotalDollarCostRedeemed', 'AvgDollarCostRedeemed',
                        'AvgRedeemRate', 'AvgDistancePerFlight',
                        'AvgCompanionshipRatio', 'AvgActivityLevel',
                        'TotalActiveMonths']

# Quick check
flights_agg.head()
```

```
Out[251... 

|   | Loyalty# | TotalFlights | AvgFlightsPerMonth | TotalFlightsWithCompanions | A |
|---|----------|--------------|--------------------|----------------------------|---|
| 0 | 100018   | 231          | 6.416667           | 50                         |   |
| 1 | 100102   | 248          | 6.888889           | 59                         |   |
| 2 | 100140   | 217          | 6.027778           | 54                         |   |
| 3 | 100214   | 112          | 3.111111           | 20                         |   |
| 4 | 100272   | 187          | 5.194444           | 53                         |   |


```

MonthsActive_PerYear

This step was implemented to replace the overall activity variable (TotalActiveMonths) with a more detailed year-by-year breakdown. By grouping the data by customer and year, we calculated the number of months each customer was active in 2019, 2020, and 2021. This approach provides a clearer view of the customer's flight consistency and engagement across different years, allowing for more accurate temporal analysis of flying behavior.

```
In [252... #split the column TotalActiveMonths per year
flights_agg = flights_agg.drop(columns=['TotalActiveMonths'])

#created new variables that return the nº of months a customer flies in a de
activity = flights.groupby(['Loyalty#', 'Year'])['ActiveMonth'].sum().unstack
activity.columns = ['monthsActive_2019', 'monthsActive_2020', 'monthsActive_2021']
flights_agg = flights_agg.merge(activity, on='Loyalty#', how='left')

flights_agg[['Loyalty#', 'monthsActive_2019', 'monthsActive_2020', 'monthsActive_2021']]
```

```
Out[252... Loyalty#  monthsActive_2019  monthsActive_2020  monthsActive_2021
```

0	100018	11	11	9
1	100102	8	8	9
2	100140	10	10	8
3	100214	7	7	7
4	100272	8	8	9

Flights_PerSeason

This step was carried out to analyze customer flight behavior by season. By grouping the dataset by customer and season, we calculated the total number of flights each customer took during autumn, spring, summer, and winter. The resulting seasonal flight variables (*Flights_Autumn*, *Flights_Spring*, *Flights_Summer*, and *Flights_Winter*) allow for a clearer understanding of seasonal travel patterns and preferences, enabling comparisons of activity intensity across different times of the year.

```
In [253... flights_season = flights.groupby(['Loyalty#', 'MonthSeason'])['NumFlights'].

# Rename columns to the desired format
flights_season = flights_season.rename(columns={
    'Autumn': 'Flights_Autumn',
    'Spring': 'Flights_Spring',
    'Summer': 'Flights_Summer',
    'Winter': 'Flights_Winter'
})

flights_agg = flights_agg.merge(flights_season, on='Loyalty#', how='left')
```

```
#check
```

```
flights_agg[['Loyalty#', 'Flights_Autumn', 'Flights_Spring', 'Flights_Summer', 'Flights_Winter']]
```

Out[253]...

	Loyalty#	Flights_Autumn	Flights_Spring	Flights_Summer	Flights_Winter
0	100018	84	47	55	45
1	100102	77	62	49	60
2	100140	62	58	48	49
3	100214	48	3	18	43
4	100272	22	64	70	31

Favorite Month and Season

This part of the analysis identifies each customer's preferred period to fly. First, the total number of flights per season and per month was calculated, allowing us to determine for each customer their PeakSeason (the season with the highest flight activity) and PeakMonth (the month with the highest number of flights). These variables provide insight into individual travel preferences and seasonality patterns. Finally, the variable MonthMatchesSeason was created to verify whether the customer's favorite month to fly belongs to their favorite season, enabling the identification of consistent or irregular travel behaviors across time.

In [254]...

```
season_totals = (
    flights.groupby(['Loyalty#', 'MonthSeason'], as_index=False)['NumFlights']
    .sum()
)

peak_season = (
    season_totals.loc[season_totals.groupby('Loyalty#')['NumFlights'].idxmax()
                     ['Loyalty#', 'MonthSeason']]
    .rename(columns={'MonthSeason': 'PeakSeason'})
)

flights['Month'] = flights['Month'].astype(int)

month_totals = (
    flights.groupby(['Loyalty#', 'Month'], as_index=False)['NumFlights']
    .sum()
)

peak_month = (
    month_totals.loc[month_totals.groupby('Loyalty#')['NumFlights'].idxmax()
                    ['Loyalty#', 'Month']]
    .rename(columns={'Month': 'PeakMonth'})
)

flights_agg = (
    flights_agg.merge(peak_season, on='Loyalty#', how='left')
    .merge(peak_month, on='Loyalty#', how='left')
)
```

```

#check if the favorite month to fly corresponds to the favorite season to fly
def match_season(row):
    if row['PeakSeason'] == 'Winter' and row['PeakMonth'] in [12, 1, 2]:
        return 1
    elif row['PeakSeason'] == 'Spring' and row['PeakMonth'] in [3, 4, 5]:
        return 1
    elif row['PeakSeason'] == 'Summer' and row['PeakMonth'] in [6, 7, 8]:
        return 1
    elif row['PeakSeason'] == 'Autumn' and row['PeakMonth'] in [9, 10, 11]:
        return 1
    else:
        return 0

flights_agg['MonthMatchesSeason'] = flights_agg.apply(match_season, axis=1)

flights_agg[['Loyalty#', 'PeakSeason', 'PeakMonth', 'MonthMatchesSeason']].h

```

Out[254]...

	Loyalty#	PeakSeason	PeakMonth	MonthMatchesSeason
0	100018	Autumn	11	1
1	100102	Autumn	12	0
2	100140	Autumn	9	1
3	100214	Autumn	12	0
4	100272	Summer	6	1

Flight Activity per Year

This step was performed to measure each customer's flight activity across different years. By aggregating the total number of flights per customer and per year, the code identifies whether the customer flew in 2019, 2020, or 2021. Binary variables were then created for each year, taking the value 1 if the customer flew and 0 otherwise. Additionally, the variable YearsActive was introduced to indicate the total number of years (from 0 to 3) in which the customer had flight activity. This provides a clear indicator of the customer's temporal consistency and engagement with the airline over time.

In [255]...

```

#Determine if the customer flew in each year (2019, 2020, 2021)
#We check whether total flights in that year > 0

#Aggregate total flights per customer and year
year_flights = flights.groupby(['Loyalty#', 'Year'], as_index=False)['NumFlights'].sum()

#Pivot to create one column per year
year_flights_pivot = year_flights.pivot(index='Loyalty#', columns='Year', values='NumFlights')

#Create binary columns for each year
year_flights_pivot['Flew_2019'] = (year_flights_pivot.get(2019, 0) > 0).astype(int)
year_flights_pivot['Flew_2020'] = (year_flights_pivot.get(2020, 0) > 0).astype(int)
year_flights_pivot['Flew_2021'] = (year_flights_pivot.get(2021, 0) > 0).astype(int)

```

```

#Count in how many years the customer flew
year_flights_pivot['YearsActive'] = (
    year_flights_pivot[['Flew_2019', 'Flew_2020', 'Flew_2021']].sum(axis=1)
)

#Keep only the relevant columns
year_flights_pivot = year_flights_pivot[['Loyalty#', 'Flew_2019', 'Flew_2020', 'Flew_2021']]

#Merge with the aggregated dataset
flights_agg = flights_agg.merge(year_flights_pivot, on='Loyalty#', how='left')

#Quick check
flights_agg[['Loyalty#', 'Flew_2019', 'Flew_2020', 'Flew_2021', 'YearsActive']]

```

```

Out[255...]

```

	Loyalty#	Flew_2019	Flew_2020	Flew_2021	YearsActive
0	100018	1	1	1	3
1	100102	1	1	1	3
2	100140	1	1	1	3
3	100214	1	1	1	3
4	100272	1	1	1	3

3.2. Exploratory Data Analysis

```

In [256...] pd.set_option('display.max_columns', None)
flights_agg.head()

```

```

Out[256...]

```

	Loyalty#	TotalFlights	AvgFlightsPerMonth	TotalFlightsWithCompanions	AvgFlightsPerMonthWithCompanions
0	100018	231	6.416667	50	1.604167
1	100102	248	6.888889	59	1.846091
2	100140	217	6.027778	54	1.488889
3	100214	112	3.111111	20	0.527778
4	100272	187	5.194444	53	1.338889

```

In [257...] flights_agg.shape

```

```

Out[257...] (16737, 31)

```

```

In [258...] flights_agg.columns

```

```
Out[258... Index(['Loyalty#', 'TotalFlights', 'AvgFlightsPerMonth',
                'TotalFlightsWithCompanions', 'AvgFlightsWithCompanions',
                'TotalDistanceKM', 'AvgDistanceKM', 'TotalPointsAccumulated',
                'AvgPointsAccumulated', 'TotalPointsRedeemed', 'AvgPointsRedeemed',
                'TotalDollarCostRedeemed', 'AvgDollarCostRedeemed', 'AvgRedeemRate',
                'AvgDistancePerFlight', 'AvgCompanionshipRatio', 'AvgActivityLevel',
                'monthsActive_2019', 'monthsActive_2020', 'monthsActive_2021',
                'Flights_Autumn', 'Flights_Spring', 'Flights_Summer', 'Flights_Winte
r',
                'PeakSeason', 'PeakMonth', 'MonthMatchesSeason', 'Flew_2019',
                'Flew_2020', 'Flew_2021', 'YearsActive'],
                dtype='object')
```

```
In [259... flights_agg.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 16737 entries, 0 to 16736
Data columns (total 31 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Loyalty#                             16737 non-null  int64
1   TotalFlights                         16737 non-null  int64
2   AvgFlightsPerMonth                   16737 non-null  float64
3   TotalFlightsWithCompanions           16737 non-null  int64
4   AvgFlightsWithCompanions             16737 non-null  float64
5   TotalDistanceKM                      16737 non-null  float64
6   AvgDistanceKM                       16737 non-null  float64
7   TotalPointsAccumulated               16737 non-null  float64
8   AvgPointsAccumulated                16737 non-null  float64
9   TotalPointsRedeemed                 16737 non-null  float64
10  AvgPointsRedeemed                   16737 non-null  float64
11  TotalDollarCostRedeemed              16737 non-null  float64
12  AvgDollarCostRedeemed                16737 non-null  float64
13  AvgRedeemRate                       16737 non-null  float64
14  AvgDistancePerFlight                 16737 non-null  float64
15  AvgCompanionshipRatio                16737 non-null  float64
16  AvgActivityLevel                    16737 non-null  float64
17  monthsActive_2019                   16737 non-null  int64
18  monthsActive_2020                   16737 non-null  int64
19  monthsActive_2021                   16737 non-null  int64
20  Flights_Autumn                      16737 non-null  int64
21  Flights_Spring                      16737 non-null  int64
22  Flights_Summer                      16737 non-null  int64
23  Flights_Winter                      16737 non-null  int64
24  PeakSeason                          16737 non-null  object
25  PeakMonth                           16737 non-null  int64
26  MonthMatchesSeason                  16737 non-null  int64
27  Flew_2019                           16737 non-null  int64
28  Flew_2020                           16737 non-null  int64
29  Flew_2021                           16737 non-null  int64
30  YearsActive                         16737 non-null  int64
dtypes: float64(14), int64(16), object(1)
memory usage: 4.0+ MB
```

```
In [260... flights_agg.dtypes
```

```

Out[260... Loyalty#                int64
TotalFlights            int64
AvgFlightsPerMonth      float64
TotalFlightsWithCompanions int64
AvgFlightsWithCompanions float64
TotalDistanceKM         float64
AvgDistanceKM           float64
TotalPointsAccumulated  float64
AvgPointsAccumulated    float64
TotalPointsRedeemed     float64
AvgPointsRedeemed       float64
TotalDollarCostRedeemed float64
AvgDollarCostRedeemed   float64
AvgRedeemRate           float64
AvgDistancePerFlight    float64
AvgCompanionshipRatio   float64
AvgActivityLevel        float64
monthsActive_2019       int64
monthsActive_2020       int64
monthsActive_2021       int64
Flights_Autumn          int64
Flights_Spring          int64
Flights_Summer          int64
Flights_Winter          int64
PeakSeason              object
PeakMonth               int64
MonthMatchesSeason      int64
Flew_2019               int64
Flew_2020               int64
Flew_2021               int64
YearsActive             int64
dtype: object

```

- Variable *PeakSeason* must be categorical
- Transform Binary Variables into int8 ?? what does this means??

```

In [261... id_col = ['Loyalty#']
categorical_cols = ['PeakSeason']
binary_cols = ['MonthMatchesSeason', 'Flew_2019', 'Flew_2020', 'Flew_2021']
numeric_cols = [col for col in flights_agg.columns if col not in id_col + ca

```

```

In [262... flights_agg[numeric_cols].describe()

```

Out[262...

	TotalFlights	AvgFlightsPerMonth	TotalFlightsWithCompanions	AvgFli
count	16737.000000	16737.000000	16737.000000	
mean	141.118898	3.969215	35.574954	
std	80.750753	2.274265	23.409824	
min	0.000000	0.000000	0.000000	
25%	70.000000	1.944444	16.000000	
50%	163.000000	4.583333	37.000000	
75%	203.000000	5.722222	53.000000	
max	349.000000	9.694444	124.000000	

The descriptive statistics summarize the main characteristics of the aggregated flight dataset. Overall, customers took an average of 141 flights, traveling about 286,000 km in total, which corresponds to roughly 7,950 km per flight. The number of flights per customer varies substantially (from 0 to 349), indicating different engagement levels with the loyalty program.

On average, customers earned around 28,600 points and redeemed approximately 8,500 points, showing that redemption behavior is less frequent than accumulation. The average redeem rate (0.35) also supports this pattern, suggesting that many customers accumulate points without using them regularly.

Seasonally, activity is relatively balanced, with a slightly higher concentration of flights during the summer months (≈ 45 flights). Most customers were active for 2 to 3 years between 2019 and 2021, with an average of around 6 active months per year.

These results confirm a heterogeneous customer base, with a mix of highly frequent flyers and less active participants, and reveal distinct temporal and seasonal travel patterns that may later influence the clustering analysis.

In [263...

```
flights_agg['PeakSeason'].value_counts(normalize=True)
flights_agg[binary_cols].mean()
```

Out[263...

```
MonthMatchesSeason    0.645396
Flew_2019              0.766744
Flew_2020              0.766744
Flew_2021              0.876800
dtype: float64
```

The categorical and binary variables provide additional insights into customers' flying behavior. Approximately 65% of customers have their favorite month of travel matching their favorite season, indicating consistent seasonal flying preferences. Regarding yearly activity, around 77% of customers flew in both

2019 and 2020, and nearly 88% remained active in 2021, suggesting strong program engagement and customer retention over time. These proportions confirm that most loyalty members are recurrent travelers with stable travel patterns across the analyzed years.

3.2.1. Checking For Missing Values

```
In [264...] flights_agg.isna().sum()
```

```
Out[264...] Loyalty#                0
TotalFlights            0
AvgFlightsPerMonth      0
TotalFlightsWithCompanions 0
AvgFlightsWithCompanions 0
TotalDistanceKM         0
AvgDistanceKM           0
TotalPointsAccumulated  0
AvgPointsAccumulated    0
TotalPointsRedeemed     0
AvgPointsRedeemed       0
TotalDollarCostRedeemed 0
AvgDollarCostRedeemed   0
AvgRedeemRate           0
AvgDistancePerFlight    0
AvgCompanionshipRatio   0
AvgActivityLevel        0
monthsActive_2019       0
monthsActive_2020       0
monthsActive_2021       0
Flights_Autumn          0
Flights_Spring          0
Flights_Summer          0
Flights_Winter          0
PeakSeason              0
PeakMonth               0
MonthMatchesSeason      0
Flew_2019               0
Flew_2020               0
Flew_2021               0
YearsActive             0
dtype: int64
```

- There is no missing values

3.2.2. Checking For Duplicated Values

```
In [265...] flights_agg.duplicated().sum()
```

```
Out[265...] np.int64(0)
```

```
In [266... flights_agg['Loyalty#'].duplicated().sum()
```

```
Out[266... np.int64(0)
```

3.3. Initial Data Preparation

```
In [267... # Deleting variables that aren't relevant (probably won't be necessary), cor  
# be wrong, etc
```

```
In [268... flights_agg['PeakSeason'] = flights_agg['PeakSeason'].astype('category')  
flights_agg[binary_cols] = flights_agg[binary_cols].astype('int8')
```

```
flights_agg.dtypes
```

```
Out[268... Loyalty#                int64  
TotalFlights            int64  
AvgFlightsPerMonth      float64  
TotalFlightsWithCompanions  int64  
AvgFlightsWithCompanions  float64  
TotalDistanceKM         float64  
AvgDistanceKM           float64  
TotalPointsAccumulated   float64  
AvgPointsAccumulated     float64  
TotalPointsRedeemed      float64  
AvgPointsRedeemed        float64  
TotalDollarCostRedeemed  float64  
AvgDollarCostRedeemed    float64  
AvgRedeemRate            float64  
AvgDistancePerFlight     float64  
AvgCompanionshipRatio    float64  
AvgActivityLevel         float64  
monthsActive_2019        int64  
monthsActive_2020        int64  
monthsActive_2021        int64  
Flights_Autumn           int64  
Flights_Spring           int64  
Flights_Summer           int64  
Flights_Winter           int64  
PeakSeason               category  
PeakMonth                int64  
MonthMatchesSeason       int8  
Flew_2019                 int8  
Flew_2020                 int8  
Flew_2021                 int8  
YearsActive              int64  
dtype: object
```

Identify numeric / categorical / binary columns

```
In [269... # Identify variable types more accurately
```

```
num_cols = [c for c in flights_agg.columns
```

```

if flights_agg[c].dtype in ['float64', 'int64']
and flights_agg[c].nunique() > 5] # exclude binary-like columns

binary_cols = [c for c in flights_agg.columns
                if flights_agg[c].nunique() == 2] # only 0/1 type columns

cat_cols = [c for c in flights_agg.columns
            if flights_agg[c].dtype == 'object' or str(flights_agg[c].dtype)

print("Numeric columns:", num_cols)
print("Binary columns:", binary_cols)
print("Categorical columns:", cat_cols)

```

Numeric columns: ['Loyalty#', 'TotalFlights', 'AvgFlightsPerMonth', 'TotalFlightsWithCompanions', 'AvgFlightsWithCompanions', 'TotalDistanceKM', 'AvgDistanceKM', 'TotalPointsAccumulated', 'AvgPointsAccumulated', 'TotalPointsRedeemed', 'AvgPointsRedeemed', 'TotalDollarCostRedeemed', 'AvgDollarCostRedeemed', 'AvgRedeemRate', 'AvgDistancePerFlight', 'AvgCompanionshipRatio', 'AvgActivityLevel', 'monthsActive_2019', 'monthsActive_2020', 'monthsActive_2021', 'Flights_Autumn', 'Flights_Spring', 'Flights_Summer', 'Flights_Winter', 'PeakMonth']

Binary columns: ['MonthMatchesSeason', 'Flew_2019', 'Flew_2020', 'Flew_2021']

Categorical columns: ['PeakSeason']

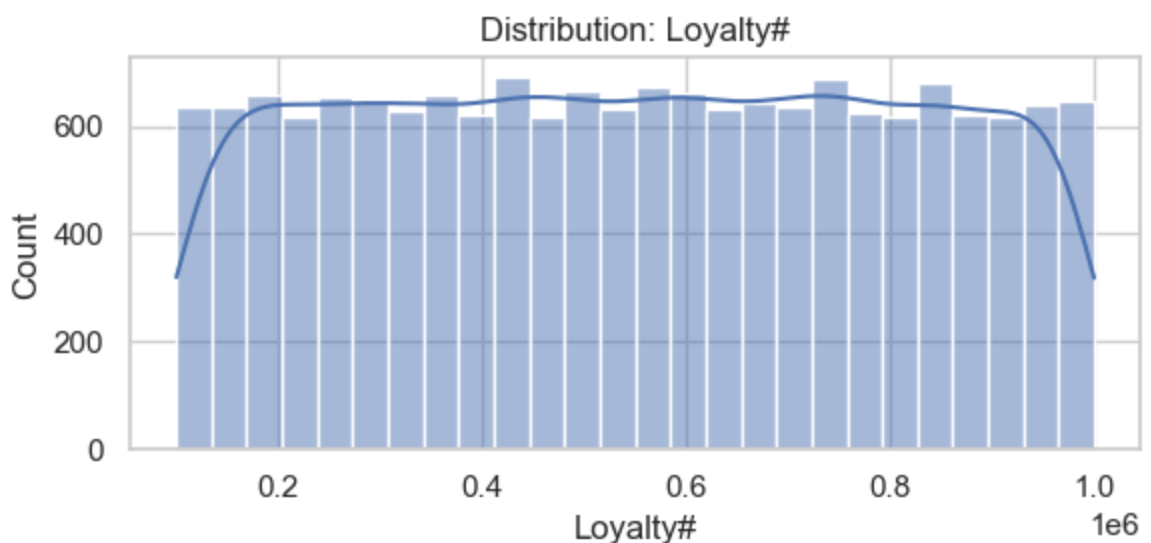
3.4. Visual Exploration

Distributions

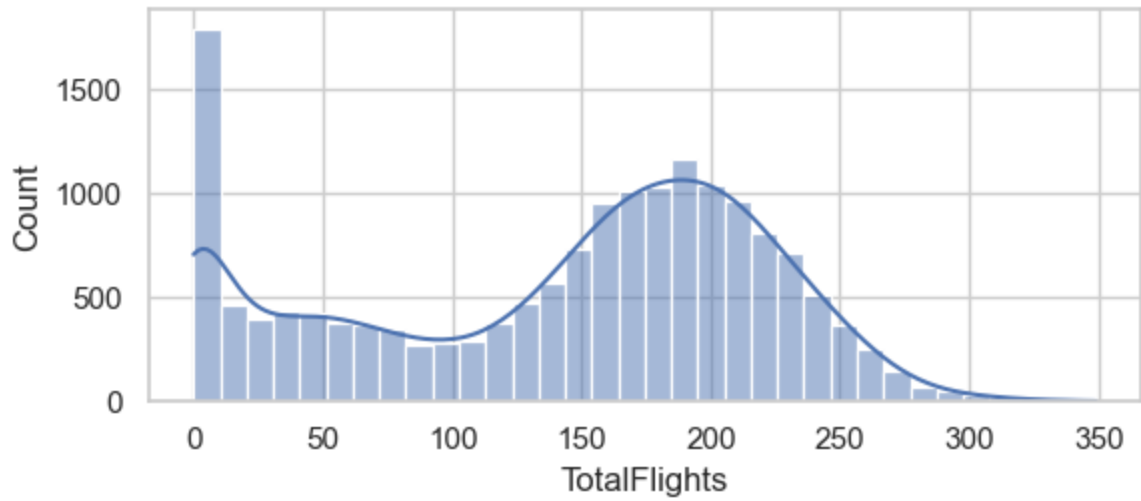
```

In [270... for c in num_cols:
    plt.figure(figsize=(6,3))
    sns.histplot(flights_agg[c].dropna(), kde=True)
    plt.title(f"Distribution: {c}")
    plt.tight_layout()
    plt.show()

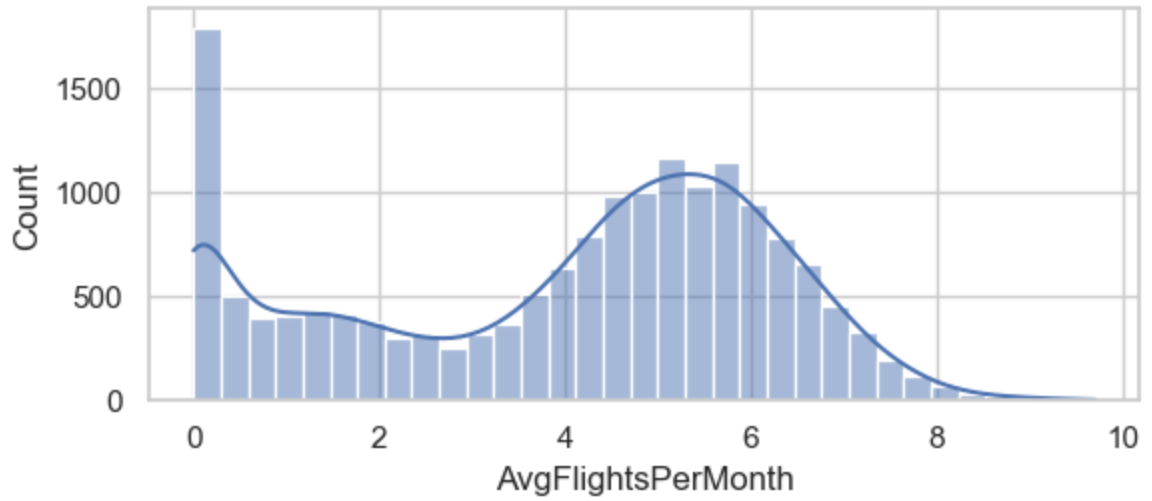
```



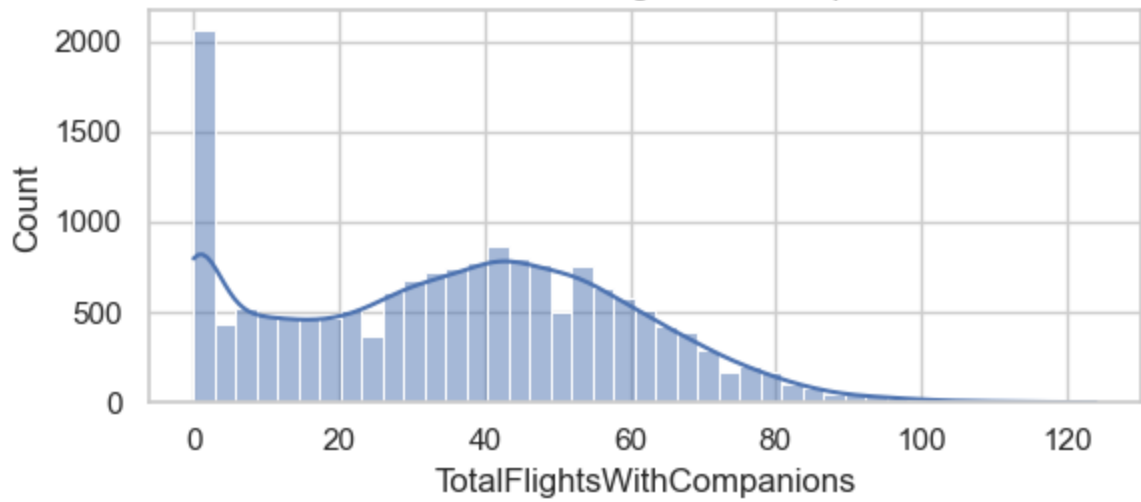
Distribution: TotalFlights

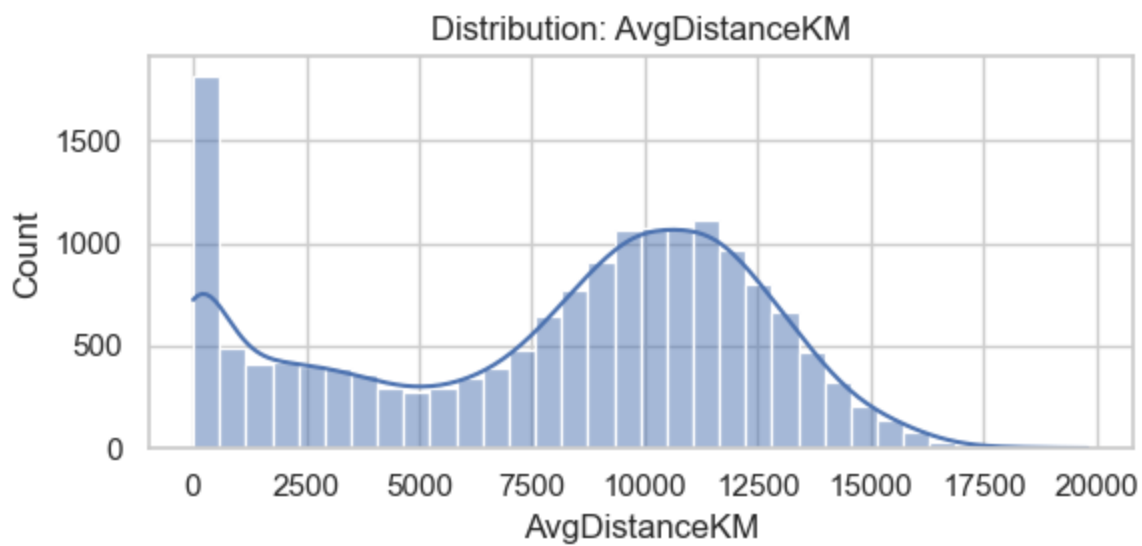
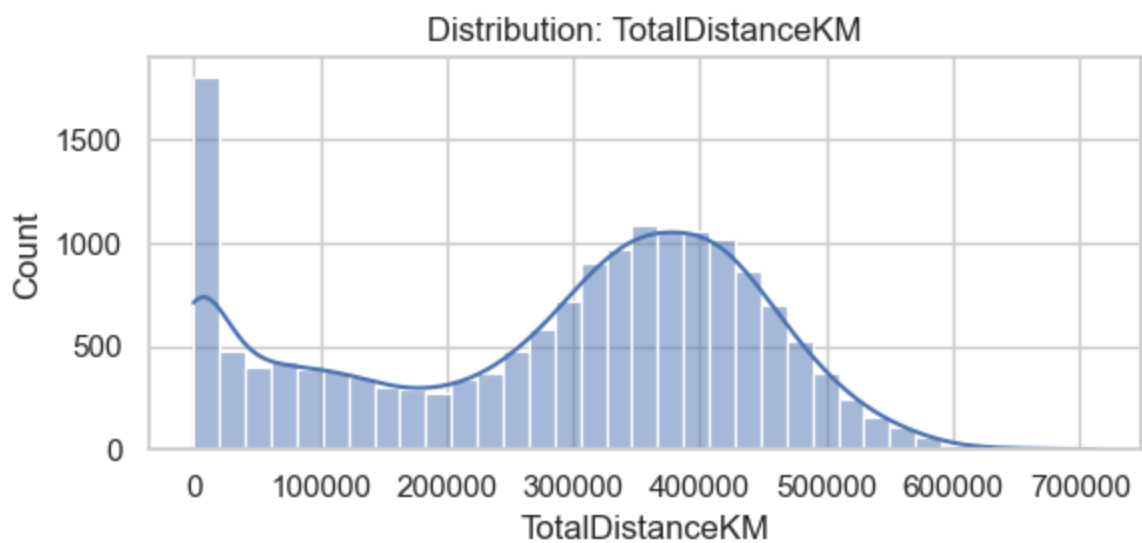
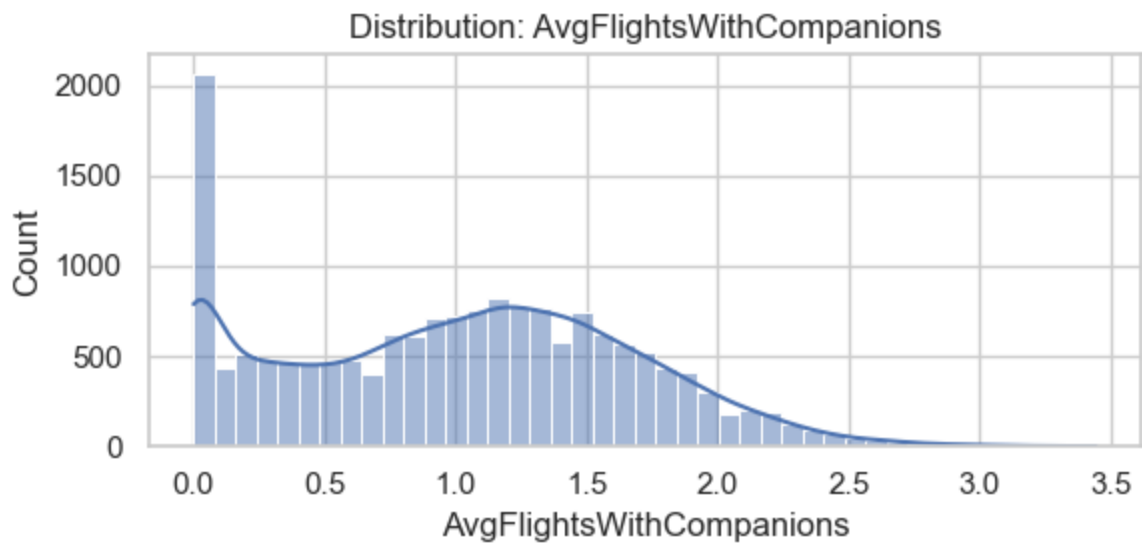


Distribution: AvgFlightsPerMonth

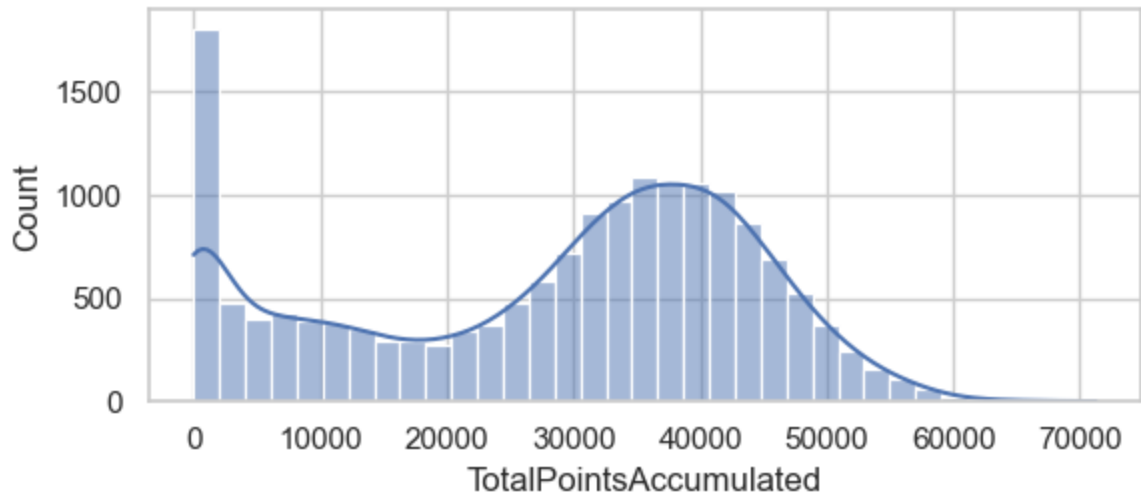


Distribution: TotalFlightsWithCompanions

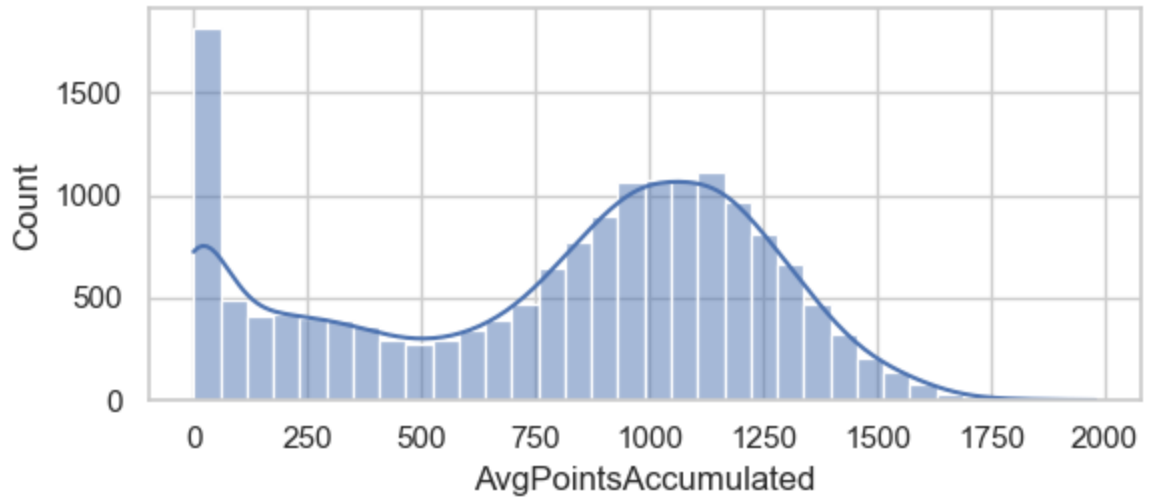




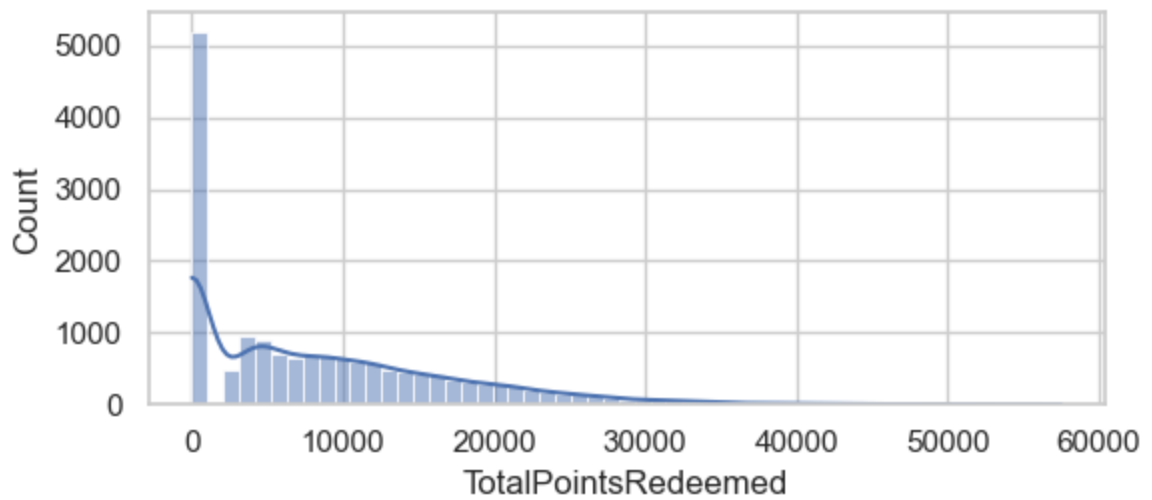
Distribution: TotalPointsAccumulated



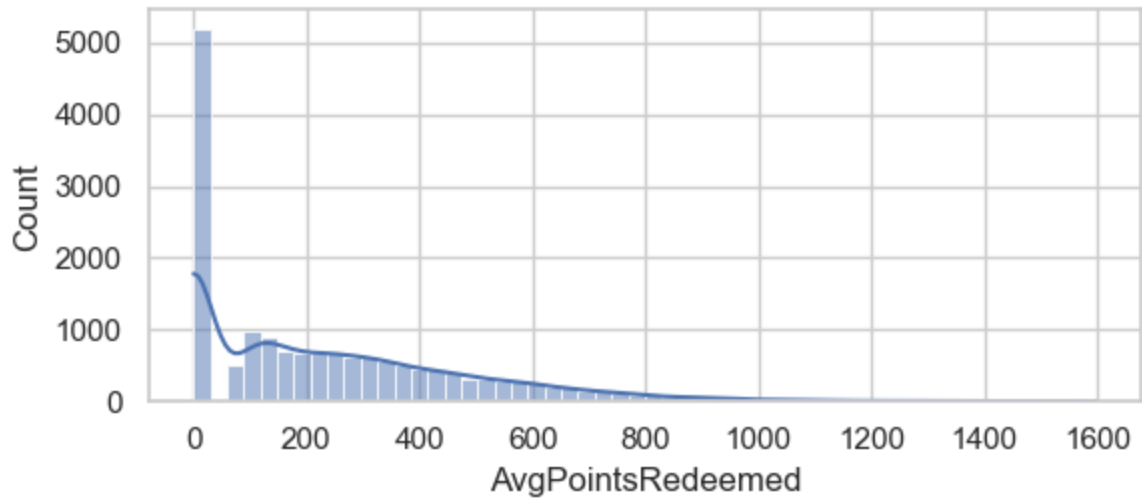
Distribution: AvgPointsAccumulated



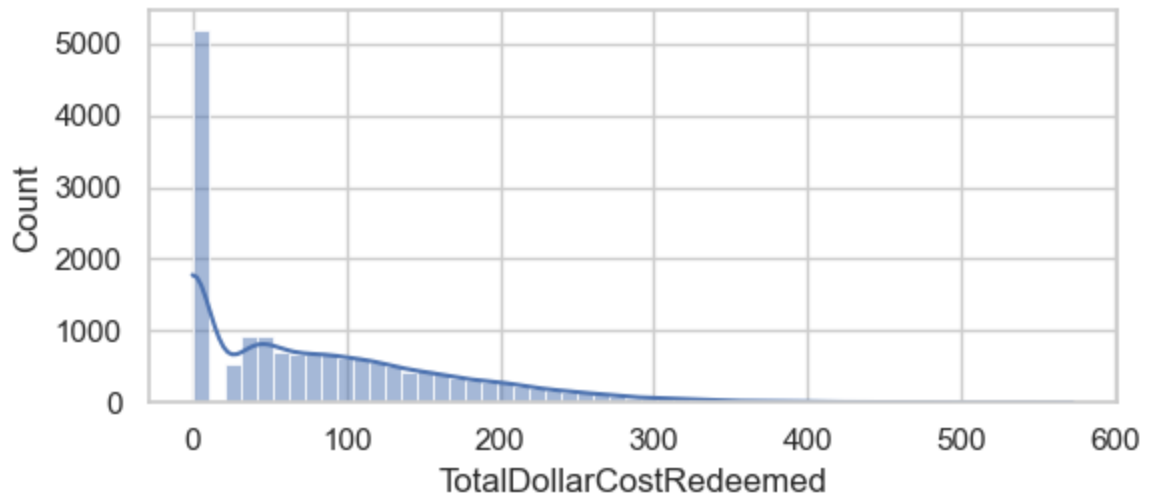
Distribution: TotalPointsRedeemed



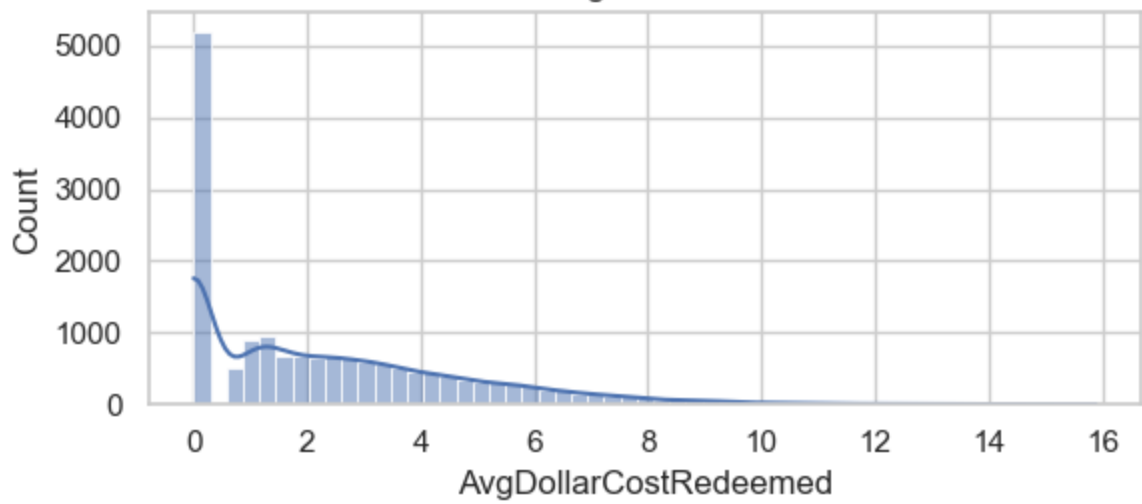
Distribution: AvgPointsRedeemed

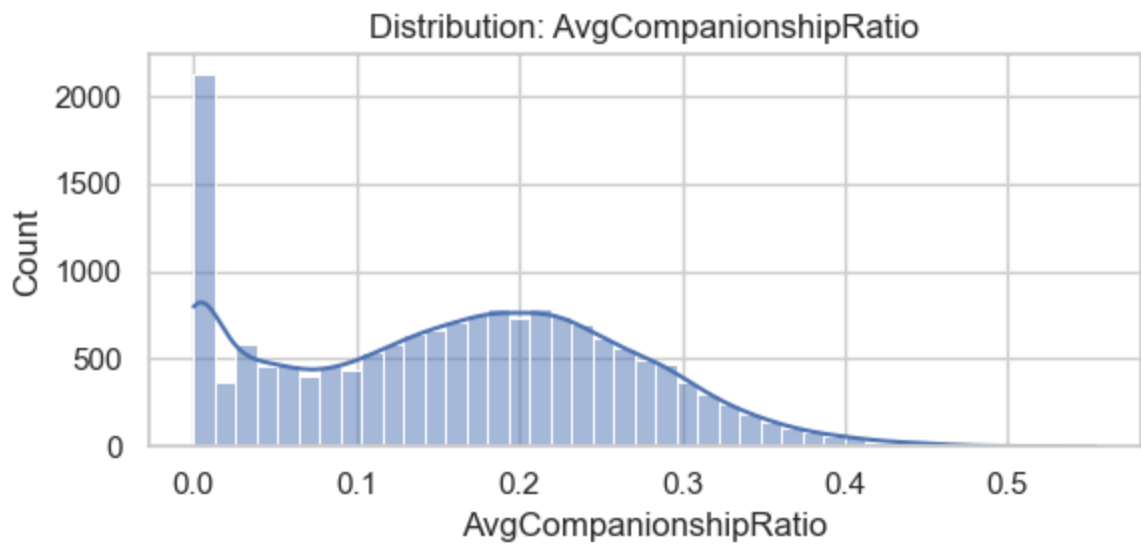
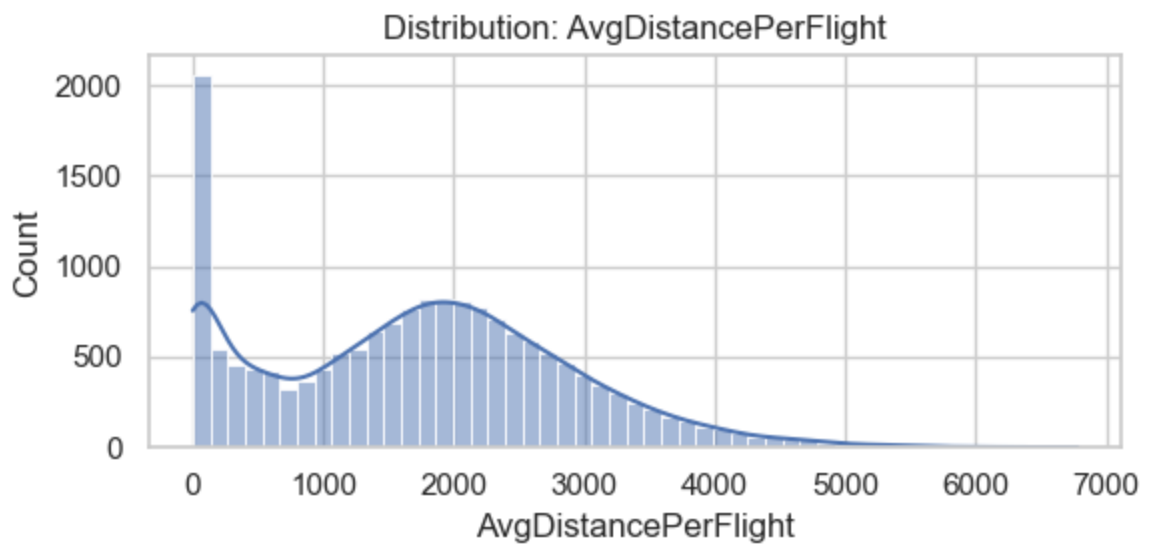
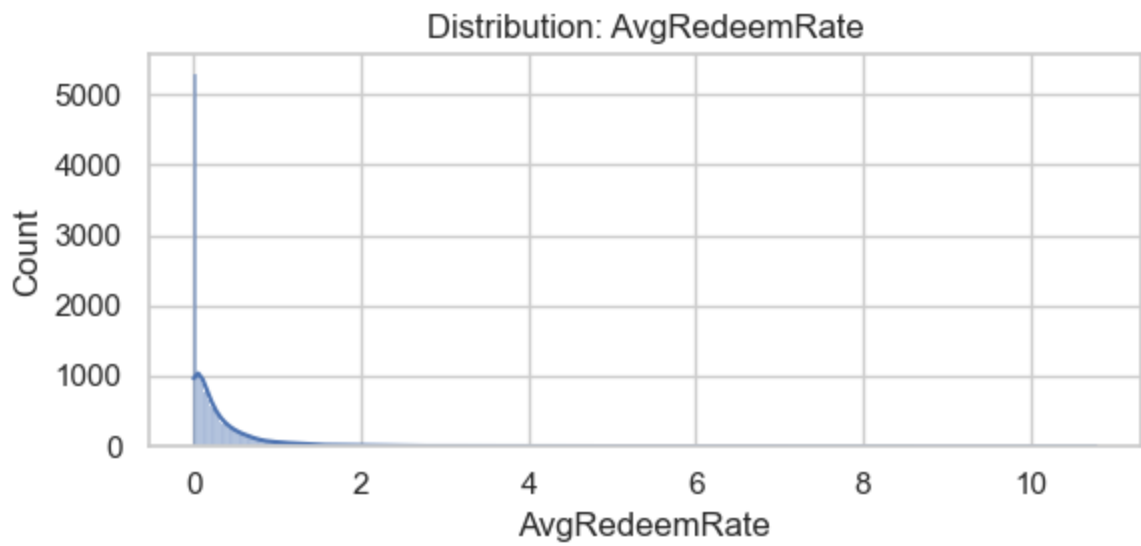


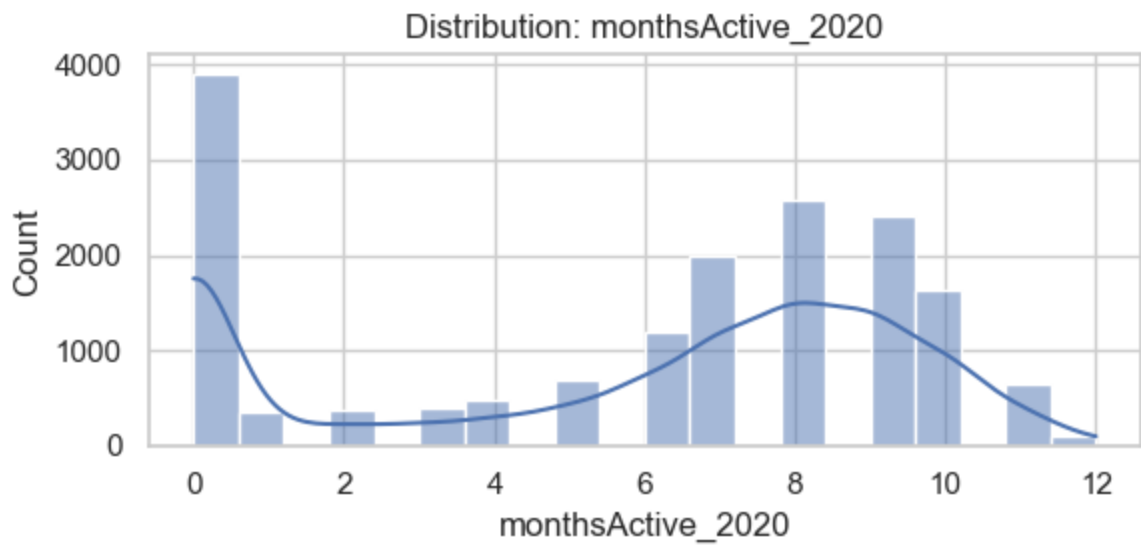
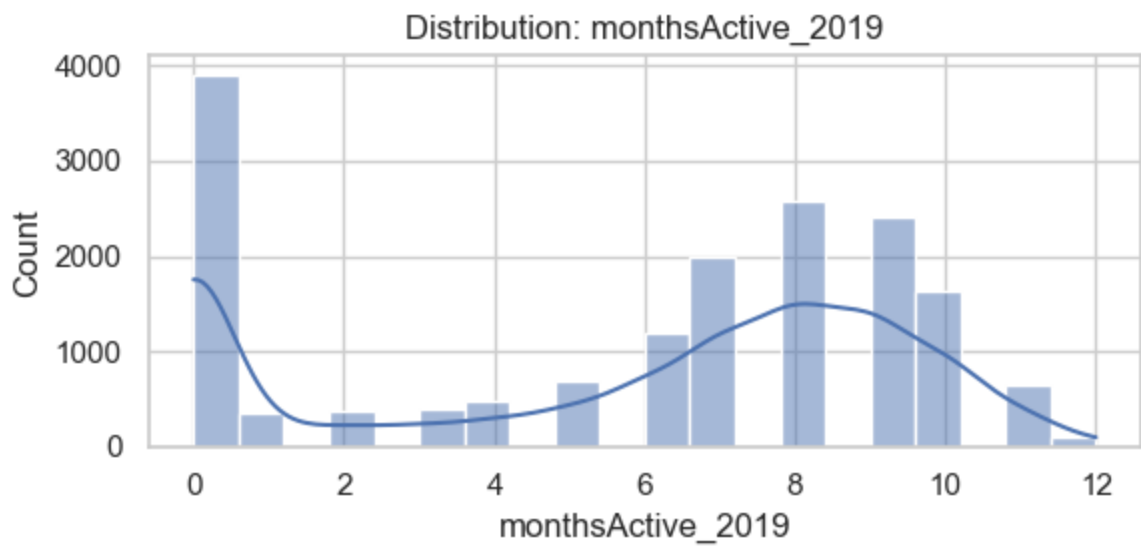
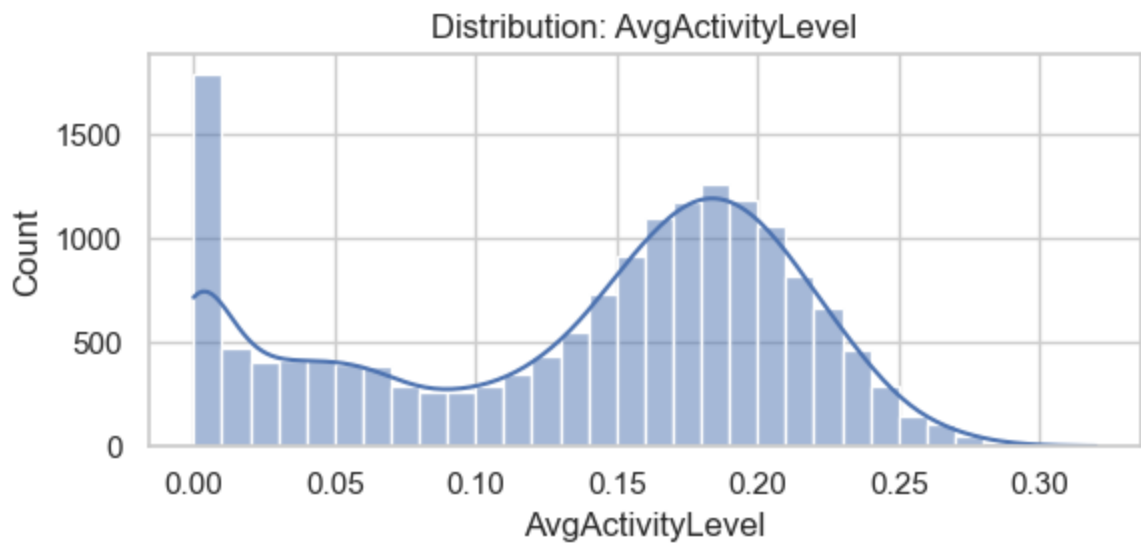
Distribution: TotalDollarCostRedeemed

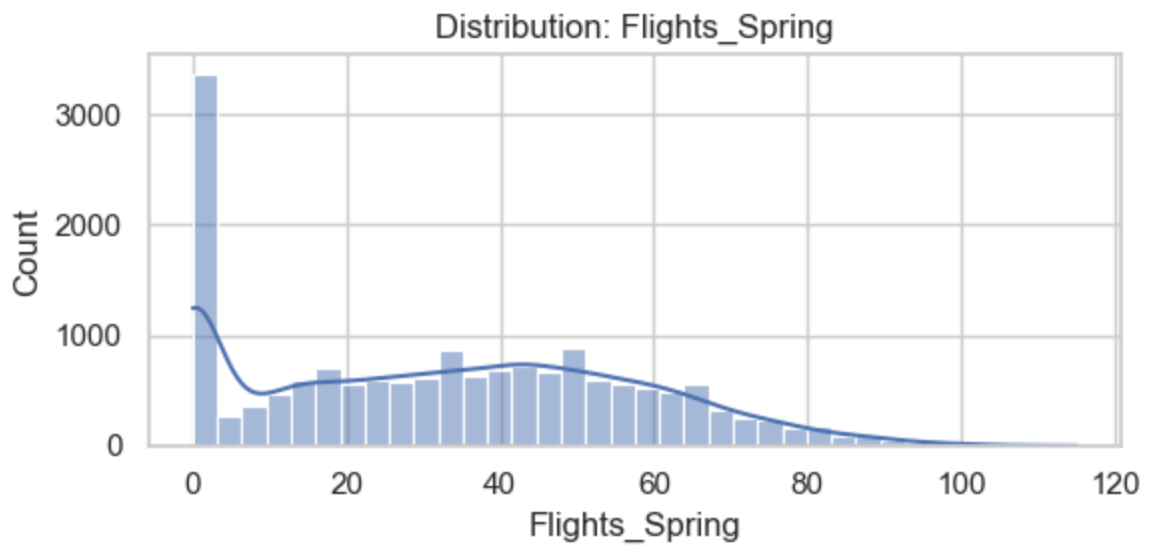
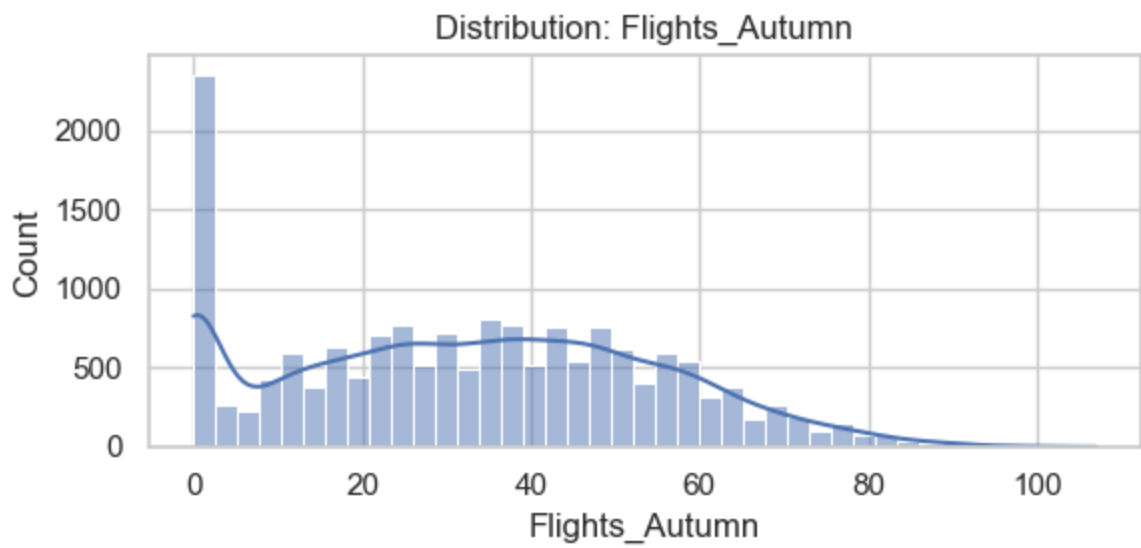
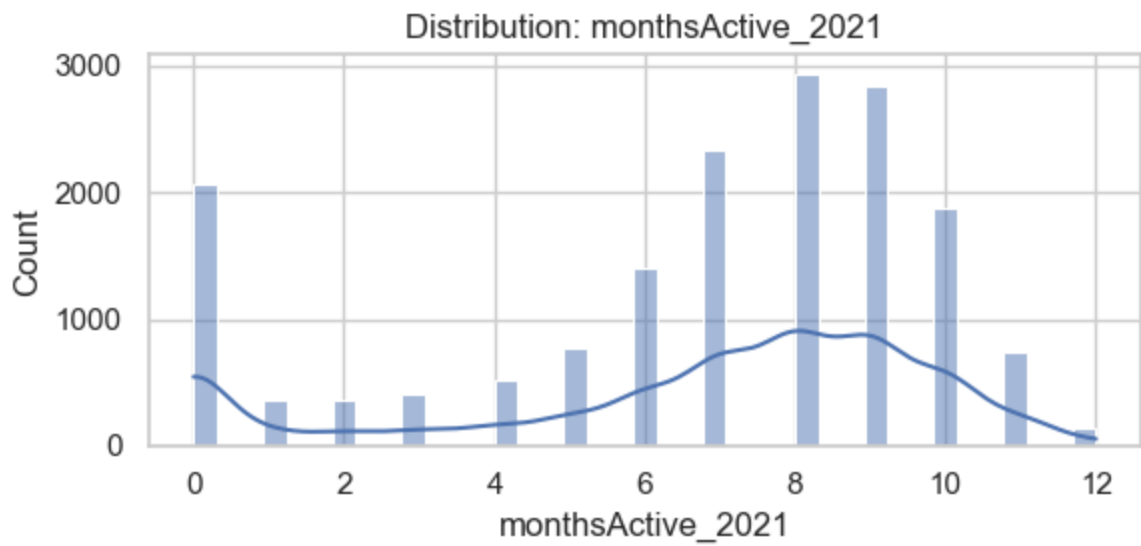


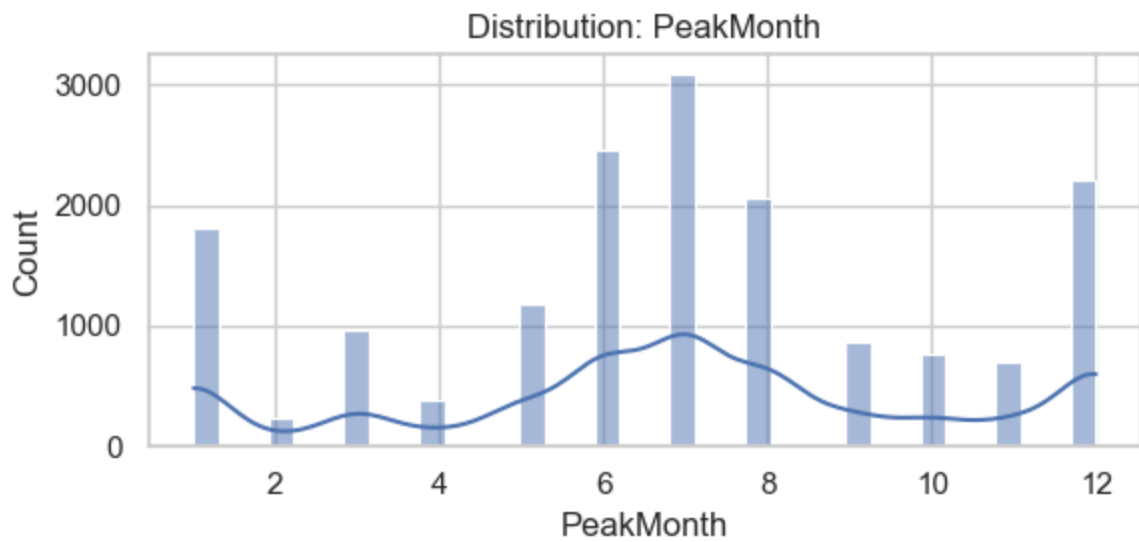
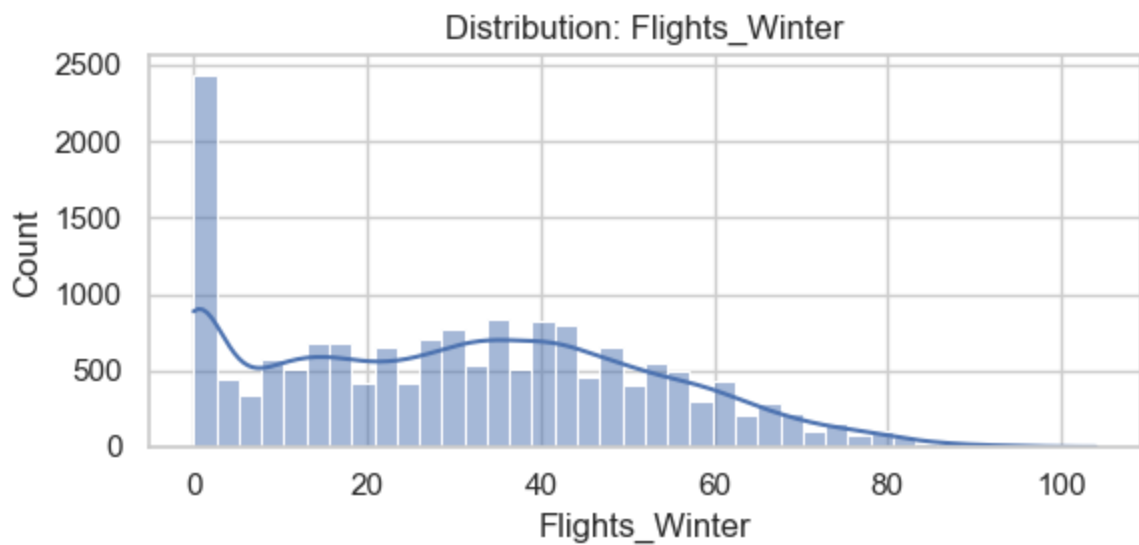
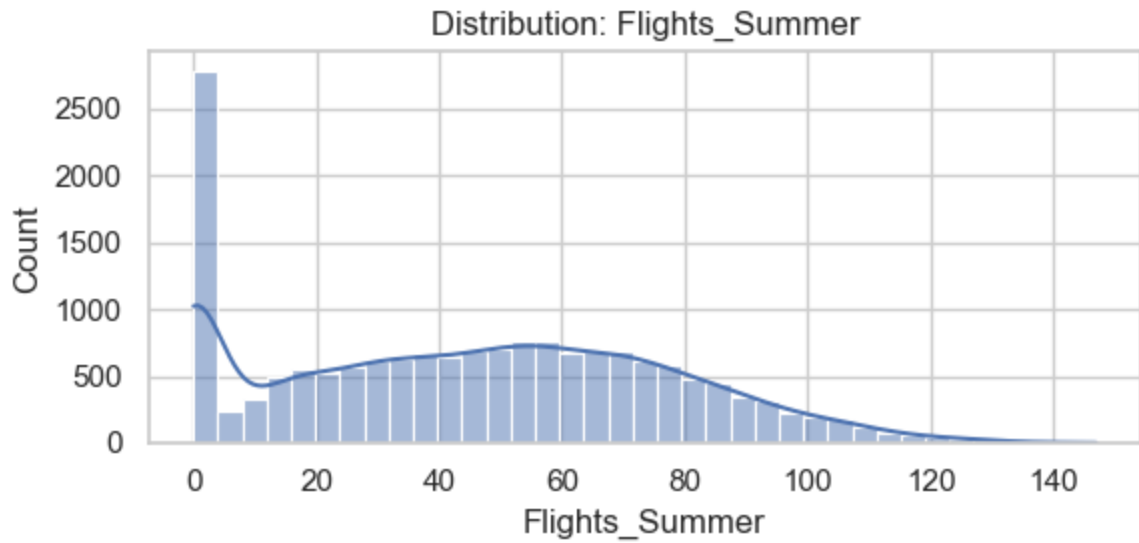
Distribution: AvgDollarCostRedeemed





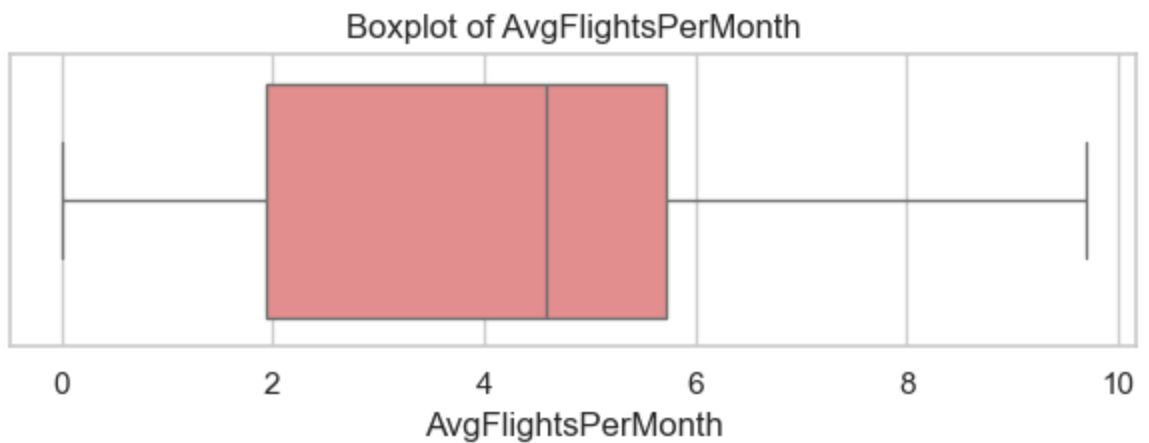
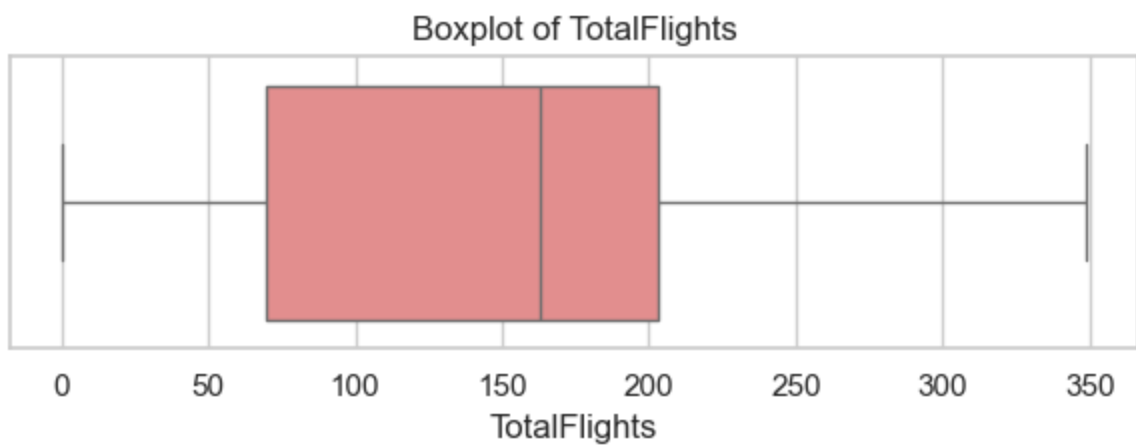
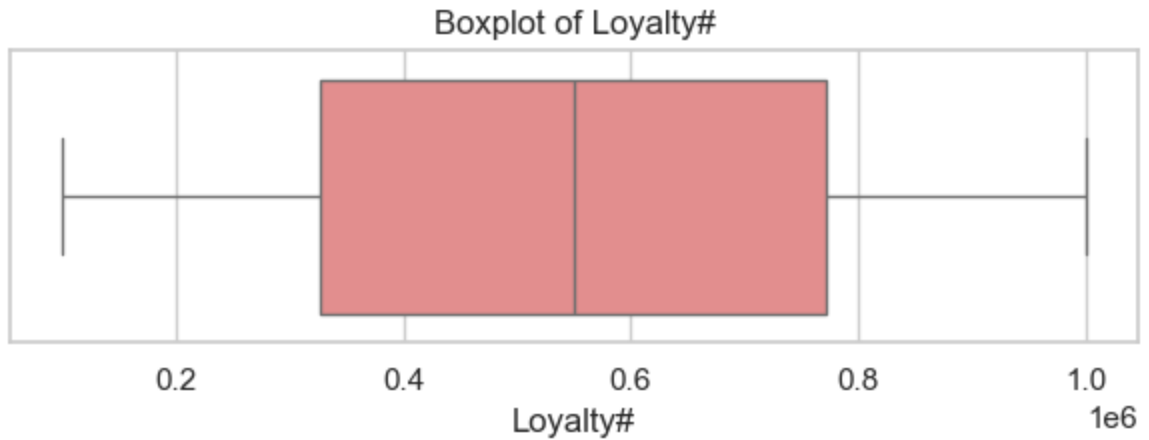




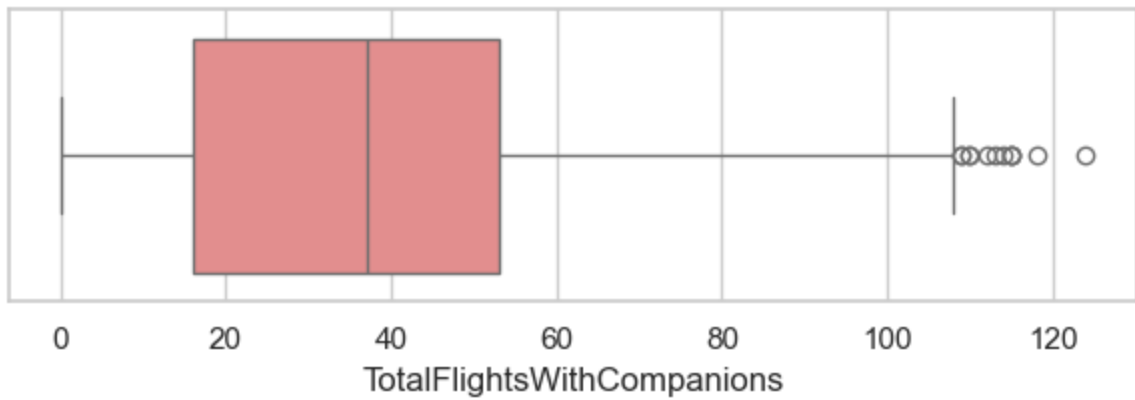


Box Plots for numerical variables

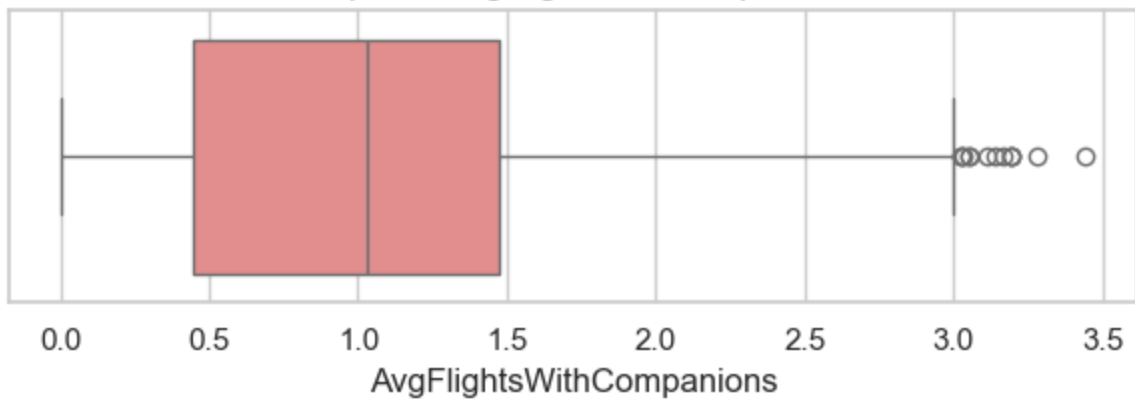
```
In [271... for c in num_cols:
plt.figure(figsize=(6,2.5))
sns.boxplot(x=flights_agg[c], color='lightcoral')
plt.title(f"Boxplot of {c}")
plt.xlabel(c)
plt.tight_layout()
plt.show()
```



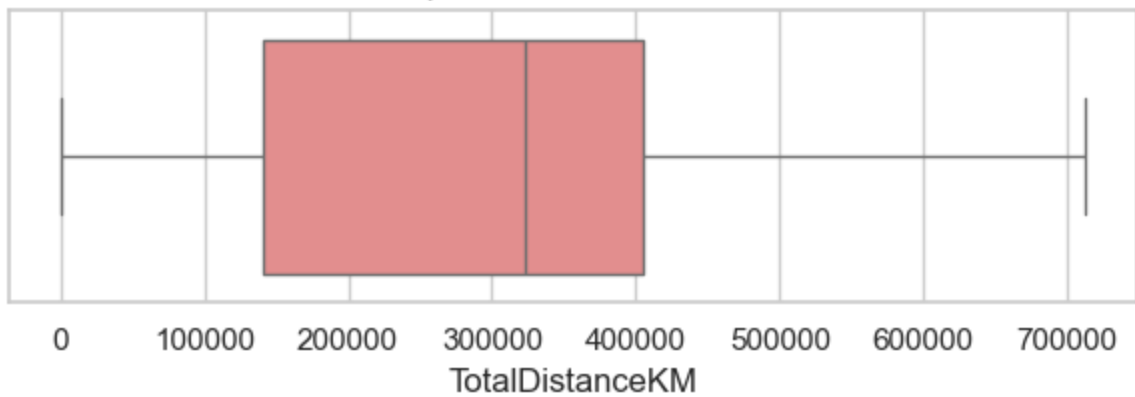
Boxplot of TotalFlightsWithCompanions



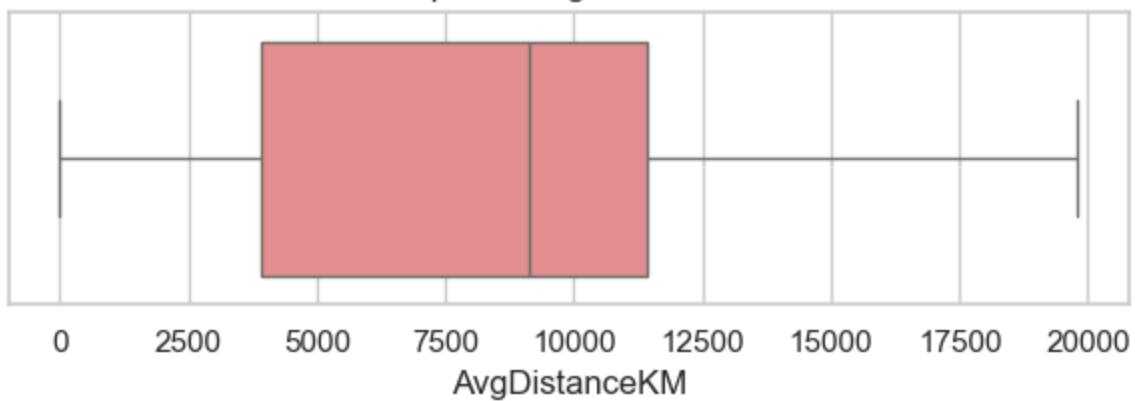
Boxplot of AvgFlightsWithCompanions



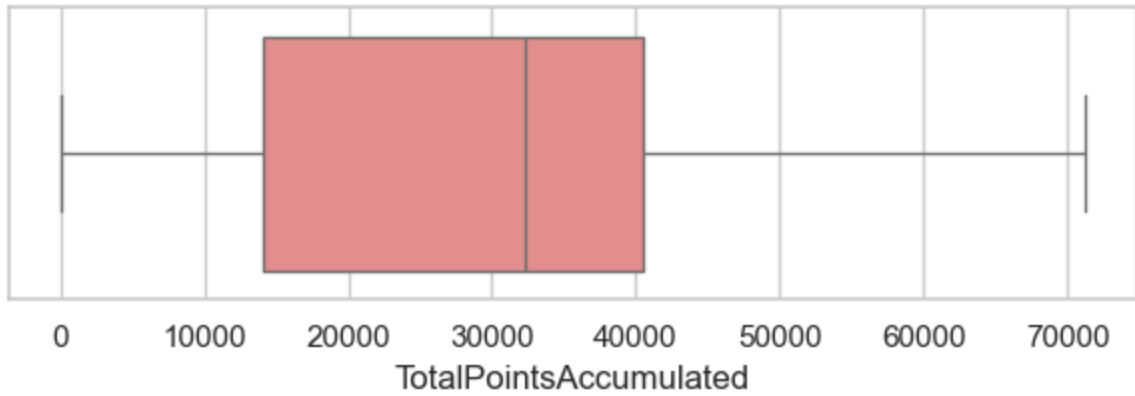
Boxplot of TotalDistanceKM



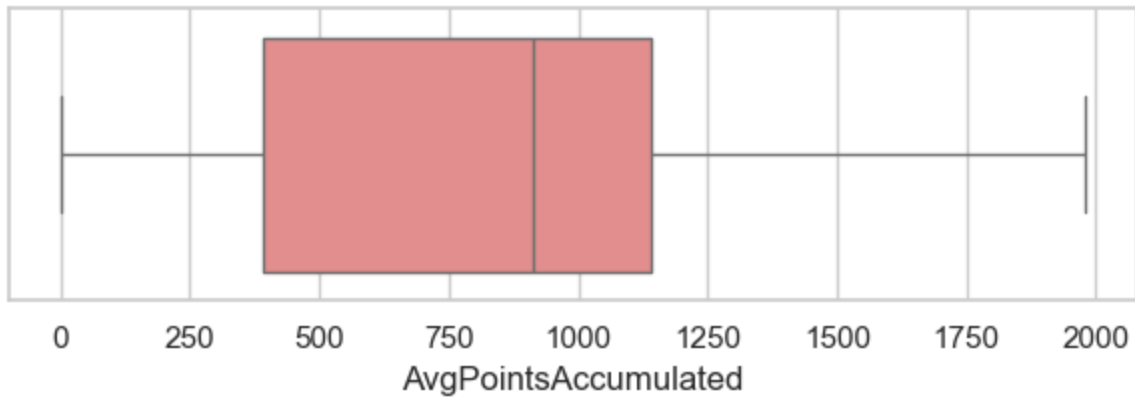
Boxplot of AvgDistanceKM



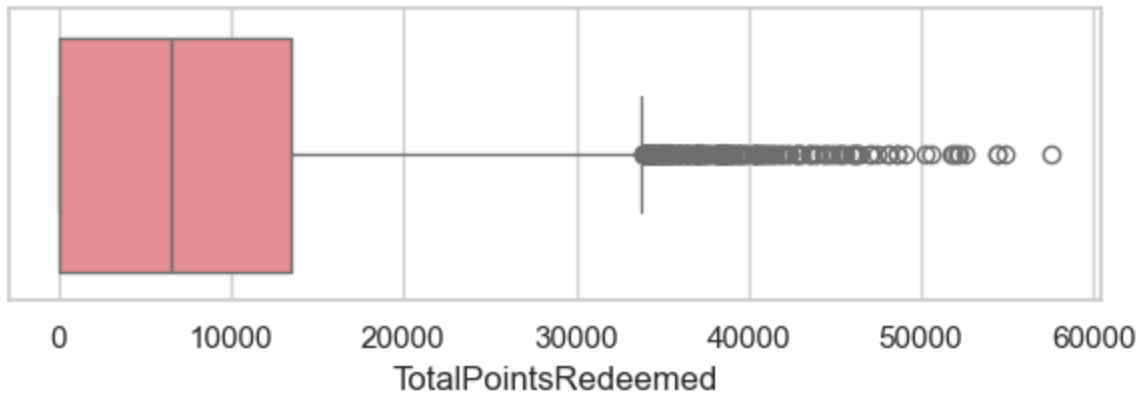
Boxplot of TotalPointsAccumulated



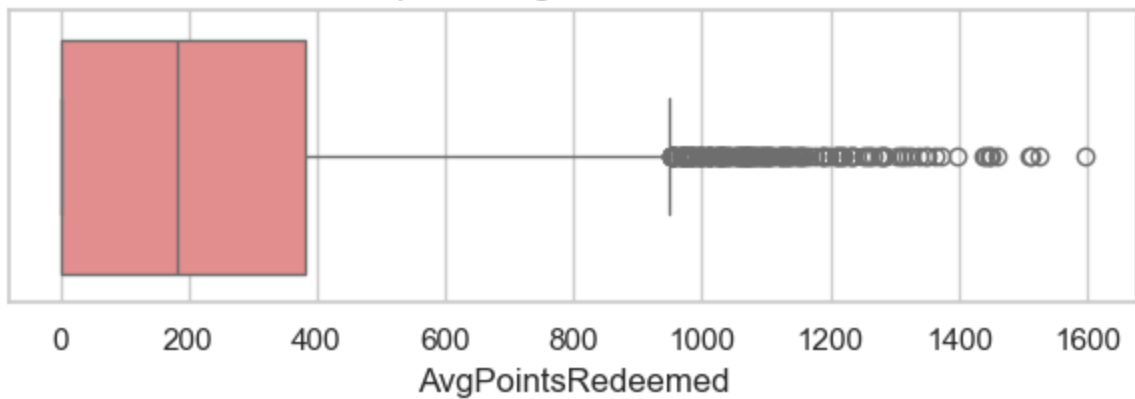
Boxplot of AvgPointsAccumulated



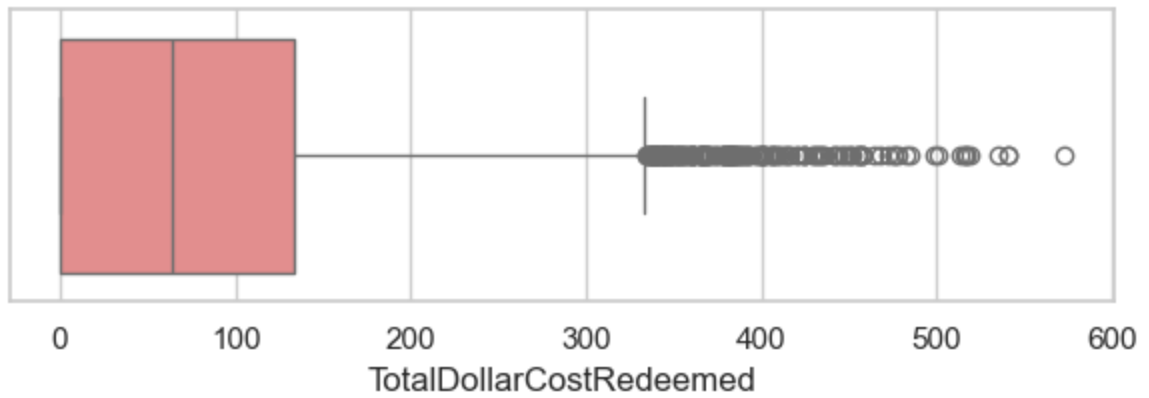
Boxplot of TotalPointsRedeemed



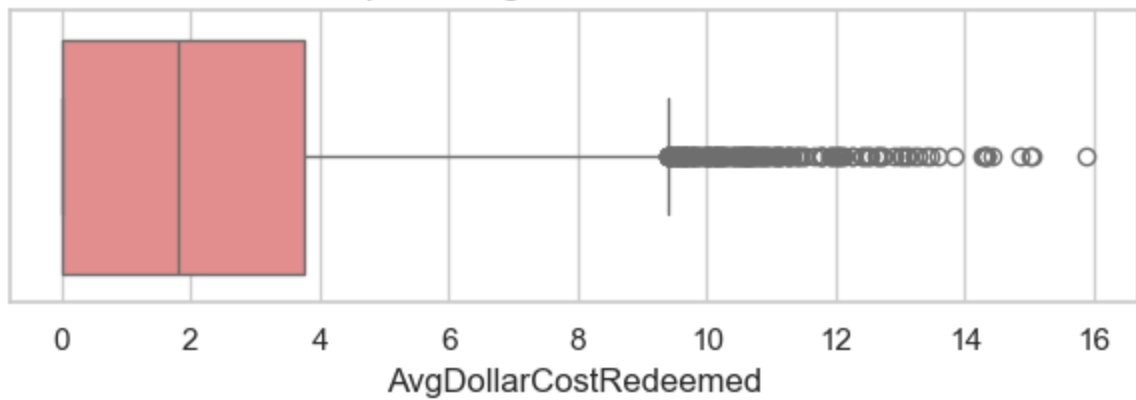
Boxplot of AvgPointsRedeemed



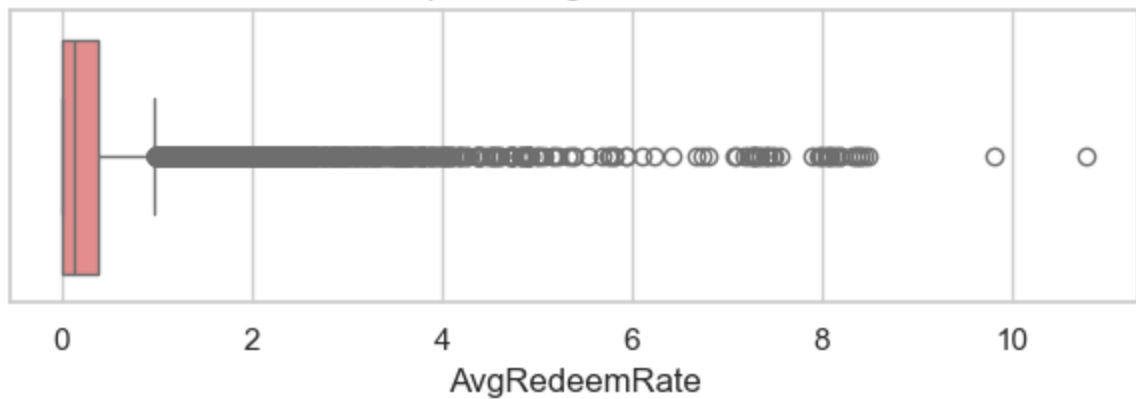
Boxplot of TotalDollarCostRedeemed



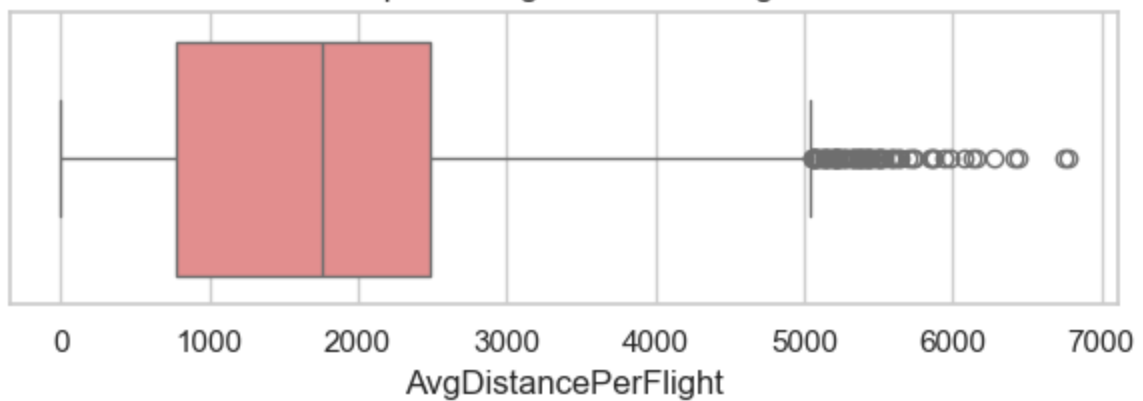
Boxplot of AvgDollarCostRedeemed



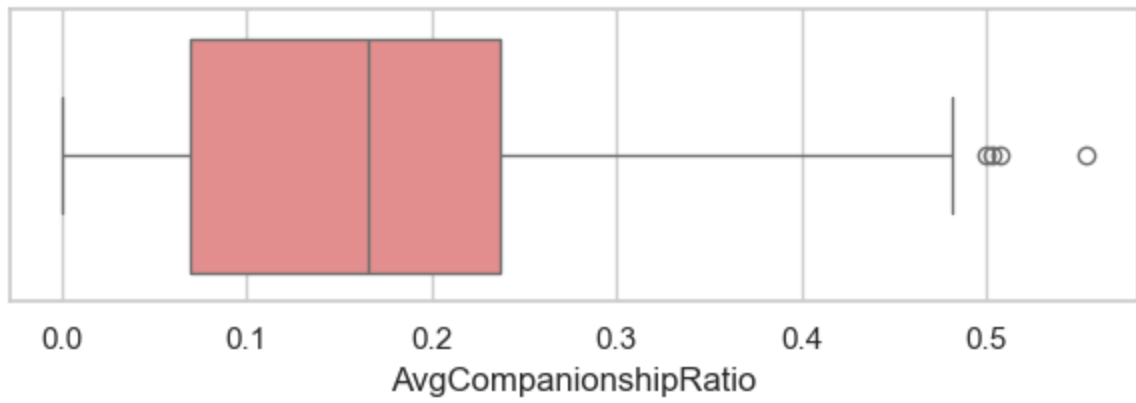
Boxplot of AvgRedeemRate



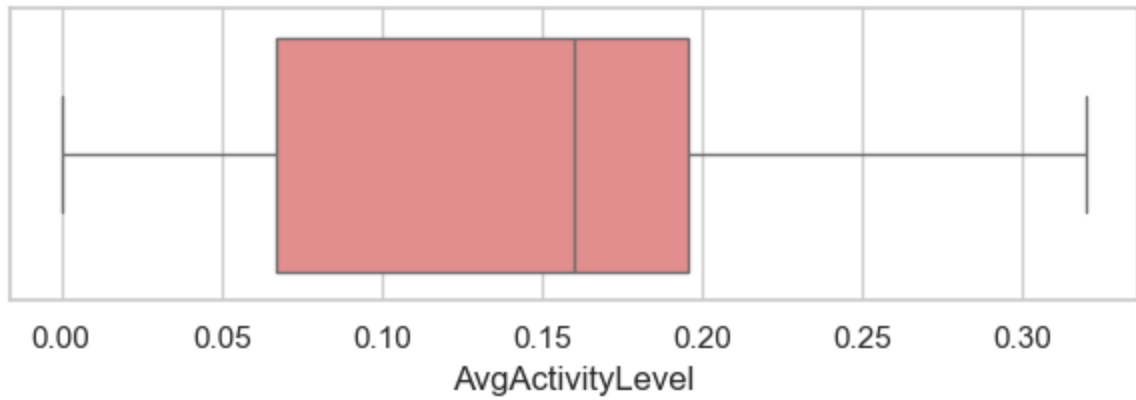
Boxplot of AvgDistancePerFlight



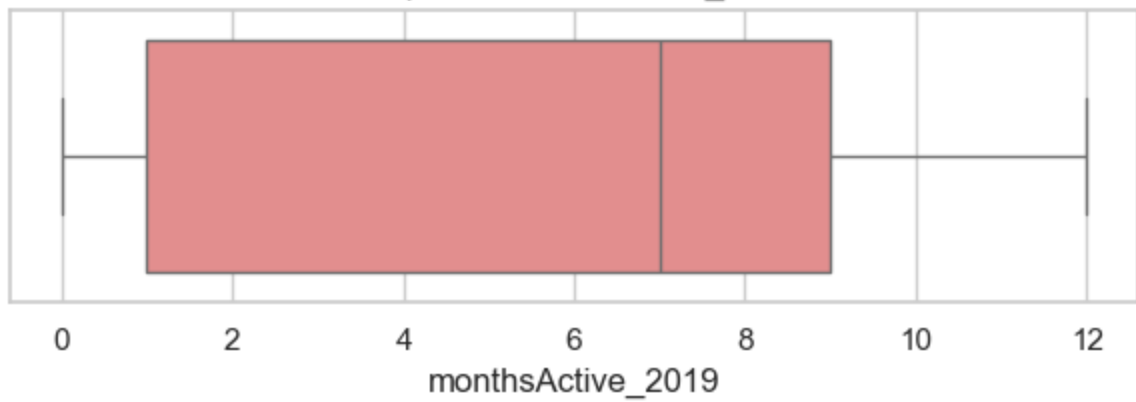
Boxplot of AvgCompanionshipRatio



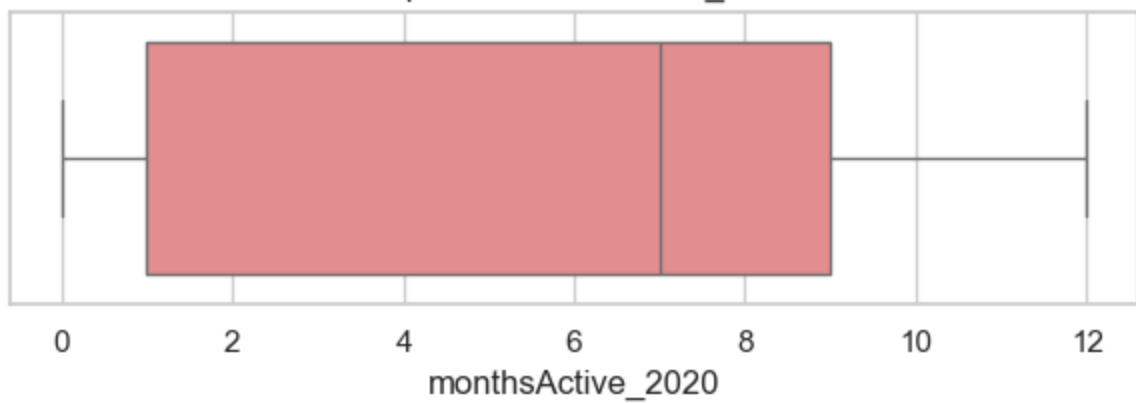
Boxplot of AvgActivityLevel



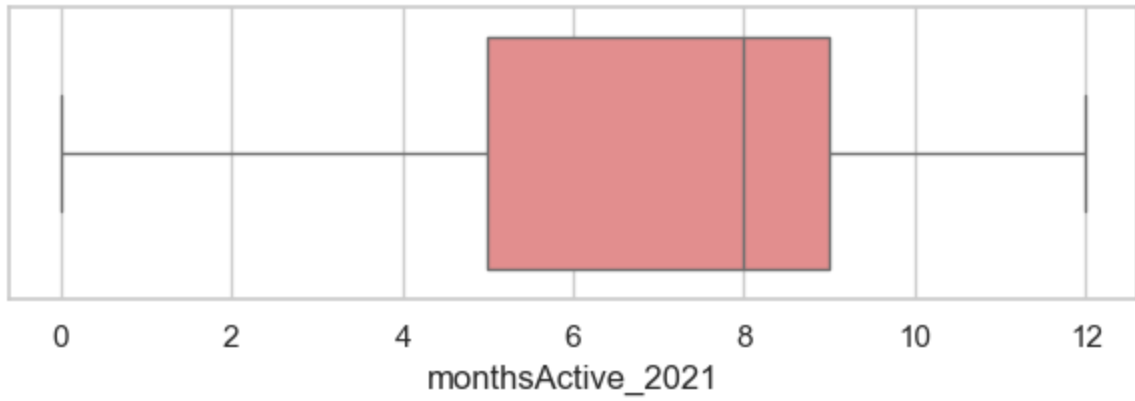
Boxplot of monthsActive_2019



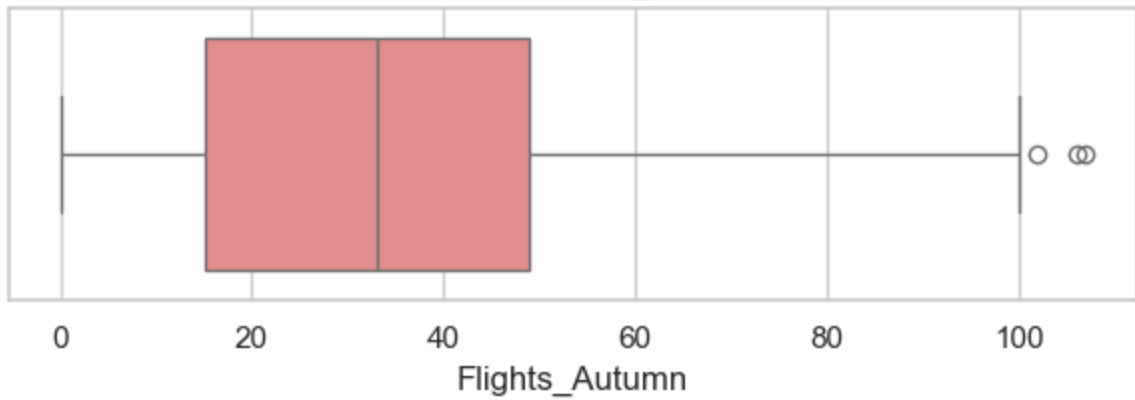
Boxplot of monthsActive_2020



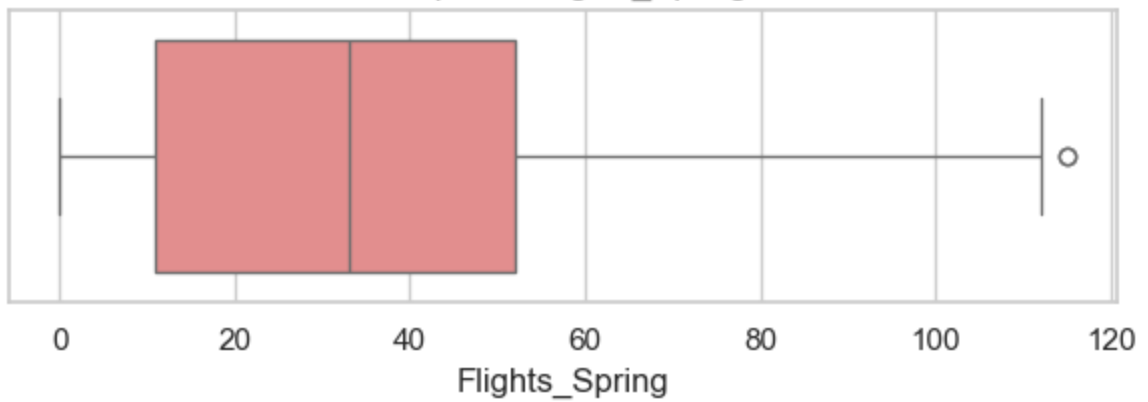
Boxplot of monthsActive_2021



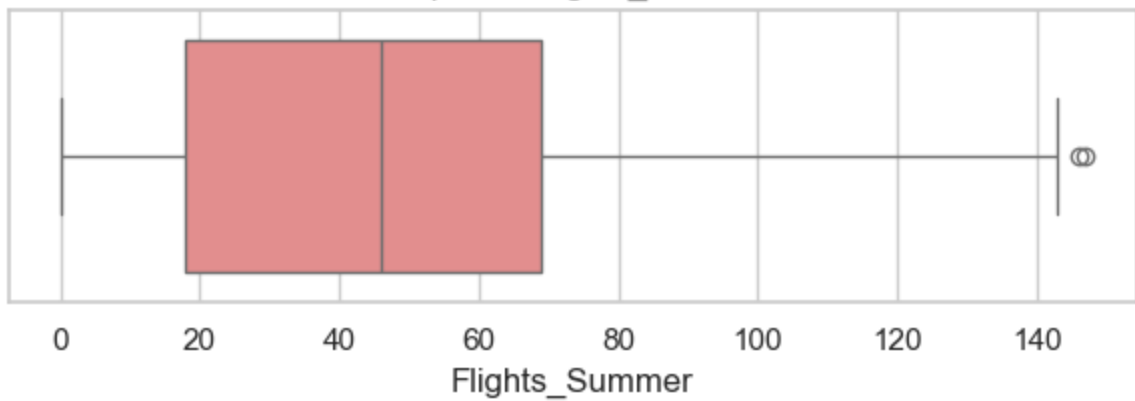
Boxplot of Flights_Autumn

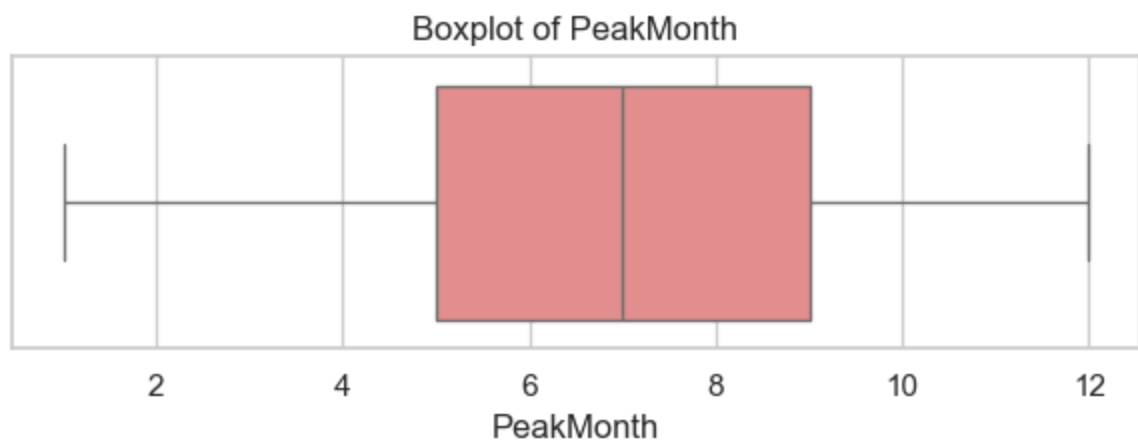
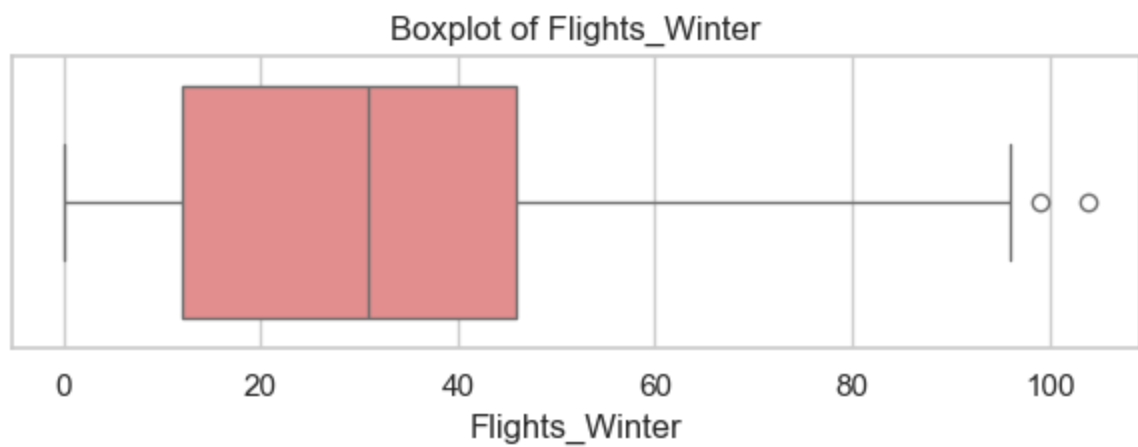


Boxplot of Flights_Spring



Boxplot of Flights_Summer

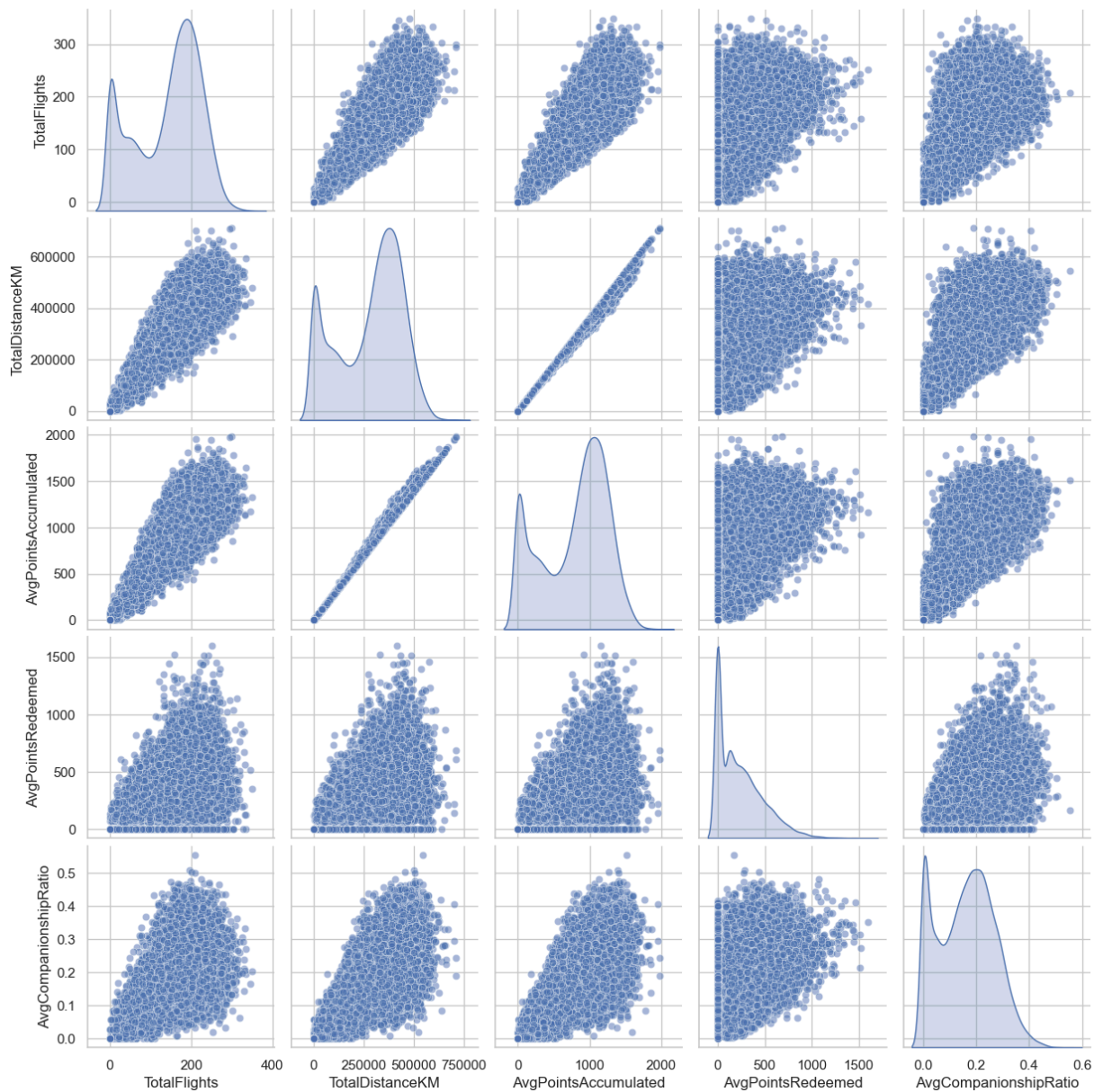




In [272... *#Here we wanted to compare multiple numerical features at once*

```
sample_cols = ['TotalFlights', 'TotalDistanceKM', 'AvgPointsAccumulated', 'A
sns.pairplot(flights_agg[sample_cols], diag_kind='kde', plot_kws={'alpha':0.
plt.suptitle("Pairplot of Selected Flight Variables", y=1.02)
plt.show()
```

Pairplot of Selected Flight Variables



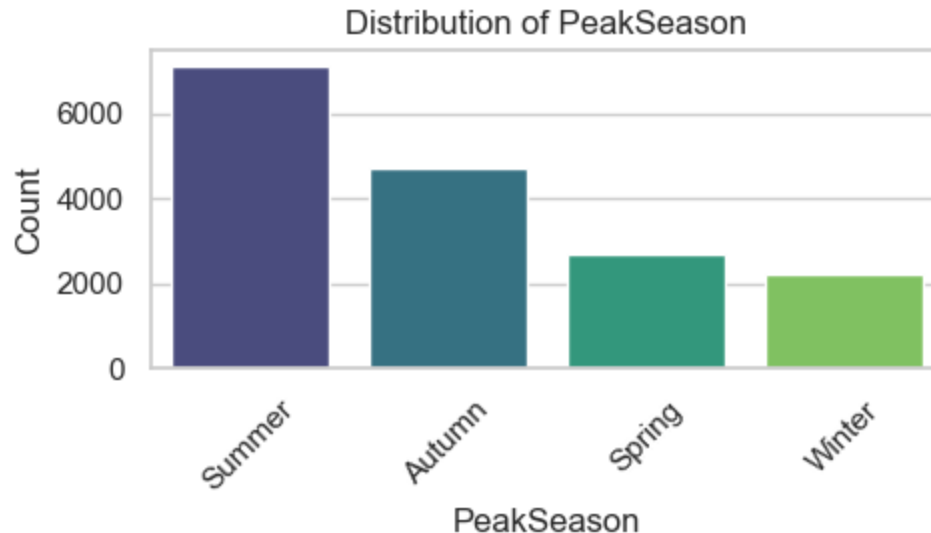
Distribution of categorical variables

```
In [273... for c in cat_cols:
    plt.figure(figsize=(5,3))
    order = flights_agg[c].value_counts().index
    sns.countplot(data=flights_agg, x=c, order=order, palette='viridis')
    plt.title(f"Distribution of {c}")
    plt.xlabel(c)
    plt.ylabel("Count")
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.show()
```

C:\Users\User\AppData\Local\Temp\ipykernel_17372\3973746353.py:4: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(data=flights_agg, x=c, order=order, palette='viridis')
```



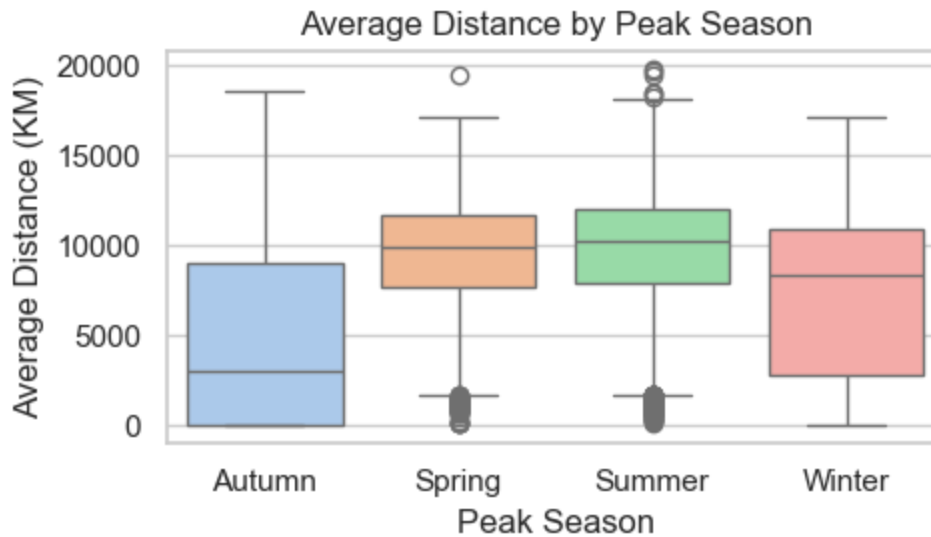
Relation with numeric variables

```
In [274... if 'PeakSeason' in flights_agg.columns and 'AvgDistanceKM' in flights_agg.columns:
    plt.figure(figsize=(5,3))
    sns.boxplot(data=flights_agg, x='PeakSeason', y='AvgDistanceKM', palette='pastel')
    plt.title("Average Distance by Peak Season")
    plt.xlabel("Peak Season")
    plt.ylabel("Average Distance (KM)")
    plt.tight_layout()
    plt.show()
```

C:\Users\User\AppData\Local\Temp\ipykernel_17372\4227483962.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.boxplot(data=flights_agg, x='PeakSeason', y='AvgDistanceKM', palette='pastel')
```



we can conclude that Summer and Spring have the highest average distances, indicating seasonal travel peaks.

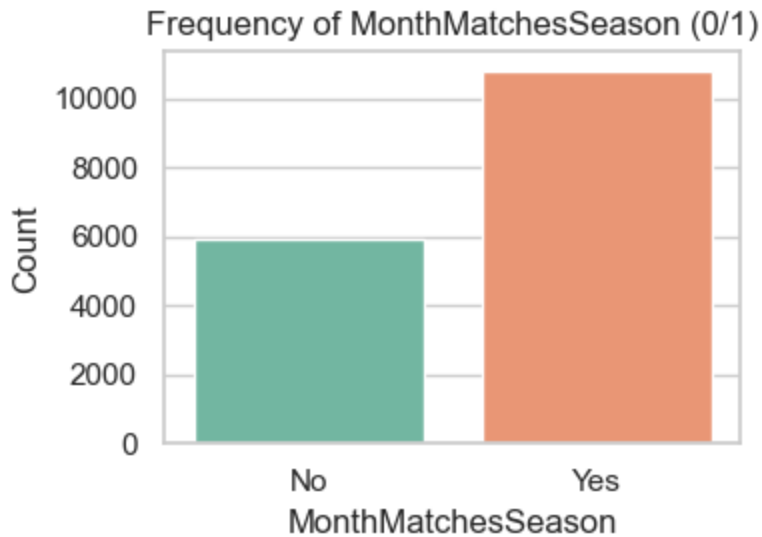
Distribution for binary variables

```
In [275... for c in binary_cols:
    plt.figure(figsize=(4,3))
    sns.countplot(x=flights_agg[c], palette='Set2')
    plt.title(f"Frequency of {c} (0/1)")
    plt.xlabel(c)
    plt.ylabel("Count")
    plt.xticks([0,1], ['No', 'Yes'])
    plt.tight_layout()
    plt.show()
```

C:\Users\User\AppData\Local\Temp\ipykernel_17372\2580106493.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

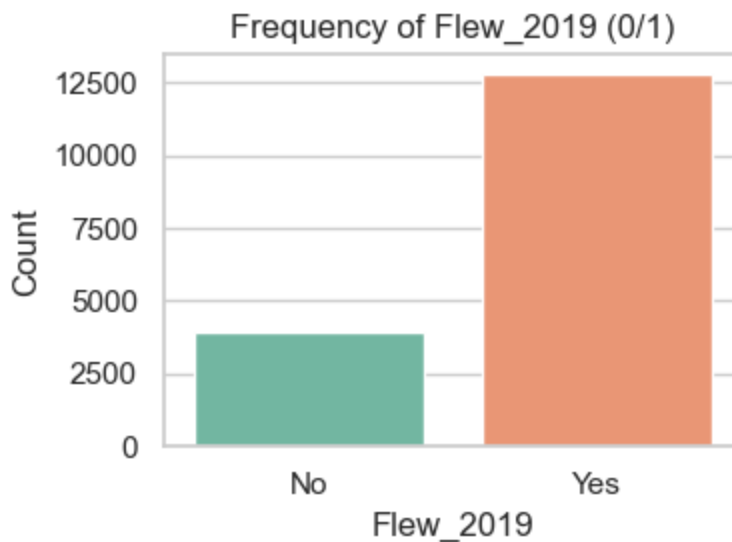
```
sns.countplot(x=flights_agg[c], palette='Set2')
```



C:\Users\User\AppData\Local\Temp\ipykernel_17372\2580106493.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

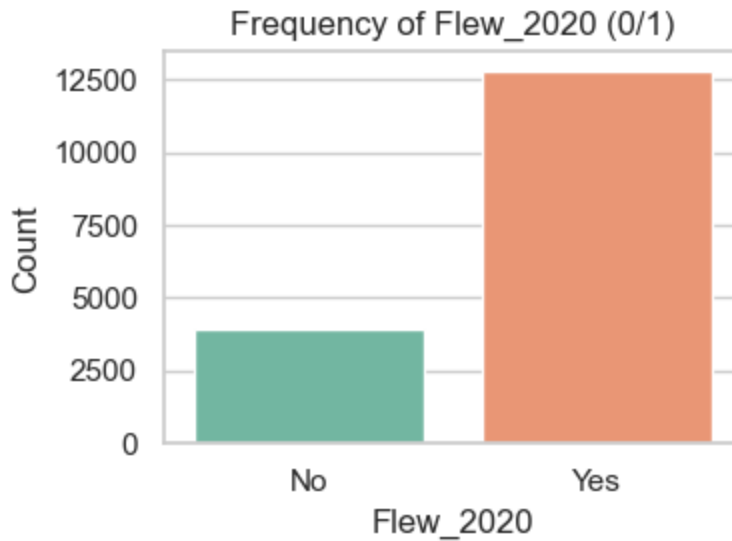
```
sns.countplot(x=flights_agg[c], palette='Set2')
```



C:\Users\User\AppData\Local\Temp\ipykernel_17372\2580106493.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

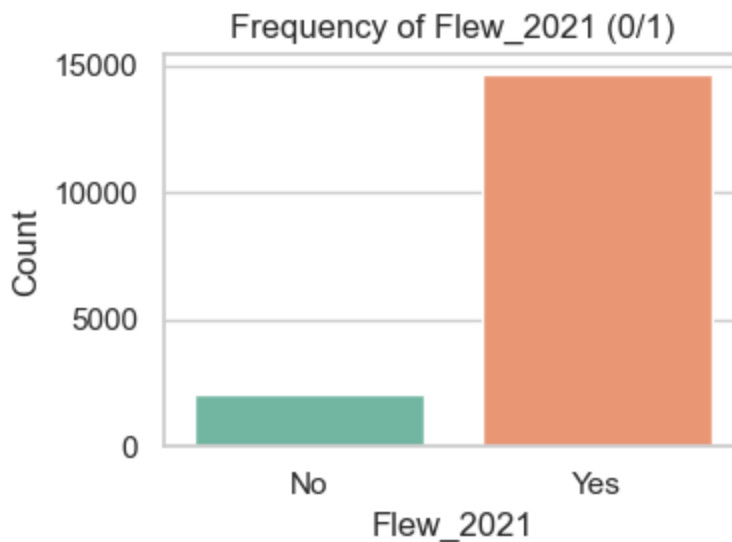
```
sns.countplot(x=flights_agg[c], palette='Set2')
```



C:\Users\User\AppData\Local\Temp\ipykernel_17372\2580106493.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(x=flights_agg[c], palette='Set2')
```



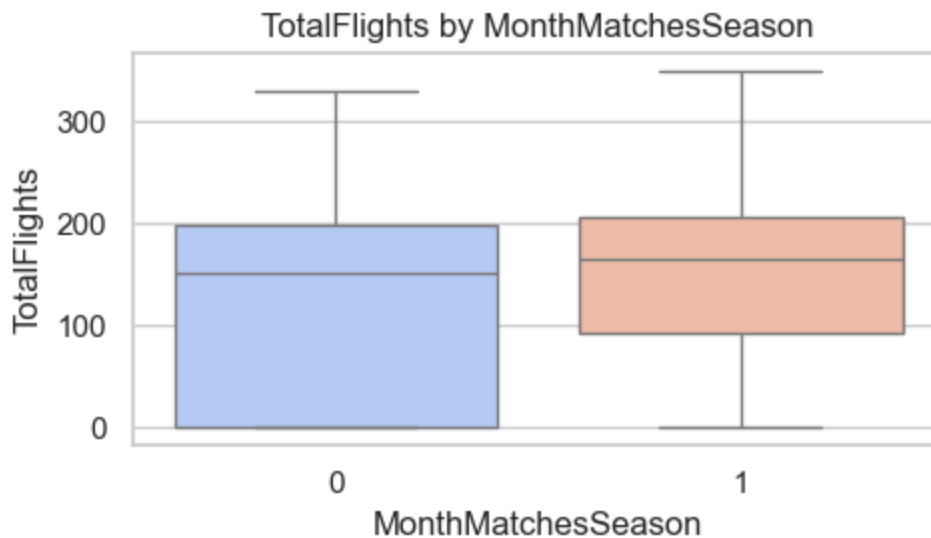
Compare binary with a key numeric variable

```
In [276... target_num = 'TotalFlights' # change if needed
for c in binary_cols:
    plt.figure(figsize=(5,3))
    sns.boxplot(x=flights_agg[c], y=flights_agg[target_num], palette='coolwa
    plt.title(f"{target_num} by {c}")
    plt.xlabel(c)
    plt.ylabel(target_num)
    plt.tight_layout()
    plt.show()
```

C:\Users\User\AppData\Local\Temp\ipykernel_17372\2991761129.py:4: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

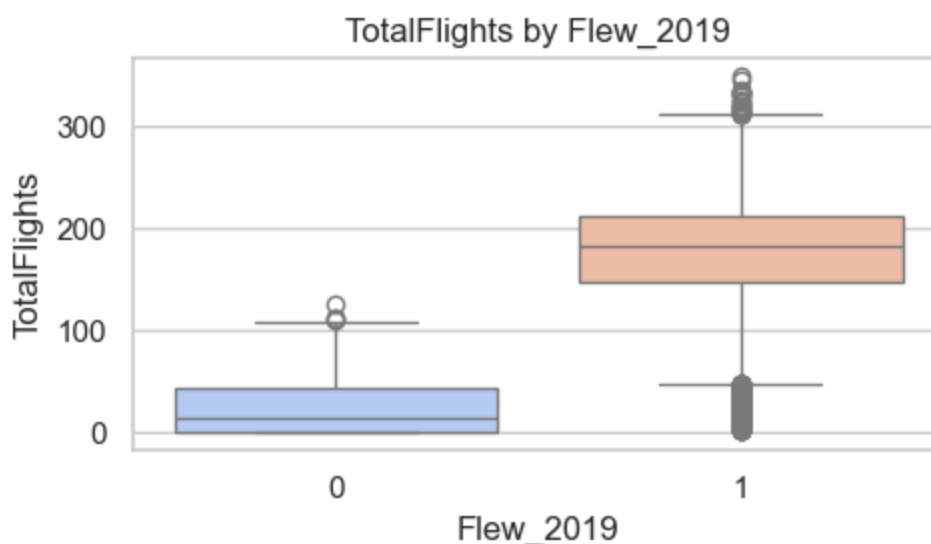
```
sns.boxplot(x=flights_agg[c], y=flights_agg[target_num], palette='coolwarm')
```



C:\Users\User\AppData\Local\Temp\ipykernel_17372\2991761129.py:4: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

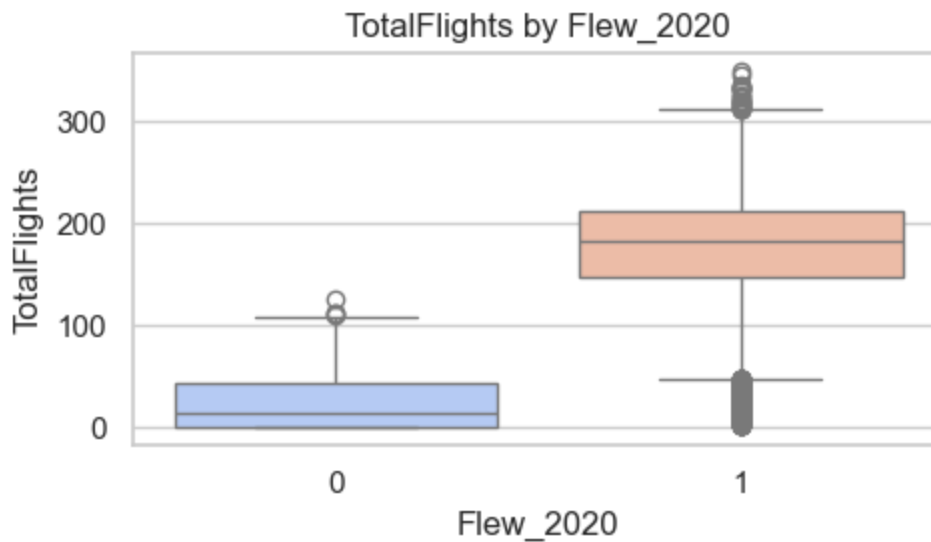
```
sns.boxplot(x=flights_agg[c], y=flights_agg[target_num], palette='coolwarm')
```



C:\Users\User\AppData\Local\Temp\ipykernel_17372\2991761129.py:4: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

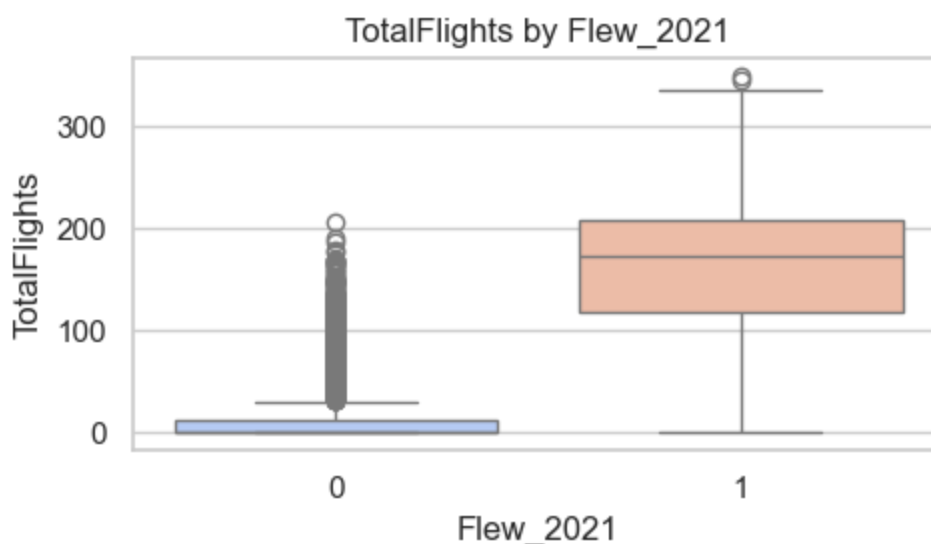
```
sns.boxplot(x=flights_agg[c], y=flights_agg[target_num], palette='coolwarm')
```



C:\Users\User\AppData\Local\Temp\ipykernel_17372\2991761129.py:4: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.boxplot(x=flights_agg[c], y=flights_agg[target_num], palette='coolwarm')
```



3.5. Pre-Processing

3.5.1. Filling Missing Values

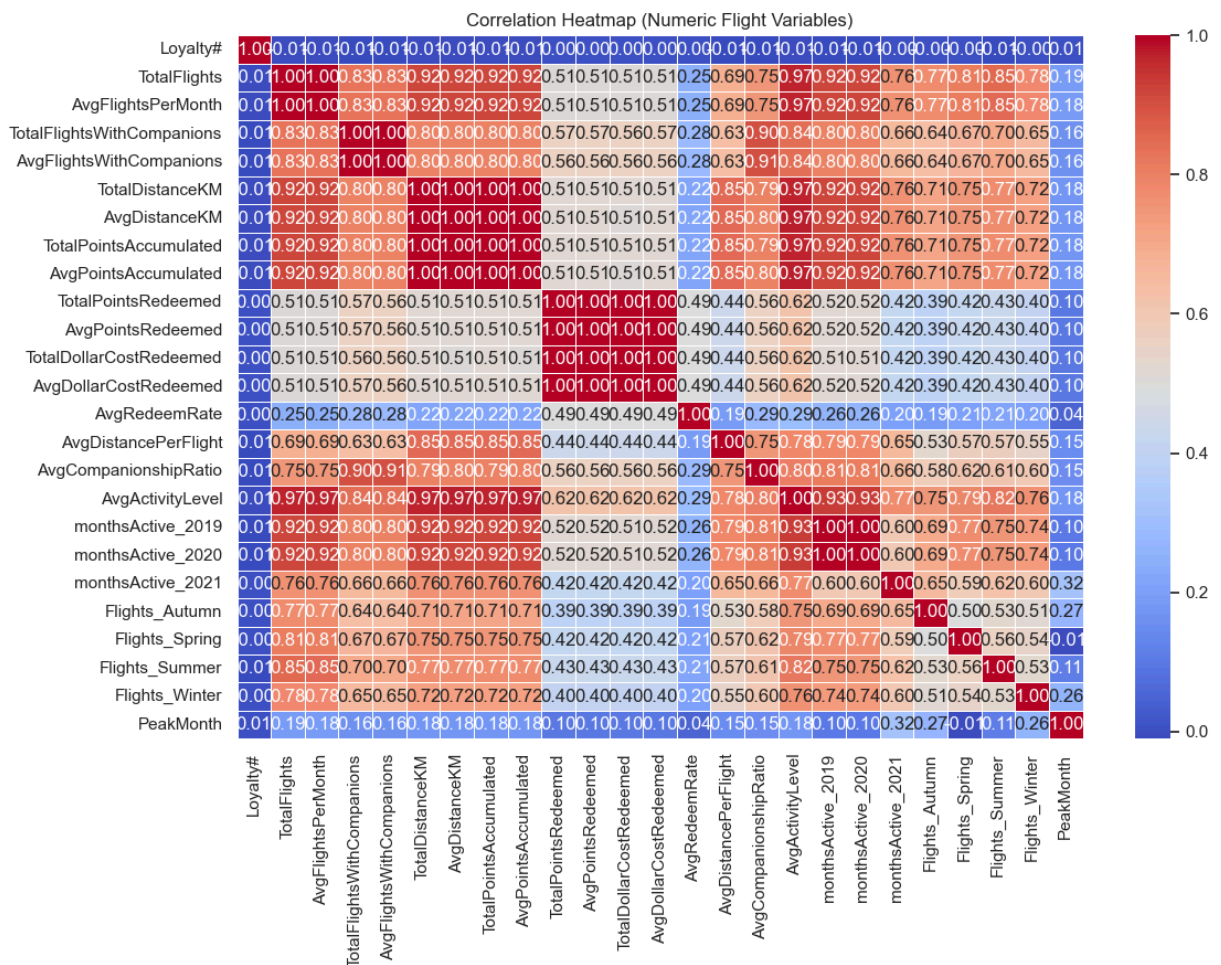
```
In [277... #there is no missing values
```

3.5.2. Feature Engineering

```
In [278... # i dont think we need to do this again tho
```

3.5.3 Correlation Matrix (Numerical Variables)

```
In [279... plt.figure(figsize=(12,9))
sns.heatmap(flights_agg[num_cols].corr(), annot=True, cmap='coolwarm', fmt="
plt.title("Correlation Heatmap (Numeric Flight Variables)")
plt.tight_layout()
plt.show()
```



We can see that the correlation matrix is very big .Thus the interpretation is difficult . We need first to focus on strong correlations

```
In [280... corr = flights_agg.corr(numeric_only=True)

# Transform matrix into a list of pairs
corr_pairs = corr.unstack().reset_index()
corr_pairs.columns = ['Variable 1', 'Variable 2', 'Correlation']

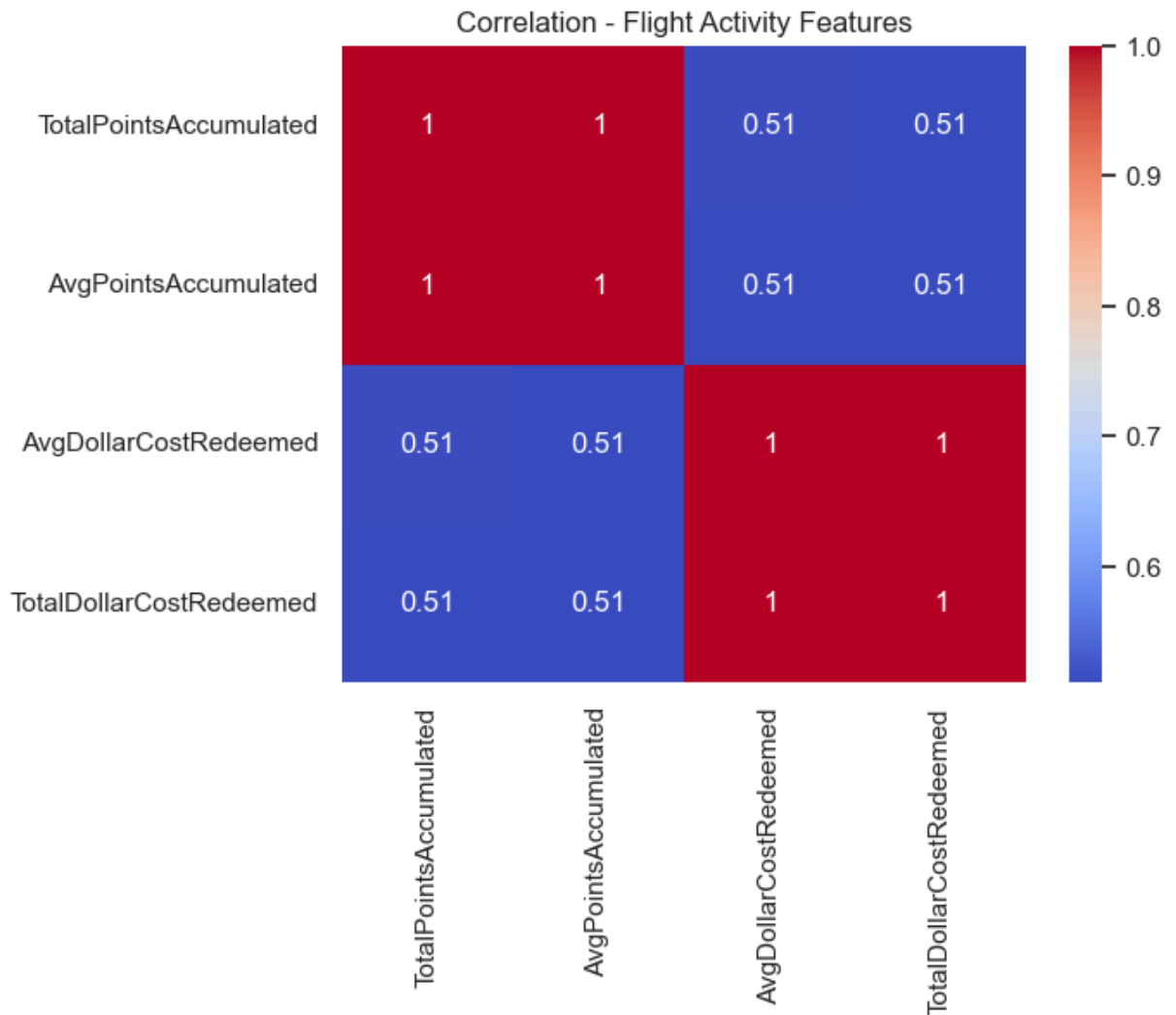
# Remove duplicates and self-correlations
corr_pairs = corr_pairs[corr_pairs['Variable 1'] < corr_pairs['Variable 2']]

# Keep only strong correlations
strong_corr = corr_pairs[abs(corr_pairs['Correlation']) >= 0.5].sort_values(
display(strong_corr.head(20))
```

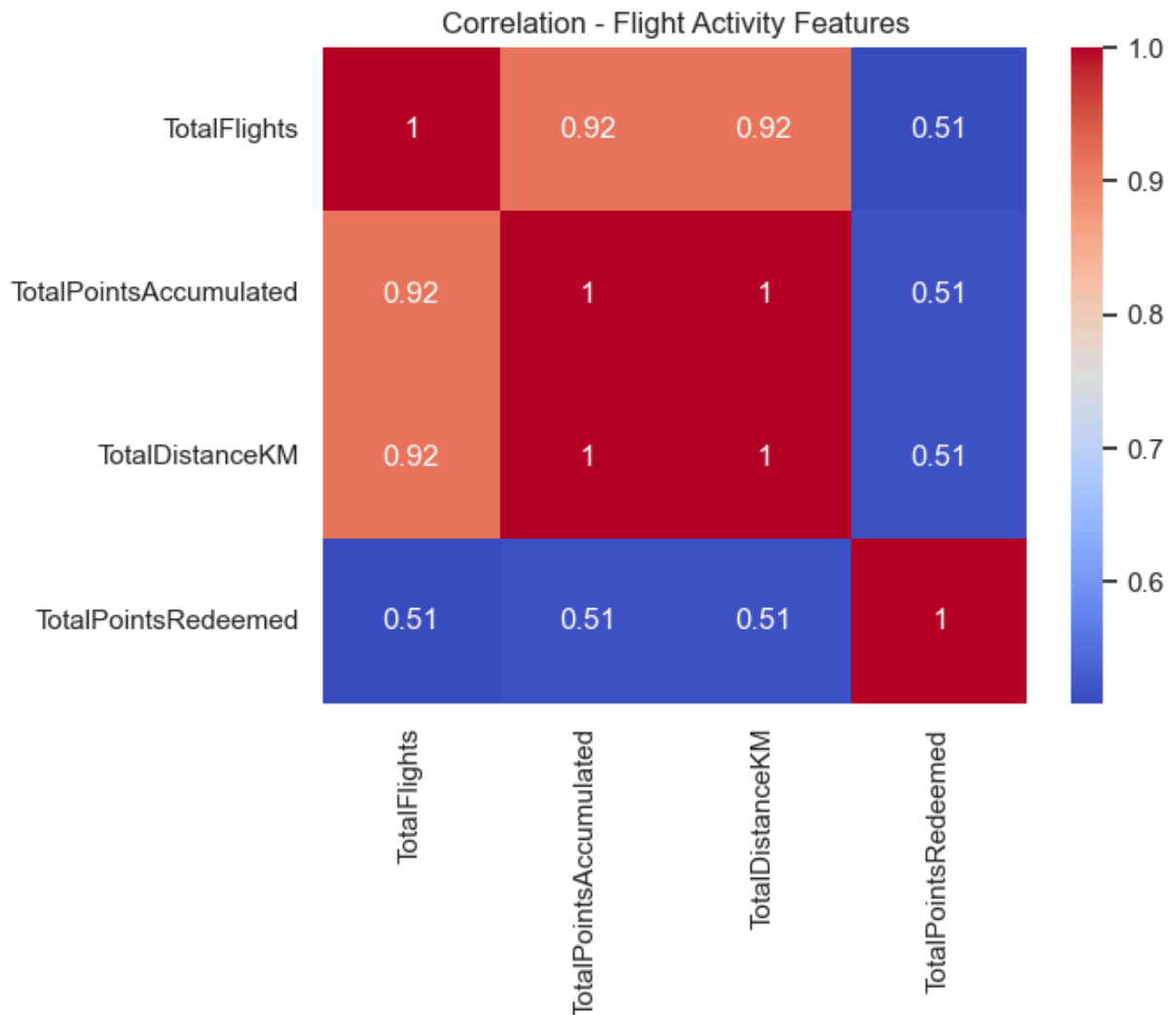
	Variable 1	Variable 2	Correlation
807	Flew_2019	Flew_2020	1.000000
528	monthsActive_2019	monthsActive_2020	1.000000
188	AvgDistanceKM	AvgPointsAccumulated	1.000000
157	TotalDistanceKM	TotalPointsAccumulated	1.000000
339	TotalDollarCostRedeemed	TotalPointsRedeemed	0.999978
370	AvgDollarCostRedeemed	AvgPointsRedeemed	0.999978
371	AvgDollarCostRedeemed	TotalDollarCostRedeemed	0.999459
309	AvgPointsRedeemed	TotalPointsRedeemed	0.999457
369	AvgDollarCostRedeemed	TotalPointsRedeemed	0.999445
311	AvgPointsRedeemed	TotalDollarCostRedeemed	0.999427
123	AvgFlightsWithCompanions	TotalFlightsWithCompanions	0.998989
61	AvgFlightsPerMonth	TotalFlights	0.998779
247	AvgPointsAccumulated	TotalPointsAccumulated	0.998778
187	AvgDistanceKM	TotalPointsAccumulated	0.998778
245	AvgPointsAccumulated	TotalDistanceKM	0.998778
185	AvgDistanceKM	TotalDistanceKM	0.998778
486	AvgActivityLevel	AvgDistanceKM	0.972379
488	AvgActivityLevel	AvgPointsAccumulated	0.972372
482	AvgActivityLevel	AvgFlightsPerMonth	0.971976
485	AvgActivityLevel	TotalDistanceKM	0.971366

The analysis shows a strong positive correlation ($r = 1$) between AvgPointsAccumulated and TotalPointsAccumulated. Another strong correlation ($r = 1$) was found between AvgDollarCostRedeemed and TotalDollarCostRedeemed.

```
In [281... cols_group1 = ['TotalPointsAccumulated', 'AvgPointsAccumulated', 'AvgDollarC
sns.heatmap(flights_agg[cols_group1].corr(), annot=True, cmap='coolwarm')
plt.title("Correlation - Flight Activity Features")
plt.show()
```



```
In [282... cols_group1 = ['TotalFlights', 'TotalPointsAccumulated', 'TotalDistanceKM',
sns.heatmap(flights_agg[cols_group1].corr(), annot=True, cmap='coolwarm')
plt.title("Correlation - Flight Activity Features")
plt.show()
```



The analysis shows a strong positive correlation ($r = 0.92$) between TotalFlights and TotalDistanceKM, confirming that frequent travelers tend to cover longer distances. Another strong correlation ($r = 0.92$) was found between TotalPointsAccumulated and Totalflights.

4. Customers and Flights Datasets Merged

4.1. Merging the two datasets

Check in both datasets

```
In [283... print("Customer dataset shape:", customers.shape)
print("Flight dataset shape:", flights_agg.shape)
```

```
Customer dataset shape: (16757, 24)
Flight dataset shape: (16737, 31)
```

```
In [284... print("Customer key unique:", customers['Loyalty#'].is_unique)
print("Flight key unique:", flights_agg['Loyalty#'].is_unique)
```

Customer key unique: True
Flight key unique: True

```
In [287... merged_df = pd.merge(customers, flights_agg, on='Loyalty#', how='left')
merged_df
```

Out[287...

	Loyalty#	Province or State	City	Latitude	Longitude	Postal code	Gender
0	480934	Ontario	Toronto	43.653225	-79.383186	M2Z 4K1	female
1	549612	Alberta	Edmonton	53.544388	-113.490930	T3G 6Y6	male
2	429460	British Columbia	Vancouver	49.282730	-123.120740	V6E 3D9	male
3	608370	Ontario	Toronto	43.653225	-79.383186	P1W 1K4	male
4	530508	Quebec	Hull	45.428730	-75.713364	J8Y 3Z5	male
...	...	...	...	...	...	...	...
16752	100012	Quebec	Quebec City	46.759733	-71.141009	Y0C 7D6	male
16753	100013	Alberta	Edmonton	53.524829	-113.546357	L3S 9Y3	female
16754	100014	New Brunswick	Moncton	46.051866	-64.825428	G2S 2B6	male
16755	100015	Quebec	Quebec City	46.862970	-71.133444	B1Z 8T3	female
16756	100016	British Columbia	Dawson Creek	55.720562	-120.160090	M4A 1E4	female

16757 rows × 54 columns

```
In [288... print("Merged dataset shape:", merged_df.shape)
```

Merged dataset shape: (16757, 54)

```
In [289... print(merged_df.isna().sum().sort_values(ascending=False).head(10))
```

CancellationDate	14471
monthsActive_2021	20
monthsActive_2020	20
monthsActive_2019	20
PeakSeason	20
Flights_Winter	20
AvgCompanionshipRatio	20
AvgActivityLevel	20
AvgDistancePerFlight	20
AvgRedeemRate	20
dtype: int64	

In [290... merged_df.info()

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 16757 entries, 0 to 16756

Data columns (total 54 columns):

#	Column	Non-Null Count	Dtype
0	Loyalty#	16757 non-null	int64
1	Province or State	16757 non-null	object
2	City	16757 non-null	object
3	Latitude	16757 non-null	float64
4	Longitude	16757 non-null	float64
5	Postal code	16757 non-null	object
6	Gender	16757 non-null	object
7	Education	16757 non-null	object
8	Location Code	16757 non-null	object
9	Income	16757 non-null	float64
10	Marital Status	16757 non-null	object
11	LoyaltyStatus	16757 non-null	object
12	EnrollmentDateOpening	16757 non-null	datetime64[ns]
13	CancellationDate	2286 non-null	datetime64[ns]
14	Customer Lifetime Value	16757 non-null	float64
15	EnrollmentType	16757 non-null	object
16	Churn	16757 non-null	int64
17	AccountDuration	16757 non-null	int64
18	YearsInProgram	16757 non-null	float64
19	HasHighIncome	16757 non-null	int64
20	MaritalGroup	16757 non-null	object
21	EnrollmentYear	16757 non-null	int32
22	EducationLevelCode	16757 non-null	int64
23	ProvinceActivity	16757 non-null	int64
24	TotalFlights	16737 non-null	float64
25	AvgFlightsPerMonth	16737 non-null	float64
26	TotalFlightsWithCompanions	16737 non-null	float64
27	AvgFlightsWithCompanions	16737 non-null	float64
28	TotalDistanceKM	16737 non-null	float64
29	AvgDistanceKM	16737 non-null	float64
30	TotalPointsAccumulated	16737 non-null	float64
31	AvgPointsAccumulated	16737 non-null	float64
32	TotalPointsRedeemed	16737 non-null	float64
33	AvgPointsRedeemed	16737 non-null	float64
34	TotalDollarCostRedeemed	16737 non-null	float64
35	AvgDollarCostRedeemed	16737 non-null	float64
36	AvgRedeemRate	16737 non-null	float64
37	AvgDistancePerFlight	16737 non-null	float64
38	AvgCompanionshipRatio	16737 non-null	float64
39	AvgActivityLevel	16737 non-null	float64
40	monthsActive_2019	16737 non-null	float64
41	monthsActive_2020	16737 non-null	float64
42	monthsActive_2021	16737 non-null	float64
43	Flights_Autumn	16737 non-null	float64
44	Flights_Spring	16737 non-null	float64
45	Flights_Summer	16737 non-null	float64
46	Flights_Winter	16737 non-null	float64
47	PeakSeason	16737 non-null	category
48	PeakMonth	16737 non-null	float64
49	MonthMatchesSeason	16737 non-null	float64
50	Flew_2019	16737 non-null	float64

```

51 Flew_2020          16737 non-null float64
52 Flew_2021          16737 non-null float64
53 YearsActive        16737 non-null float64
dtypes: category(1), datetime64[ns](2), float64(34), int32(1), int64(6), object(10)
memory usage: 6.7+ MB

```

```
In [291...] merged_df.head()
```

```
Out[291...]

```

	Loyalty#	Province or State	City	Latitude	Longitude	Postal code	Gender	Educ
0	480934	Ontario	Toronto	43.653225	-79.383186	M2Z 4K1	female	Ba
1	549612	Alberta	Edmonton	53.544388	-113.490930	T3G 6Y6	male	C
2	429460	British Columbia	Vancouver	49.282730	-123.120740	V6E 3D9	male	C
3	608370	Ontario	Toronto	43.653225	-79.383186	P1W 1K4	male	C
4	530508	Quebec	Hull	45.428730	-75.713364	J8Y 3Z5	male	Ba

Feature Engineering

Feature: Churn Indicator

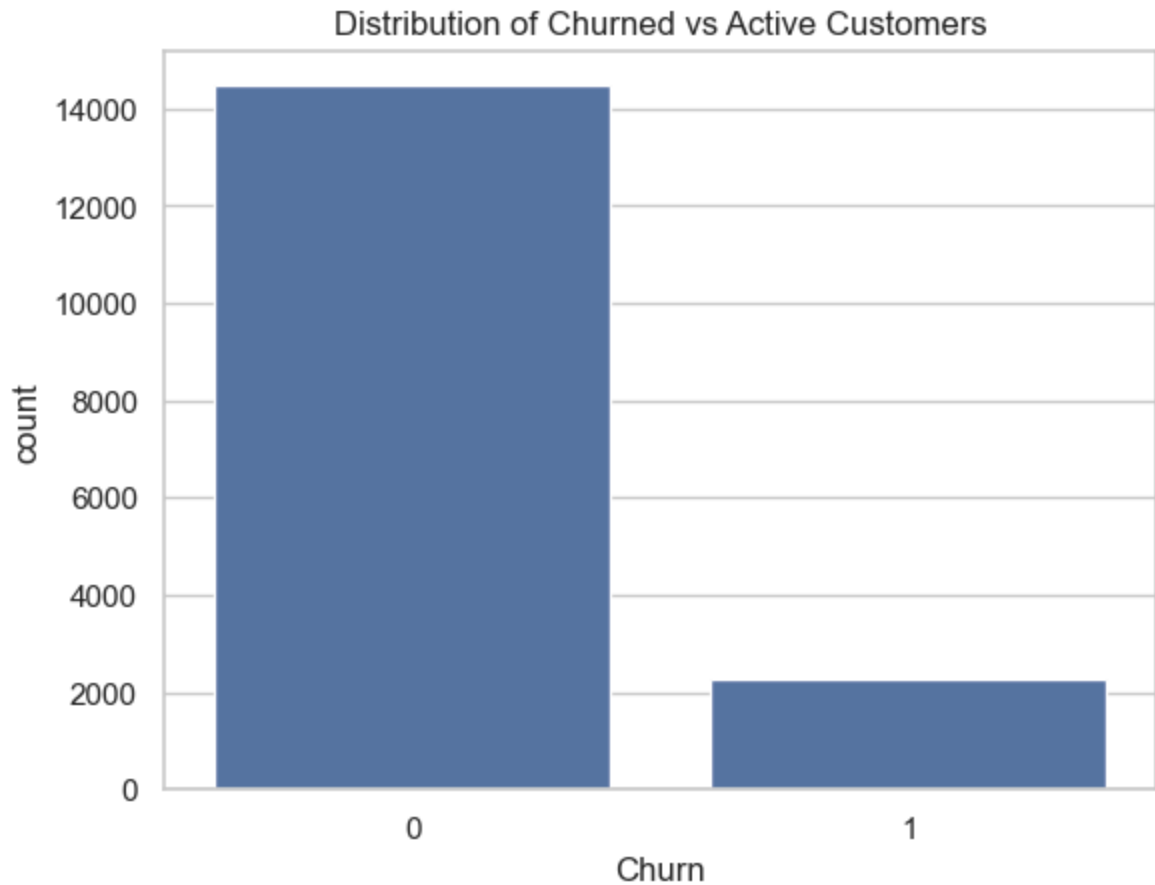
The variable `CancellationDate` was transformed into a binary indicator `Churn`, where:

- `1` indicates that the customer canceled their loyalty membership (churned).
- `0` indicates that the customer is still active.

This new feature provides valuable insight into customer retention behavior and can later be used for segmentation or predictive modeling of churn risk.

```
In [297...] merged_df['Churn'] = merged_df['CancellationDate'].notnull().astype(int)
```

```
In [298...] sns.countplot(x='Churn', data=merged_df)
plt.title('Distribution of Churned vs Active Customers')
plt.show()
```

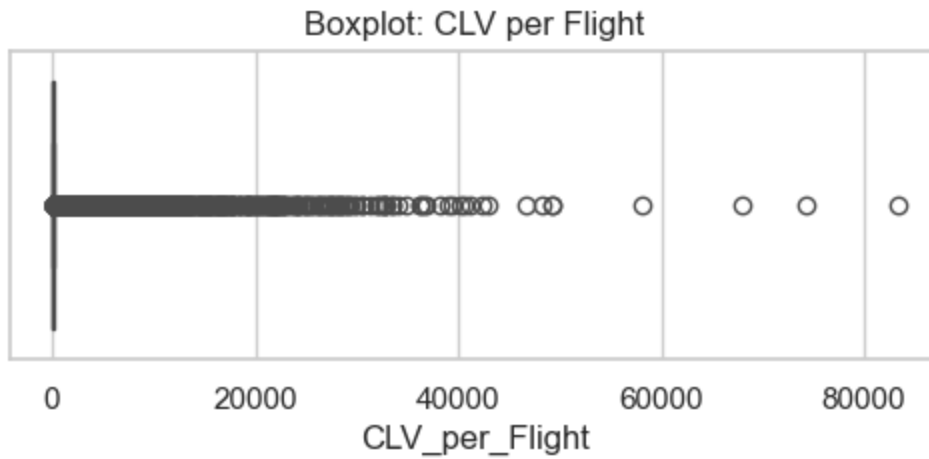


Feature: CLV_per_Flight

This variable measures how much value each flight brings to the airline for a customer. It helps identify high-value frequent flyers vs. low-value occasional ones.

```
In [294...] merged_df['CLV_per_Flight'] = merged_df['Customer Lifetime Value'] / (merged
```

```
In [296...] plt.figure(figsize=(6,2))  
sns.boxplot(x=merged_df['CLV_per_Flight'])  
plt.title('Boxplot: CLV per Flight')  
plt.show()
```



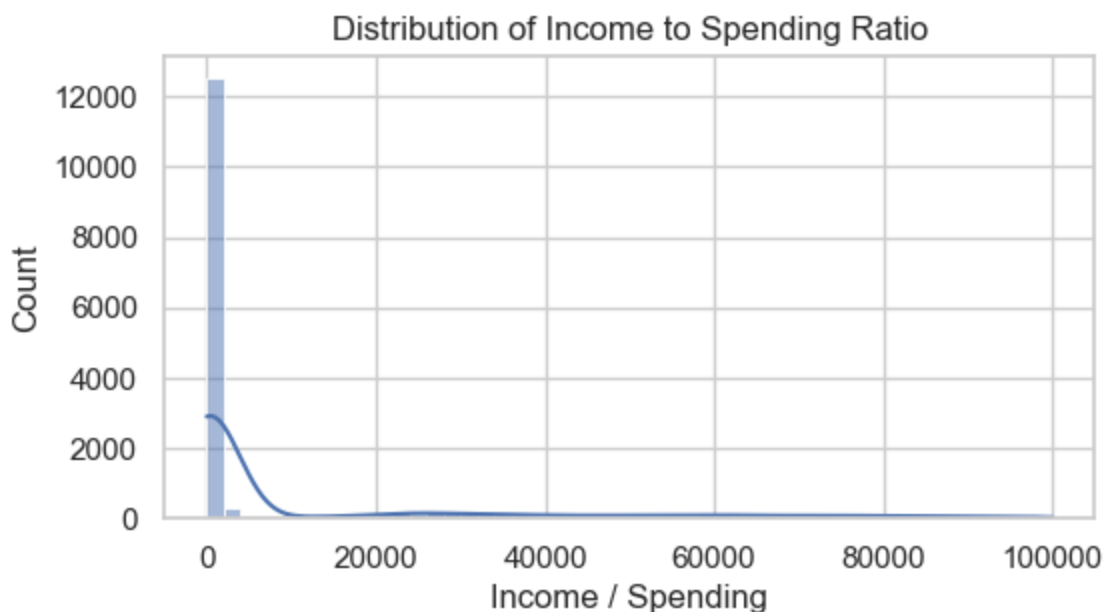
Feature: Income_to_Spending_Ratio

This feature compares a customer's income to their spending (DollarCostRedeemed), revealing financial behavior patterns:

- Higher ratios → spend little relative to income (possibly price-sensitive).
- Lower ratios → spend a lot relative to income (possibly loyal / high engagement).

```
In [299...] merged_df['Income_to_Spending_Ratio'] = merged_df['Income'] / (merged_df['TotalCostRedeemed'] + 1)
```

```
In [300...] plt.figure(figsize=(6,3))
sns.histplot(merged_df['Income_to_Spending_Ratio'], bins=50, kde=True)
plt.title('Distribution of Income to Spending Ratio')
plt.xlabel('Income / Spending')
plt.ylabel('Count')
plt.show()
```

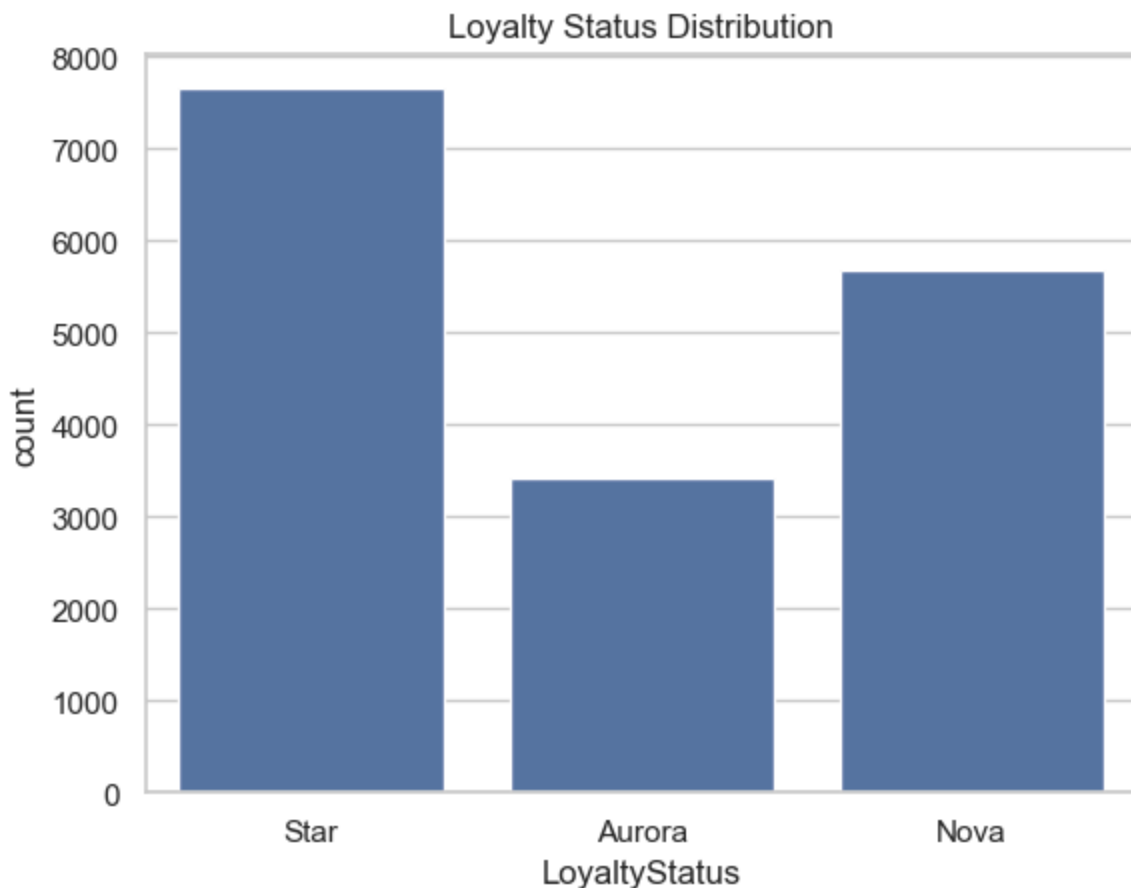


Three engineered features were developed to better describe customer value and behavior. CLV_per_Flight measures how much value each flight brings to the airline, highlighting high-value frequent flyers. Income_to_Spending_Ratio captures the balance between customer income and redeemed spending, helping differentiate cost-sensitive vs. high-engagement customers. Finally, Churn, derived from the cancellation date, identifies customers who have left the program, supporting retention analysis.

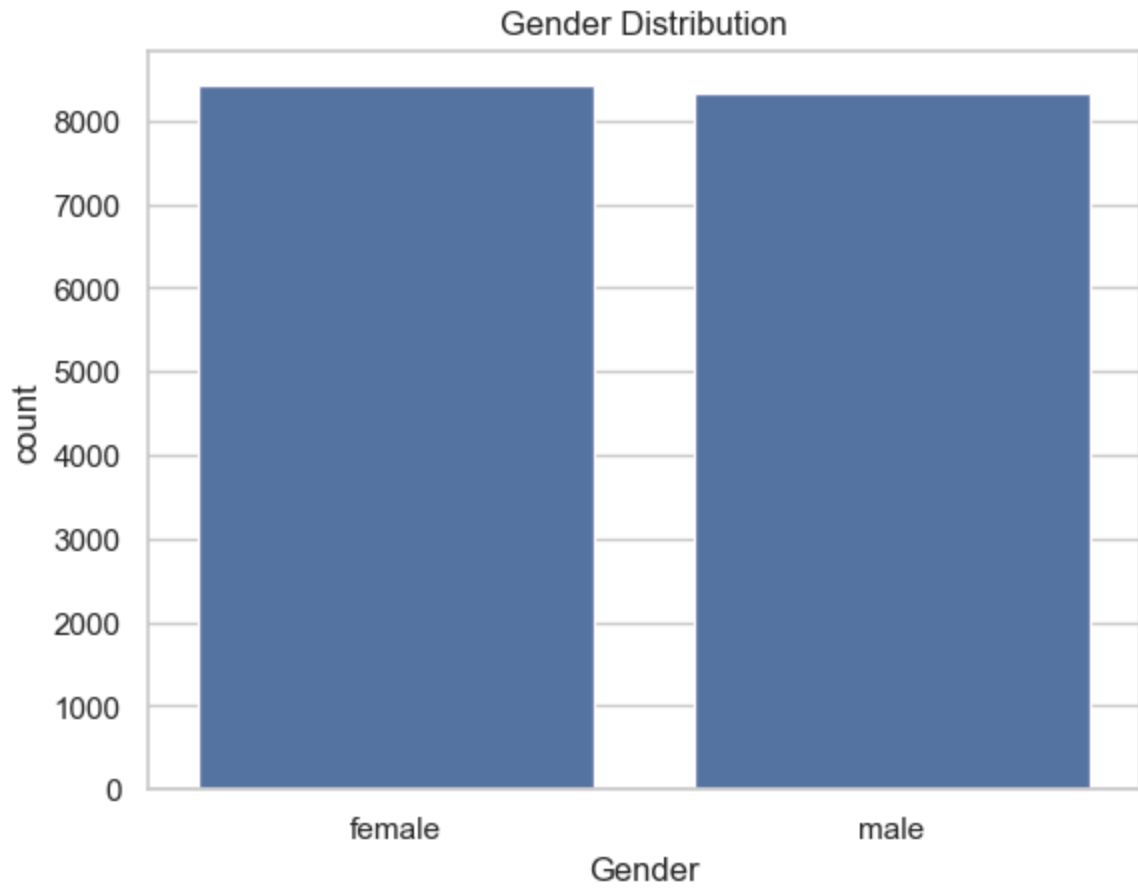
Visualisation

General Overview

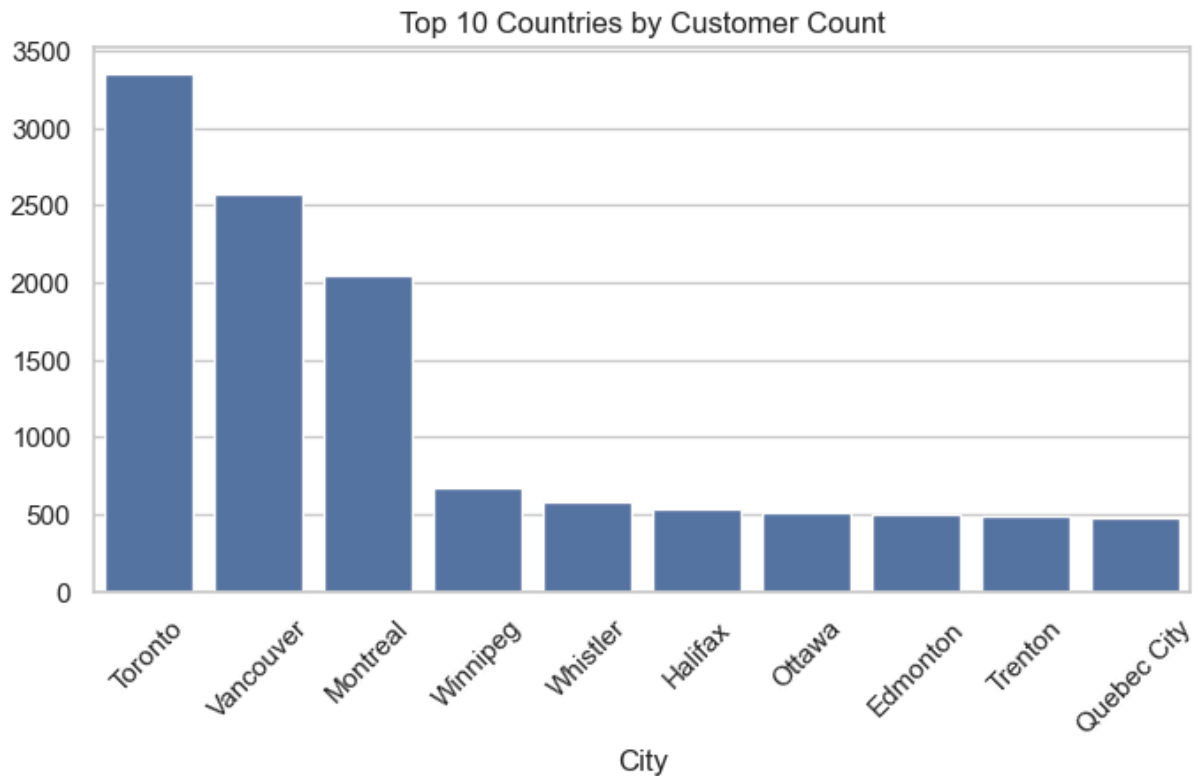
```
In [302... sns.countplot(x='LoyaltyStatus', data=merged_df)
plt.title('Loyalty Status Distribution')
plt.show()
```



```
In [305... sns.countplot(x='Gender', data=merged_df)
plt.title('Gender Distribution')
plt.show()
```



```
In [304... #Top cities
top_countries = merged_df['City'].value_counts().head(10)
plt.figure(figsize=(8,4))
sns.barplot(x=top_countries.index, y=top_countries.values)
plt.title('Top 10 Countries by Customer Count')
plt.xticks(rotation=45)
plt.show()
```

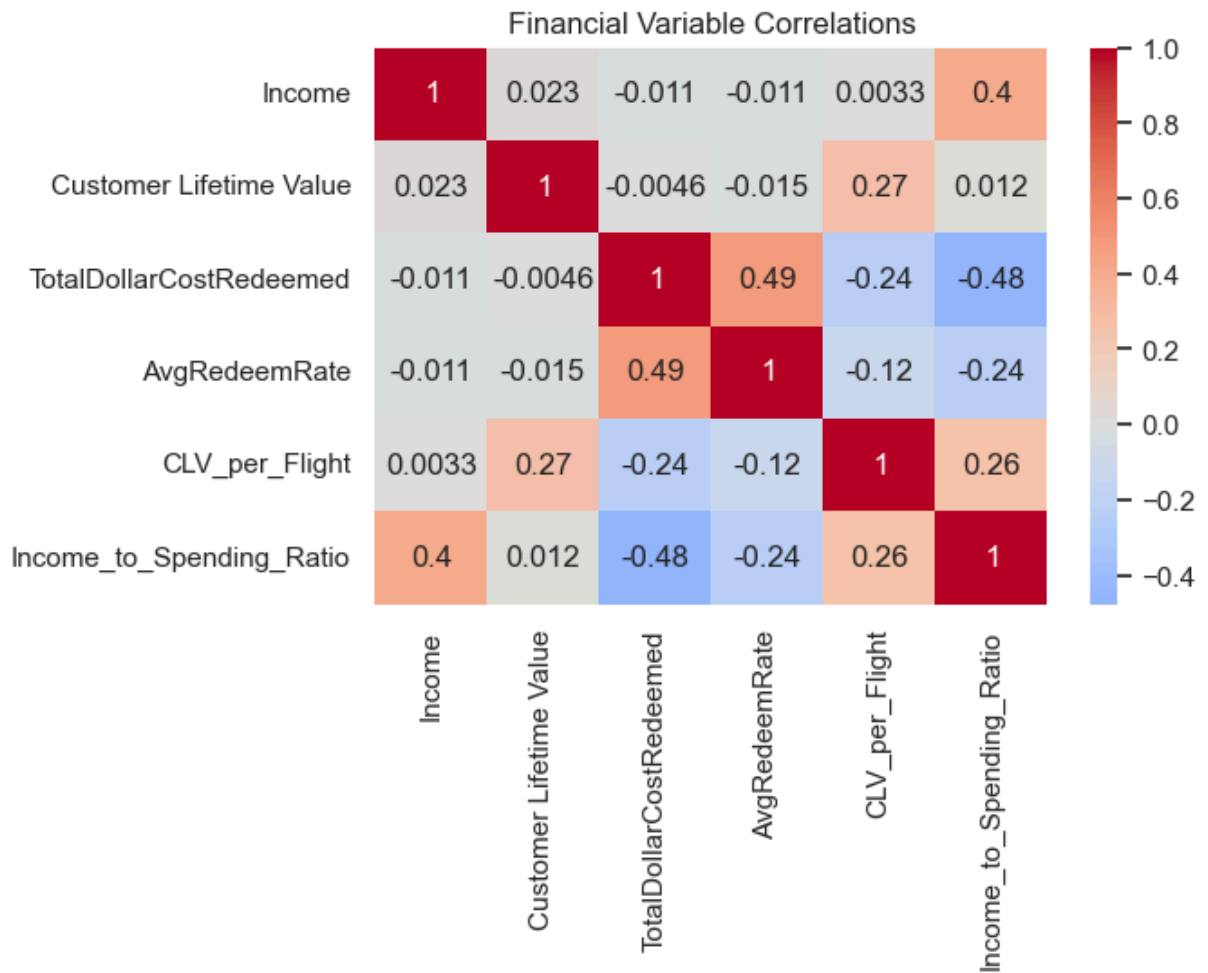


--> Most loyalty members come from Toronto. The gender distribution is balanced, suggesting no bias in participation across demographics.

Correlation Matrix

1- Financial Related Variables

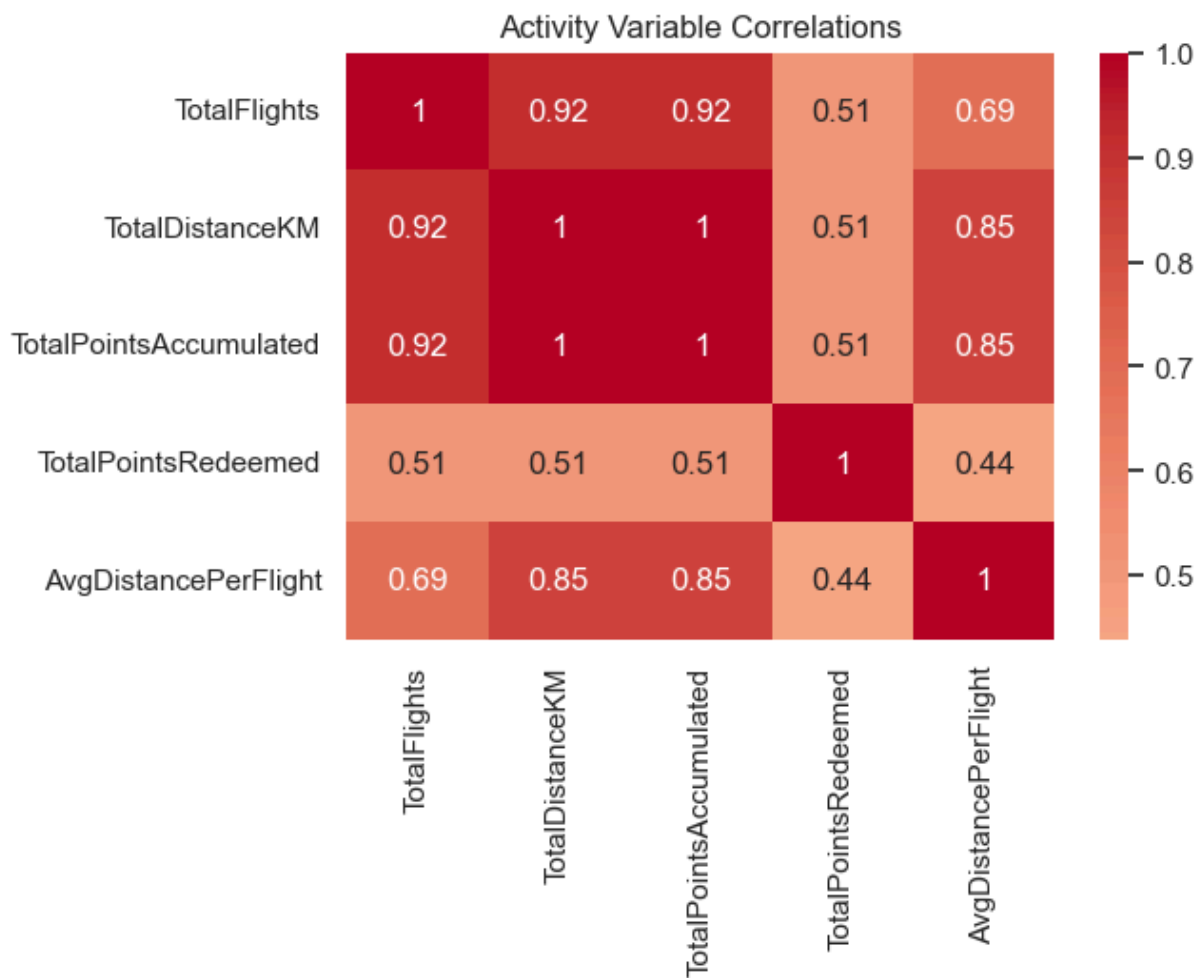
```
In [306... financial_vars = ['Income', 'Customer Lifetime Value', 'TotalDollarCostRedeem',  
                  'AvgRedeemRate', 'CLV_per_Flight', 'Income_to_Spending_Rat  
  
plt.figure(figsize=(6,4))  
sns.heatmap(merged_df[financial_vars].corr(), annot=True, cmap='coolwarm', c  
plt.title('Financial Variable Correlations')  
plt.show()
```



2-Activity Related Variables

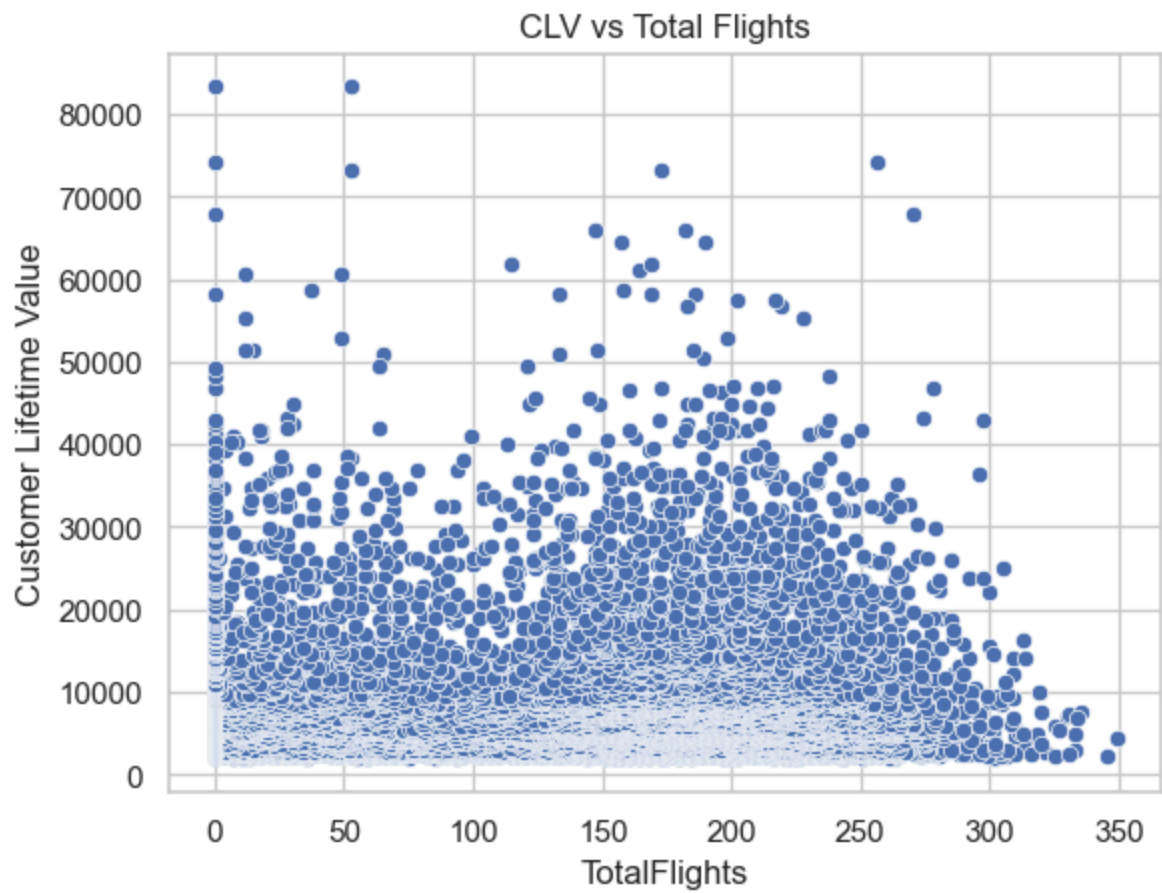
```
In [307... activity_vars = ['TotalFlights', 'TotalDistanceKM', 'TotalPointsAccumulated',
                        'TotalPointsRedeemed', 'AvgDistancePerFlight']

plt.figure(figsize=(6,4))
sns.heatmap(merged_df[activity_vars].corr(), annot=True, cmap='coolwarm', ce
plt.title('Activity Variable Correlations')
plt.show()
```

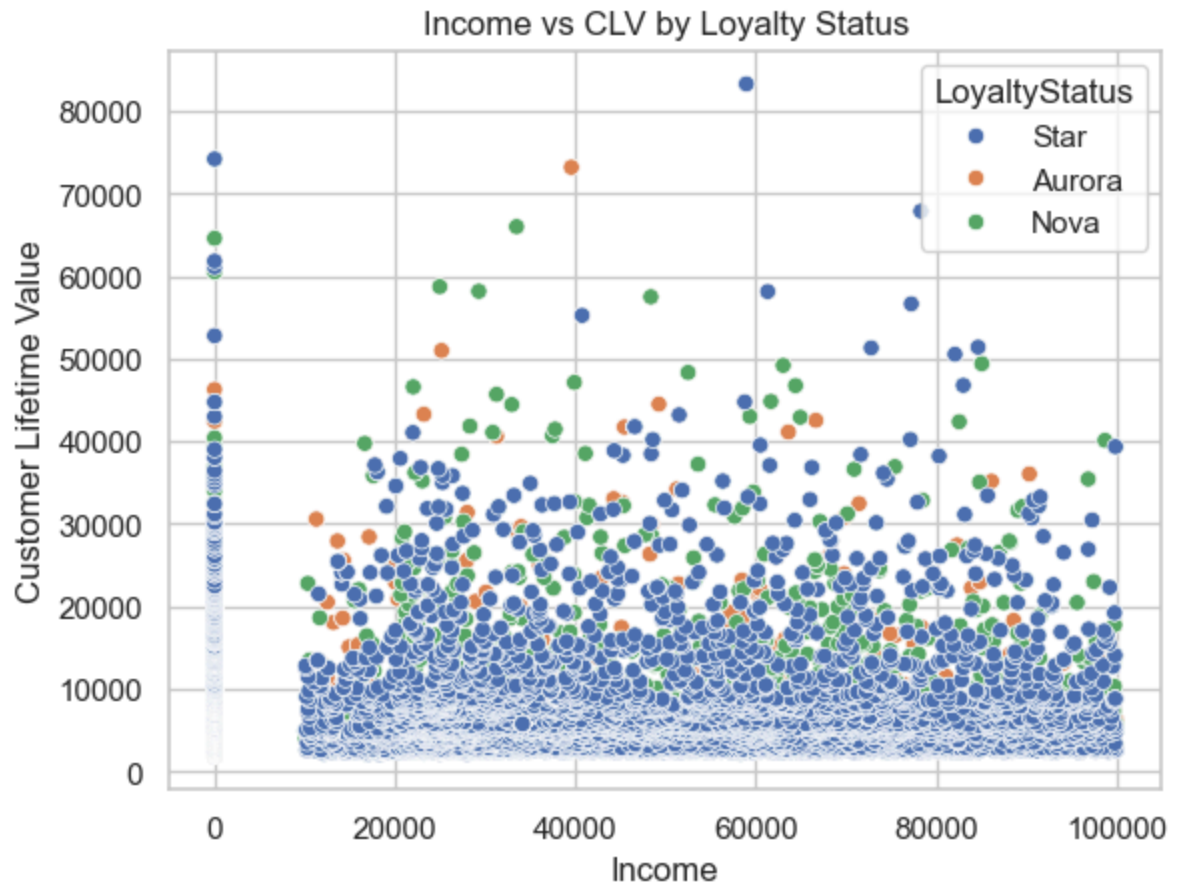


Feature Relationships

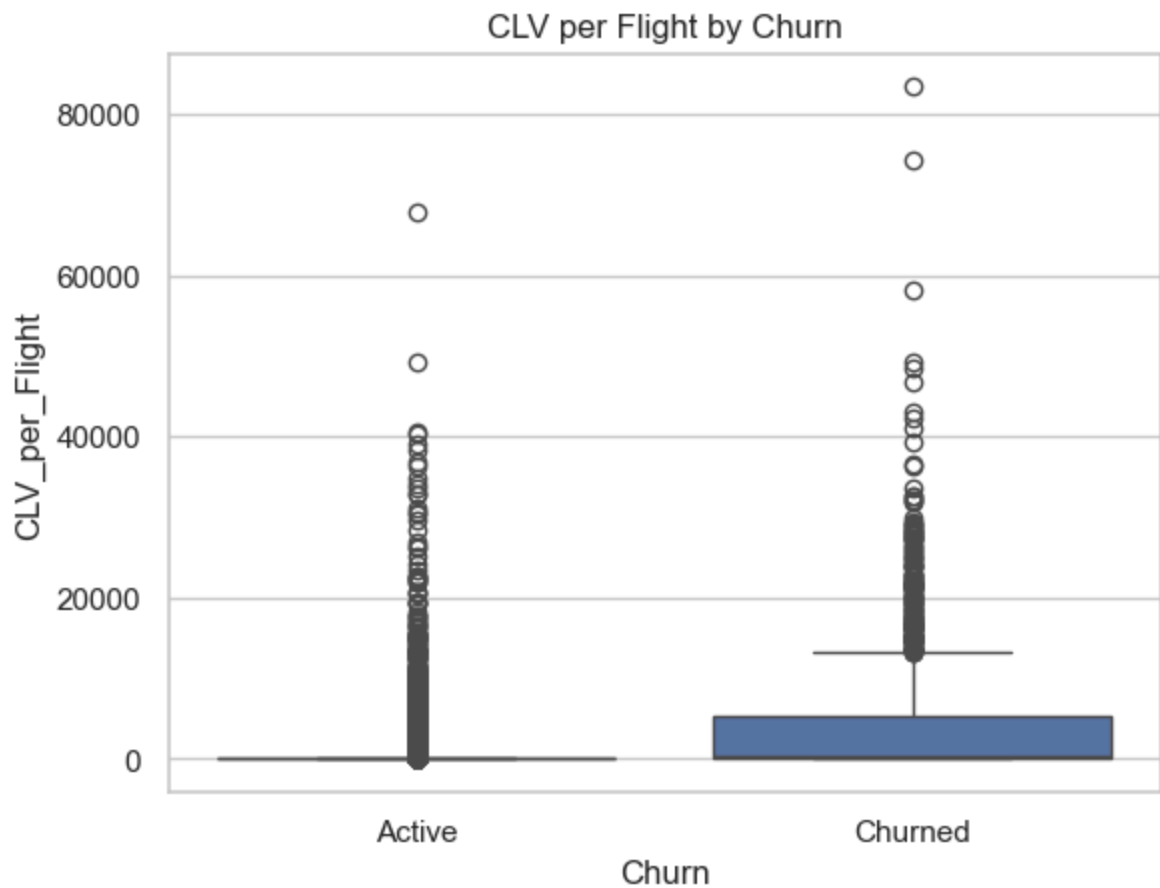
```
In [308... #CLV vs Total Flights  
sns.scatterplot(x='TotalFlights', y='Customer Lifetime Value', data=merged_c  
plt.title('CLV vs Total Flights')  
plt.show()
```



```
In [309... #Income vs CLV
sns.scatterplot(x='Income', y='Customer Lifetime Value', data=merged_df, hue
plt.title('Income vs CLV by Loyalty Status')
plt.show()
```



```
In [310... #CLV per Flight by Churn
sns.boxplot(x='Churn', y='CLV_per_Flight', data=merged_df)
plt.title('CLV per Flight by Churn')
plt.xticks([0,1], ['Active', 'Churned'])
plt.show()
```



Customers with high CLV per flight show lower churn probability, highlighting the profitability of loyal frequent flyers

In []:

In []:

In []:

In []:

In [312... *# at the end of the project we should be probably just write a conclusion to
do we think will exist later, etc*

In []: